

分类号
学校代码 10487

学号
密级

华中科技大学

硕士学位论文

(学术型 ☐ 专业型 ☒)

基于元强化学习的多类型车间及其多目标问题调度优化研究

学位申请人： 李安

学科专业： 工业工程

指导教师： 刘琼 教授

答辩日期： 2026 年 5 月 11 日

**A Thesis Submitted in Partial Fulfillment of the Requirements for
the Professional Master Degree**

**Research on Scheduling Optimization for Multi-Type
shops and Their Multi-Objective Problems Based on
Meta-Reinforcement Learning**

Candidate : LI An

Major : Industrial Engineering

Supervisor : Prof. LIU Qiong

Huazhong University of Science and Technology

Wuhan 430074, P. R. China

May, 2026

独创性声明

本人声明所呈交的学位论文是我个人在导师的指导下进行的研究工作及取得的研究成果。尽我所知，除文中已标明引用的内容外，本论文不包含任何其他人或集体已经发表或撰写过的研究成果。对本文的研究做出贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

学位论文作者签名：

日期： 年 月 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权华中科技大学可以将本学位论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本论文属于 ☐ 保 密 ☐，在 ____ 年解密后适用本授权书。
☐ 不保密 ☐。

（请在以上方框内打“√”）

学位论文作者签名：

指导教师签名：

日期： 年 月 日

日期： 年 月 日

摘要

车间调度问题是制造执行层的核心决策环节，直接影响设备利用率、生产周期与交付可靠性。随着制造业向多品种、小批量的柔性生产模式转型，不同类型的产品适合不同的生产流程，且在实际生产调度中通常需要同时优化多个相互冲突的目标。然而，现有调度方法在面对因产品更换导致的生产类型变化时，泛化能力强的通用方法通常求解效率低下或优化效果有限，而针对特定场景设计的优化方法虽然求解质量较高，却难以适应多样化的生产场景，缺乏必要的泛用性。针对上述问题，本文提出一种面向多类型车间的统一元强化学习调度框架，通过异构图对多类型车间进行统一结构化表示，结合元强化学习增加泛化能力，实现对于多类型车间问题的高效求解。本文具体内容如下：

首先，构建了面向多类型车间及其多目标问题的元强化学习求解框架。采用异构图对不同类型的车间进行结构化表示，将工序节点与机器节点及其关系编码为统一的图结构，并在此基础上构建了马尔可夫决策过程框架，定义了状态、动作、状态转移与奖励函数，为后续学习方法的设计提供了问题表示基础。

进一步地，提出了基于元强化学习的多类型车间调度方法。采用图注意力网络聚合机器节点特征，多层感知机提取工序节点特征，引入特征调控模块缓解跨任务训练中的噪声梯度干扰，并设计了包含内外循环的元训练算法与面向求解问题的适应算法。算法采用随机生成的数据集进行训练，通过在随机算例和标准算例上的测试结果，验证了所提方法的有效性与优越性。

在此基础上，提出了基于分解的多目标元强化学习方法。引入元启发式算法的分解思想，将多目标问题分解为一组权重子问题，在此基础上，设计了基于邻域动作蒸馏的元训练方法，在每个邻域中选取代表权重执行完整的内外循环元更新，邻域内其余权重通过代表策略的动作分布进行蒸馏更新，显著降低了训练开销。通过参数继承机制加速算法收敛和提高所得解在目标空间中的连续性。实验结果表明所提方法能够为多类型车间的多目标调度问题提供兼顾解集质量与计算效率的求解方案。

关键词：车间调度；元强化学习；异构图；多目标优化；多类型车间

Abstract

Shop scheduling is a core decision-making process at the manufacturing execution level, directly affecting equipment utilization, production cycle time, and delivery reliability. As manufacturing shifts toward flexible production with high product variety and small batch sizes, different product types require different process routes, and practical scheduling often needs to optimize multiple conflicting objectives simultaneously. However, when production types change due to product switching, existing approaches still face a fundamental trade-off: general-purpose methods with strong generalization usually exhibit low solving efficiency or limited optimization performance, whereas methods tailored to specific scenarios can achieve higher solution quality but are difficult to transfer across diverse production environments. To address these challenges, this thesis proposes a unified meta-reinforcement learning framework for multi-type shop scheduling. By using heterogeneous graphs to build a unified structured representation of different shop types and integrating meta-reinforcement learning to enhance generalization, the proposed framework enables efficient solving of multi-type shop scheduling problems. The main contributions are summarized as follows:

First, a meta-reinforcement learning framework is established for multi-type shops and their multi-objective scheduling problems. Heterogeneous graphs are used to represent different shop types in a unified way, where operation nodes, machine nodes, and their relations are encoded into a common graph structure. On this basis, a Markov Decision Process framework is formulated by defining states, actions, state transitions, and rewards, which provides the problem-representation foundation for subsequent learning-method design.

Second, a meta-reinforcement learning method is proposed for multi-type shop scheduling. Graph Attention Networks are employed to aggregate machine-node features, and Multi-Layer Perceptrons are adopted to extract operation-node features. A feature modulation module is introduced to mitigate noisy-gradient interference in cross-task training. In addition, a meta-training algorithm with inner and outer loops and an adaptation algorithm

for target instances are developed. The method is trained on randomly generated datasets, and tests on both random and benchmark instances verify its effectiveness and superiority.

Third, a decomposition-based multi-objective meta-reinforcement learning method is proposed. Inspired by decomposition strategies in metaheuristics, the multi-objective problem is decomposed into a set of weighted subproblems. Based on this formulation, a neighborhood action-distillation meta-training strategy is designed: in each neighborhood, a representative weight executes complete inner- and outer-loop meta-updates, while the remaining weights are updated by distilling the action distribution of the representative policy, thereby significantly reducing training overhead. A parameter inheritance mechanism is further introduced to accelerate convergence and improve the continuity of solutions in the objective space. Experimental results show that the proposed method provides an effective solution for multi-objective scheduling in multi-type shops, balancing solution-set quality and computational efficiency.

Keywords: Shop Scheduling; Meta-Reinforcement Learning; Heterogeneous Graph; Multi-Objective Optimization; Multi-Type Shop

目 录

摘 要.....	I
Abstract.....	II
主要符号对照表.....	VI
1 绪论	
1.1 课题来源.....	(1)
1.2 研究背景与意义.....	(1)
1.3 国内外研究现状.....	(3)
1.4 本文组织结构.....	(12)
2 面向多类型车间及其多目标问题的元强化学习框架	
2.1 问题描述与建模.....	(14)
2.2 元强化学习基础.....	(19)
2.3 多类型车间的元强化学习框架.....	(22)
2.4 本章小结.....	(27)
3 基于元强化学习的单目标多类型车间调度方法	
3.1 元强化学习模型的整体架构.....	(28)
3.2 基于异构图的特征提取方法.....	(29)
3.3 多类型车间问题的 MRL 算法.....	(30)
3.4 实验与结果分析.....	(38)
3.5 本章小结.....	(52)
4 基于分解的多目标多类型车间问题调度方法	
4.1 多目标优化理论.....	(54)
4.2 面向多目标的元强化学习方法.....	(55)
4.3 实验与结果分析.....	(60)
4.4 本章小结.....	(68)

5 总结与展望

5.1 研究工作总结	(69)
------------------	------

5.2 创新点归纳	(69)
-----------------	------

5.3 研究工作展望	(70)
------------------	------

致 谢	(71)
-----------	------

参考文献	(72)
------------	------

附录 1 攻读硕士学位期间取得的研究成果	(78)
----------------------------	------

主要符号对照表

J	工件集合, $J = \{J_1, J_2, \dots, J_n\}$ 。
M	机器集合, $M = \{M_1, M_2, \dots, M_m\}$ 。
n	工件总数。
m	机器总数; 在多目标分解中表示目标数量。
O_{ij}	工件 i 的第 j 道工序。
M_{ij}	工序 O_{ij} 的可选机器集合。
p_{ijk}	工序 O_{ij} 在机器 M_k 上的加工时间。
P_{ij}	工序 O_{ij} 的实际加工时间。
S_{ij}	工序 O_{ij} 的开始加工时间。
C_{ij}	工序 O_{ij} 的完工时间。
C_i	工件 i 的完工时间。
d_i	工件 i 的交货期。
x_{ijk}	二元决策变量; 若工序 O_{ij} 分配至机器 M_k 加工则取 1, 否则取 0。
$C_{\max}$	最大完工时间目标。
T_{sum}	总拖期目标。
$G = (V, C, D)$	多类型车间调度问题的异构图, 其中 V 为节点集合, C 为有向工艺约束边集, D 为工序与机器之间的无向可加工边集。
$\mu_{i,j}$	工序节点 $O_{i,j}$ 的嵌入表示。
v_k	机器节点 M_k 的嵌入表示。
θ_{meta}	元强化学习中的共享元参数。
θ_i	任务或权重子问题 i 对应的策略网络参数。
ψ_i	权重子问题 i 对应的价值网络参数。
π_{θ_i}	参数为 θ_i 的策略函数。
α	内循环学习率; 在第四章参数继承中表示继承系数 α_i 。
β	外循环(元)学习率; 在第四章中 β_{student} 表示学生更新学习率。
K	内循环梯度更新步数; 在第四章中也表示邻域大小。
$\mathcal{D}_{\text{sup}}$	内循环训练所使用的支持集轨迹数据。
$\mathcal{D}_{\text{que}}$	外循环评估所使用的查询集轨迹数据。
Λ	多目标分解生成的权重向量集合。

λ_i	第 i 个权重子问题对应的偏好向量。
λ, b	FiLM 模块生成的特征缩放与平移参数（与权重向量 λ_i 含义不同，依上下文区分）。
$\mathcal{N}(i)$	权重子问题 i 的邻域集合。
Q	权重子问题总数。
T_v	训练中执行一次验证与邻域协作判断的周期长度。
$Fr(t)$	时刻 t 的剩余工序机器柔性指数。
$H_q(t)$	时刻 t 的队列长度分布熵。
$CV(t)$	时刻 t 的工件进度系数。
HV	超体积指标，用于评价 Pareto 解集的收敛性与分布范围。
IGD	反向代际距离指标，用于评价解集逼近真实 Pareto 前沿的程度。

1 绪论

1.1 课题来源

本文研究的课题来源于国家重点研发计划项目“退役机电产品逆向物流技术及其跨组织信息集成服务平台”(项目编号为2020YFB1712900)下的子课题“面向退役工程机械的逆向物流信息集成服务平台建设与示范”,其中课题组负责基于数据挖掘的工程机械再制造智能决策与逆向物流资源动态调度优化。

1.2 研究背景与意义

1.2.1 研究背景

随着智能制造与工业互联网的持续推进,制造业的生产模式正在经历结构性变革:由过去大批量、少品种、稳定节拍的流水线生产,逐步转向多品种、小批量、高度定制化与快速换型并存的柔性制造模式^[1,2]。产品更迭速度加快,企业订单结构日趋碎片化,产品工艺路径频繁变动,生产环境日益复杂。车间调度问题作为制造执行层的核心决策环节,其本质是在一组工件、机器与时间约束构成的复杂约束网络下,合理安排工序加工顺序与机器分配方案,以最优化完工时间、机器利用率或交期满足率等绩效指标,直接决定了设备利用率、生产周期与交付可靠性,是现代制造系统效率提升的关键技术支撑^[1,3,4]。车间调度问题在数学本质上属于NP难的组合优化问题,其求解复杂度随规模扩大呈指数级增长^[5]。

然而,多类型车间之间的结构差异给现有调度方法带来了严峻挑战。根据工艺路径结构的不同,车间调度问题可以分为流水车间调度(Flow-shop Scheduling Problem, FSP)、作业车间调度(Job-shop Scheduling Problem, JSP)与柔性作业车间调度(Flexible Job-shop Scheduling Problem, FJSP)三大类。在实际生产中,不同产品根据其产品特点往往适合不同的车间类型:工艺成熟的标准件适合固定路径的流水车间模式,而新研发的定制化产品则需要灵活的作业车间乃至柔性作业车间。当企业切换生产品类时,工件的生产流程随之改变,原有调度模型往往直接失效。在求解方法的演进方面,早期的精确算法(如整数规划、分支定界法)可以保证最优解,但仅适用于小规模问题^[6,7];以遗传算法、粒子群优化、禁忌搜索为代表的元启

发式算法虽能获得近优解，然而当问题规模扩大时，其计算复杂度显著增加，导致求解时间急剧上升，难以满足对时效性的要求^[8-12]。近年来，以深度强化学习为代表的智能优化方法展现出强大的表征学习与自适应决策能力^[13-17]，但现有方法普遍针对特定车间类型构建，面对新类型车间或新规模实例时模型无法直接复用，需要消耗大量样本从头训练^[2,3]。现有方法不能主动地从多类型车间场景中提炼通用调度知识，无法实现快速迁移与泛化，这已成为实现灵活高效的多类型车间调度所需解决的关键问题。

同时，多目标优化难以统一处理是另一关键挑战。在实际调度场景中，企业不仅追求最小化最大完工时间（Makespan）这一单一目标，还需同时优化总拖期、机器负载均衡等多个相互冲突的目标，形成多目标调度优化问题。多目标问题的求解目的是获得一组权衡不同目标的 Pareto 最优解集，以便决策者从中选取适合实际需求的方案^[18,19]。传统的以加权求和将多目标转化为单目标的先验方式高度依赖权重设定；以进化算法为代表的后验方式虽能输出 Pareto 解集，但在大规模问题上求解速度较慢^[20]。基于深度强化学习的多目标方法大多针对固定权重输出单点解，难以直接生成覆盖 Pareto 前沿的完整近似解集，且通常局限于单一车间类型。如何同时处理多类型车间的调度与多目标优化，给研究带来了新的挑战。

针对上述挑战，元强化学习（Meta-Reinforcement Learning）作为元学习与强化学习的交叉方向，为构建通用调度框架提供了重要契机。其核心思想是通过在多个任务上进行元训练，增强模型的泛化能力，在面对求解问题时通过少量样本即可快速适应^[21-23]。这一特性与多类型车间调度的实际需求契合：将不同车间类型视为不同任务，元训练阶段覆盖多种车间场景以积累跨类型通用调度知识，元测试阶段对新车间类型或新实例进行快速适配；同时，结合基于分解的多目标优化框架，可以在元学习框架内实现 Pareto 近似解集的高效生成^[24-26]。将元强化学习思想与面向多类型车间的建模相结合，可以为解决多类型车间调度问题提供一个有效的途径。

1.2.2 研究意义

理论上，本文构建了面向 FSP、JSP、FJSP 等多类型车间调度问题的元强化学习框架。通过引入带注意力机制的异构图神经网络对多类型车间的异构结构进行统一表征，通过元强化学习同时学习不同类型的任务并提取调度知识。在多目标优化方面，提出了基于分解思想的元强化学习多目标调度方法，通过将多目标问题分解为

一系列带权的子问题并以端到端学习方式近似求解 Pareto 最优解集，拓展了元强化学习在组合优化与多目标决策领域的理论适用边界。

所提出的统一调度框架能够在不修改网络结构的前提下处理 FSP、JSP、FJSP 等多种车间类型。当企业面临产品品类切换或工艺调整时，无需重新设计调度模型与训练流程，仅需在新车间数据上进行少量微调即可实现高效适配，可将调度系统的适配周期从传统方法的数周缩短至数小时，大幅减少模型重复开发的人力与时间成本。同时，提出的多目标方法能够在可接受时间内输出一组质量较好的 Pareto 近似解集，缩短了调度决策的响应时间，使决策者可以快速获得兼顾完工时间与交付准时率的多样化调度方案，提升了制造系统在复杂多变产品需求下的敏捷响应与高效运营能力。

1.3 国内外研究现状

针对车间调度问题的求解，国内外学者从不同角度开展了大量研究，主要涵盖以传统精确法、启发式与元启发式算法为代表的传统调度优化方法，以强化学习与深度强化学习为代表的机器学习调度方法，以元强化学习为代表的跨任务快速适应方法，以及针对多目标车间调度优化的各类方法。以下分别从上述四个方向梳理国内外代表性研究进展。

1.3.1 传统车间调度优化方法

早期的车间调度研究以精确方法为主，包括线性规划、整数规划、分支定界法和动态规划。这类方法在理论上能够保证找到全局最优解，但其计算复杂度随问题规模呈指数级增长，在实际应用中仅适合求解小规模实例。Portmann 和 Vignier 等人^[6]将遗传算法与分支定界法相结合，在保留精确方法最优性保证的同时利用遗传算法加速搜索过程，有效提升了对中小规模车间调度问题的求解效率。Neron 等人^[7]运用分支定界法并融合能量推理（Energetic Reasoning）与全局操作（Global Operations）技术，针对最大完工时间最小化目标进行求解，有效拓展了精确方法在中等规模实例上的适用边界。然而，随着工件数与机器数的增加，精确方法的计算代价迅速变得昂贵，因此学者们转而研究近似求解方法。

上世纪 80 年代后，启发式规则成为工业调度的主流手段。启发式规则基于问

题的特定规律与工程经验,能够快速得到满意近似解,适合实时性要求高的生产现场。Gupta 和 Tunc^[27]提出了四种针对含设置时间与移除时间的多阶段车间调度求解最大完工时间的启发式规则,并通过实验对比验证了各规则的适用范围。Lee 和 Vairaktarakis^[28]设计了面向多机环境下最大完工时间最小化的启发式方法,并给出了算法的近似比分析。在多机环境优先规则研究方面,Hunsucker 和 Shah^[29]系统对比了多种优先级规则在约束多处理机环境下的性能表现,为工业调度中优先规则的选择提供了参考依据。另外,越民义与韩继业^[5]早期对 n 个零件在 m 台机床上的加工顺序问题进行了深入的理论分析,奠定了国内调度理论研究的基础。方子丞等人^[30]则针对多车间协同场景下的调度问题,提出了带负载均衡约束的混合算法,在大规模场景下验证了元启发式方法处理复杂约束的有效性。

尽管启发式规则具有高效、易于实现的优势,但其解质量高度依赖于规则设计与问题特性,难以保证全局最优性,且在面对多类型车间或复杂约束时泛化能力不足。为此,元启发式算法(Metaheuristic)以其更好的全局搜索与解质量成为主流研究方向。Liao 等人^[8]将粒子群优化算法(PSO)与瓶颈启发式(Bottleneck Heuristic)混合,用于求解最大完工时间最小化目标,在多类标准算例上取得了优异的近优解质量。Engin 和 Döyen^[9]提出了基于人工免疫系统(Artificial Immune System, AIS)的车间调度算法,通过克隆选择与变异机制实现高效解空间搜索,并验证了其在最小化完工时间上的竞争力。Zandieh 等人^[10]在免疫算法框架下进一步考虑了带序列相关设置时间的车间场景,设计了适应性免疫操作策略,有效提高了复杂约束下的求解质量。Yang 等人^[11]将禁忌搜索(Tabu Search, TS)与仿真优化相结合,针对含多处理机的车间以最大完工时间为目标进行优化,在实际工厂案例验证中取得了良好效果。Wang 等人^[12]提出了基于模拟退火(Simulated Annealing, SA)的车间调度方法,专门处理含多处理机任务场景下的最大完工时间最小化问题,通过合理的邻域操作设计实现了高质量解的稳定收敛。Tseng 和 Liao^[31]也提出了针对含多处理机任务的 PSO 调度算法,并在多个标准测试实例上与其他方法进行了对比验证。

然而,元启发式算法的求解需要大量迭代,计算成本随着规模的变大而提高,不能满足及时调度的需求^[2,3]。同时上述各类传统方法均为单次求解方法,无法从历史经验中积累调度知识,在面对多类型车间时缺乏知识迁移能力。

1.3.2 基于机器学习的调度优化方法

以深度学习与强化学习为代表的机器学习技术的兴起，为车间调度问题的求解提供了新思路。与传统方法依赖人工规则或迭代搜索不同，机器学习方法通过从大量历史数据或交互经验中自动学习调度规律，能够构建出具有一定泛化能力的调度策略模型^[13,32]。

强化学习将调度问题建模为马尔可夫决策过程 (Markov Decision Process, MDP)，通过智能体与环境的持续交互来学习优化调度策略，在处理序列决策场景时具有明显优势^[13]。早期的强化学习调度研究以表格型 Q 学习为主。Naimi 等人^[14] 提出了面向车间调度的多目标 Q 学习方法，利用智能体在仿真环境中的交互学习，自动选择最优的调度策略以同时优化生产效率与能耗目标，实现了对复杂多目标调度规律的自动提取。Zhao 等人^[33] 将深度 Q 网络 (DQN) 应用于作业车间调度，设计了融合工时层、结果层等多维信息的状态表示方案，并以十种经典启发式调度规则作为动作空间，通过 DQN 自适应选择最佳规则，在多组标准基准实例上取得了优于单一规则 and 传统 Q 学习的调度质量与通用性。在多智能体协同方面，Pu 等人^[34] 提出了基于图神经网络的分布式多智能体强化学习架构，通过 Actor-Critic 框架实现多智能体协同决策，在大规模调度任务与绿色制造目标优化中均表现出优于单智能体方法的性能。Zhang 等人^[35] 提出了基于注意力机制与析取图嵌入的协作智能体强化学习框架 (CARL)，将工序选择与机器分配建模为多智能体协同决策过程，通过注意力机制捕捉工序间的全局依赖关系，在多个标准基准测试集上取得了领先的调度质量。

在强化学习与传统近似算法的混合方面，Luo 等人^[36] 提出了深度多智能体强化学习方法，通过集中式训练与分散式执行框架实现实时调度决策，在生产扰动频发的场景下展现出优于传统方法的响应速度与解质量，为强化学习在大规模车间中的实时部署提供了可行路径。景轩等人^[37] 基于图卷积网络设计了多智能体深度强化学习调度系统，通过集中式学习与分散式执行的多智能体框架实现了工序选择与机器分配的协同优化，在多个标准测试集上验证了方法的有效性。Chen 等人^[38] 提出了基于强化学习的自学习遗传算法 (SLGA)，通过 Q 学习自动调节遗传算子的选择与参数配置，在最小化最大完工时间的标准测试集上验证了混合策略的有效性。Long 等人^[39] 将 Q 学习嵌入人工蜂群算法 (Artificial Bee Colony, ABC) 中，构建自学习 ABC 算法 (SLABC)，通过强化学习自动调整搜索策略与邻域操作，明显改善

了 ABC 在车间调度上的收敛速度与解质量。

随着深度学习技术的成熟，深度强化学习（Deep Reinforcement Learning, DRL）将神经网络强大的表征能力引入强化学习框架，大幅拓展了其在高维、复杂状态空间下的决策能力^[40-43]。在深度强化学习调度研究中，车间状态的有效表示是影响策略学习效果的关键因素之一。Han 等人^[15]通过构建工时层、结果层与机床层三个矩阵来表征车间调度环境的状态，结合对抗双 DQN（Dueling Double DQN）算法验证了矩阵状态表示在 JSP 上的求解能力，但矩阵表示对工件动态插入类问题处理能力有限。Christiano 等人^[44]提出了基于人类偏好的深度强化学习，通过人类反馈代替精确奖励函数来引导策略优化，这一奖励设计思路对调度问题中奖励构造困难的问题具有启示意义。

肖鹏飞等人^[45]自定义了 15 个表达制造过程状态的特征值，结合启发式算法构造动作空间，使用深度时序差分强化学习算法训练车间调度模型，在标准测试集上验证了该特征工程方法的灵活性与适应性。李宝帅和叶春明^[46]针对最小化最大完工时间目标，以平均机器利用率为奖励函数，通过深度强化学习求解作业车间调度问题，在标准基准实验中取得了良好结果。Luo^[16]设计了 7 种调度规则作为 DQN 的动作空间，将车间调度求解建模为深度强化学习问题，并利用 Double DQN 与软目标权重更新技术稳定训练过程，构造了融合延迟率与机器利用率的奖励函数，取得了优于传统规则方法的调度质量。Zhao 和 Zhang^[47]将深度强化学习引入车间生产控制系统，提出了基于 DRL 与调度规则融合的车间调度方法，验证了深度强化学习在生产控制场景下的实用性。

在端到端深度强化学习方法方面，Wang 等人^[48]提出了基于 D3QN（Dueling Double DQN）算法的大规模车间实时调度框架，通过多优先级网络表示车间状态，解决了大规模制造车间的实时调度问题，显示了 DRL 在工业级场景下的规模化应用潜力。谭远良等人^[49]提出了改进的 Actor-Critic 深度强化学习算法，通过设计感知加工优先级的状态表示与奖励机制，在实际工程场景中验证了 DRL 对复杂排序问题的适用性。

图神经网络（Graph Neural Network, GNN）与强化学习的结合是近年来调度领域的重要进展之一。Graph 形式的状态表示能够自然刻画工序依赖关系与机器资源约束的结构，并具备良好的规模泛化能力。Park 等人^[17]将 JSP 表示为析取图（Disjunctive Graph），设计了 GNN-RL 框架，通过图神经网络提取工序节点与机器

节点的图嵌入, 实现了调度策略的端到端学习, 在 Taillard 等多个标准基准测试集上显示了较强的泛化性。Song 等人^[50] 将图神经网络与深度强化学习相结合, 设计了面向 FJSP 的端到端调度方法, 通过异构图编工序与机器之间的关系, 验证了 GNN-DRL 在柔性车间场景下的高效性与泛化优势。Wang 等人^[51] 提出了基于双注意力网络 (Dual Attention Network) 的强化学习调度方法, 通过对工序与机器之间的注意力关系进行建模, 同时决策工序选择与机器分配, 在多个基准测试集上超越了当时主流方法。Du 等人^[52] 进一步拓展了车间调度的建模范围, 在包含运输与准备时间的扩展场景下设计了强化学习调度方法, 在真实工业约束下验证了 RL 调度系统的工程可行性。

然而, 上述基于深度强化学习的调度方法在面对多类型车间问题时存在以下不足: 其模型设计通常针对单一车间类型, 状态表征、网络等均围绕特定场景构建, 缺乏对多类型调度问题的统一考量。这导致模型能力固化于训练场景, 一旦车间类型改变, 原有模型的性能会显著下降甚至完全失效。面对新场景时, 企业需要耗费大量计算与时间成本重新训练^[2,53]。

1.3.3 基于元强化学习的相关方法

元学习 (Meta-Learning) 是机器学习领域中一个新的研究方向^[54]。其核心原理是通过与大量多样化任务的交互经验, 学习一种通用的学习策略, 使模型在面对新任务时仅需少量样本即可快速适应^[21,22]。元学习方法已被成功应用于自然语言处理、图像处理、机器人控制等多种场景^[55-57], 有效提升了模型在新任务上的学习速度与泛化能力。

元学习与深度强化学习的结合即形成元强化学习 (Meta-Reinforcement Learning, Meta-RL), 其本质是一个双层优化过程: 内层优化负责利用少量样本对基学习器进行快速适配, 外层优化则基于多任务性能跨任务调整元知识, 使模型获得面对新任务快速泛化的能力^[21-23]。

Finn 等人^[21] 提出了与模型和任务无关的元强化学习算法 MAML (Model-Agnostic Meta-Learning), 利用梯度信息对模型参数进行元训练, 在面对新任务时仅需少步梯度更新即可实现高效适应, 奠定了基于梯度的元学习方法的基础。Melo^[58] 提出了基于 Transformer 架构的情景记忆元强化学习方法, 通过近期工作记忆的递归构建实现任务上下文的在线推断, 进一步增强了智能体对任务分布的跨场景泛化能

力。黄宁馨等人^[59]结合元学习对深度强化学习算法进行改进,提出了一种结合元学习先验知识的改进 DRL 方法,在加速新任务学习速度方面取得了良好效果。邓天翊和张耕培^[60]研究了基于元学习与数据增强相结合的小样本泛化方法,验证了元学习框架在提升模型面对小样本新任务时泛化性能上的明显优势。元强化学习在车间调度领域的应用较少,但是其强泛化能力已在机器人控制、自动驾驶、能源与边缘计算等多个序列决策领域得到广泛验证,为其引入车间调度场景提供了理论基础。在机器人控制领域,Jin 等人^[61]设计了基于多任务强化学习的无人水面艇软编队控制方法,证明了元强化学习在多场景环境扰动下的有效控制能力;Belmonte-Baeza 等人^[25]提出了无模型元强化学习方法,使四足机器人能够在多种地形场景下快速适配运动策略,验证了元强化学习在连续控制空间中的快速迁移能力;Shin 等人^[26]通过元学习策略与模型预测控制的动态切换机制,使移动机器人在动态环境中兼顾稳健性与灵活性;Lee 等人^[62]将引导式元强化学习引入自动驾驶策略学习,通过多样化场景元训练使策略对未知道路环境展现出更强的鲁棒适应性。上述研究表明元强化学习在面对多场景的问题时可以提供有效的解决方案。值得注意的是,Jia 等人^[24]将船舶避碰问题建模为马尔可夫决策过程,将基于梯度的元强化学习框架应用其中,内循环积累不同海洋场景的避碰知识,外循环优化元策略初始参数,最终模型在未知海域展现出卓越的适应性与安全性。这一框架与车间调度问题的 MDP 建模在结构上高度一致,同样是面对多类型任务分布与快速适应需求。上述文献充分说明了元强化学习从控制领域向调度领域迁移的方法论可行性。在能源与边缘计算等调度领域,元强化学习已取得有价值的工程验证。沈欢^[63]提出的 MAML-SAC 算法以少量数据快速适应不同微电网环境,证明了元强化学习在小样本资源调度场景下的实用价值。王志强^[64]面向任务类型异构的边缘计算环境在 MAML 框架基础上提出 TayMAML 算法,通过偏向性抽样策略与泰勒展开一阶梯度估计大幅降低元更新开销,有效提升了模型对异构任务的泛化能力。这些研究为元强化学习在车间调度的应用提供了参考。元强化学习在上述领域的应用验证了其处理多类型任务分布的通用能力,该能力与多类型车间调度问题的需求吻合。同时近两年已有研究将元强化学习引入车间调度并取得成果:马瑞琳 (2025)^[65]将 MAML 与深度 Q 网络相结合,在标准数据集上验证了该方法在调度质量与适应速度上均优于传统启发式算法与深度强化学习基线,为元强化学习在车间调度场景的可行性提供了实验支撑;吴林聪

(2025)^[66] 针对包含机器故障、紧急插单等动态扰动的 DFJSP，提出 MAML-PPO 框架，MAML 的二阶梯度优化机制赋予策略参数快速适应新场景的能力，PPO 的信赖域约束保障了策略更新的稳定性，实验结果在跨车间场景泛化能力与动态事件响应能力上均明显优于基准。这些研究共同表明，元强化学习在车间调度中具备切实可行的理论基础。谭晓阳和张哲^[23] 对元强化学习进行了系统综述，梳理了基于优化、基于模型与基于记忆的三大类方法的理论脉络，指出元强化学习在减少训练样本与提升新任务适应性方面前景良好。Beck 等人^[22] 从更宏观视角对元 RL 方法进行了全面调研，深入分析了各类方法的分类特点与未来挑战。综合来看，深度强化学习在调度领域普遍面临样本效率低、收敛慢、跨任务泛化能力不足等核心挑战^[65,66]，而元强化学习通过构建基于任务分布的元知识库，提高策略参数的泛化能力，其以 MAML 为代表的基于梯度的方法从机制上契合了多类型车间场景快速切换的需求，是克服上述瓶颈的有效途径^[22,23]。

尽管上述研究已分别在微电网能源调度、边缘计算任务调度、作业车间调度和柔性车间调度等场景中验证了元强化学习的可行性与优势，但这些工作均局限于特定的单一调度场景^[2,4]。虽然元强化学习在其他调度领域展现出应用潜力，也有研究开始探索其在单一类型车间中的应用，但尚未有研究将其拓展至多类型车间问题。尽管从方法上看，将多类型车间的数据混合训练有望提取更具泛化能力的元知识，但由于不同车间在结构约束、状态空间与决策上存在差异，如何在多类型的车间任务分布下设计稳定的元训练机制，仍是一个亟待深入研究的关键问题。

1.3.4 多目标车间调度优化方法

在实际生产调度中，决策者往往需要在多个相互冲突的目标之间进行权衡，形成多目标调度优化问题。多目标优化的目标是求出一组 Pareto 最优解集，由决策者从中依据实际需求选择合适方案。

在传统多目标进化算法方面，张青富提出的基于分解的多目标进化算法 (MOEA/D)^[18] 通过将多目标问题分解为一系列带权的单目标子问题，利用相邻子问题之间的信息进行协作搜索，在众多多目标基准测试问题上显示出优异的求解性能，成为多目标优化领域最具影响力的算法之一。Deb 等人^[19] 提出的非支配排序遗传算法 NSGA-II (Non-dominated Sorting Genetic Algorithm II) 引入精英策略与拥挤度计算，兼顾 Pareto 前沿的收敛性与多样性，在 FJSP 等多目标调度问题中得到了广泛应

用，被各类改进方法作为基准。

在改进多目标进化算法方面，Xiao 等人^[20]提出了针对多目标柔性作业车间调度的改进 MOEA/D 算法，综合考虑生产效率与成本两类目标，引入混合初始化与局部强化搜索策略，在多组标准测试实例上显示出优于 MOEA/D-DRA、NSGA-III 等算法的收敛性与多样性。姜一啸等人^[67]通过 Tent 混沌映射与融合 AHP 的贪婪解码策略形成质量较高的初始种群，并对 NSGA-II 进行改进，求解多目标柔性作业车间低碳调度问题，在多组标准测试例上验证了改进方法在低碳多目标场景下的优势。Sassi 等人^[68]提出了基于分解的多目标人工蜂群（ABC）算法，以最大完工时间、总机器负荷与最大机器负荷为目标，在多目标 FJSP 上验证了解析思想下 ABC 算法的多目标求解能力。Chen 等人^[69]提出了结合变邻域搜索与 Q 学习参数自适应的增强型多目标进化算法 Q-MOEA/D，面向柔性作业车间的节能调度问题，在保证 Pareto 解集质量的同时显著提升了算法的收敛效率。

大规模多目标进化算法的主要局限在于求解速度随问题规模增大而明显下降，且 Pareto 前沿的质量受参数设置影响较大，工程部署存在一定门槛。为克服这一局限，机器学习技术被引入多目标调度优化中以提升求解效率。Li 等人^[70]提出了基于强化学习的 RMOEA/D 算法，针对模糊完工时间与总机器负荷的双目标 FJSP，利用强化学习自适应引导种群个体选择最佳参数组合，并指导变邻域搜索选择高成功率的局部搜索策略，在保证 Pareto 解集质量的同时，提升了算法的收敛速度与鲁棒性。

在端到端深度强化学习多目标方法方面，Jing 等人^[71]将图卷积网络与多智能体强化学习相结合，通过集中学习与分散执行的框架求解多目标柔性作业车间调度问题，利用图结构表示实现了对工序与机器关系的高效编码，在多个 FJSP 基准上达到了当时最优水平。张丽丛^[72]结合 MOEA/D 的分解思想，将多目标调度优化问题分解为多个单目标子问题，再将每个子问题建模为神经网络决策任务，通过神经网络的前向传播实现调度决策，从而输出覆盖 Pareto 前沿的多样化近似最优解集。张韵^[73]将多目标调度问题拆解为单目标优化与多目标协调两个层次，设计了双层深度强化学习模型，上层负责多目标权重的自适应协调，下层负责单目标调度策略的学习与执行，两层协同工作实现了更高质量的多目标 Pareto 解集近似。

然而，基于端到端学习的多目标调度方法目前仍面临着局限。一方面，传统多目标进化算法虽能输出 Pareto 解集，但需要同时维护多样化的种群并频繁评估解之间的 Pareto 支配关系，在大规模问题上求解速度随问题维度显著下降；另一方面，基

于深度强化学习的多目标方法通常只能在固定权重下输出单点解，难以直接输出覆盖 Pareto 前沿的完整近似解集，而每次权重改变均需重新训练模型，同时，上述方法大多仍局限于特定车间类型，在多类型车间结构下同时求解多目标调度问题的相关研究少，亟需设计能够在统一框架下兼顾多类型车间通用性与多目标 Pareto 解集高效生成的新型方法。

1.3.5 研究现状总结与本文主要内容

综合以上文献梳理，当前车间调度研究存在以下问题：

(1) 传统调度方法的启发式方法对全局信息的利用不充分，解的质量差；精确算法与元启发式算法的较优解依赖大量计算或迭代。基于机器学习的调度方法可以在求解速度与解质量之间实现更优的平衡，但其现有研究通常针对单一车间类型进行设计，面对包含多种类型的调度场景时，模型的泛化与适应能力不足。

(2) 状态表征是调度策略学习的基础，其质量直接影响决策模型的性能。尽管异构图神经网络已在单一类型车间的调度中得到应用，能够通过工序与机器节点的异构拓扑有效捕捉特定车间结构下的约束关系，但尚未有研究验证基于异构图的状态提取方法在多类型车间场景中的有效性，以及其输出能否支持后续学习的训练。

(3) 传统多目标算法的计算成本随问题规模的增加的幅度大于单目标优化算法；深度强化学习中基于分层的多目标方法只能输出单一偏好的解，无法快速生成覆盖 Pareto 前沿的多样化解集，且仍局限于单一车间类型。

针对上述不足，本文着力于开展以下研究：

(1) 构建基于元强化学习的多类型车间调度方法，将调度问题建模为 MDP。在此基础上，设计 MRL 元强化学习训练框架，通过在覆盖多种类型车间实例的任务分布上进行双层元优化，再通过适应算法快速获得新调度问题的策略。并设计特征调控模块以进一步提升元训练过程中的收敛稳定性。

(2) 多类型车间调度问题建模与统一表征。针对不同类型的车间调度问题，建立统一的数学模型，提出基于异构图对多类型车间进行统一结构化表示的方式。异构图中包含工序节点与机器节点两类实体，以先后序、可加工与已分配三类关系边描述调度过程的结构信息与动态演化，设计带注意力机制的异构图神经网络提取机器与工序的图嵌入特征。

(3) 将多目标优化引入元强化学习中。将调度问题拓展为同时最小化完工时间

与总拖期的双目标优化问题，引入分解的思想，将多目标问题分解为一系列均匀分布在权重空间上的单目标子问题。结合元强化学习训练机制，设计面向不同目标权重的调度策略参数化方案，使单一模型能够在多权重设置下输出覆盖 Pareto 前沿的多样化解集。

1.4 本文组织结构

本文共分为五章，各章主要内容如下。

第一章为绪论，介绍课题研究背景与意义，系统综述传统车间调度方法、基于机器学习的调度方法、元强化学习及多目标车间调度优化的国内外研究现状，归纳现有方法存在的局限与不足，提出研究问题，并给出全文的主要研究内容与组织结构。

第二章为面向多类型车间及其多目标问题的元强化学习框架。对三类车间调度问题进行统一建模，建立数学规划模型，设计以工序节点与机器节点为核心的异构图表示方案，并在此基础上构建适用于多类型车间统一求解的元强化学习框架，详细说明了马尔可夫决策过程的状态，动作，奖励等要素，为第三章和第四章的方法设计奠定理论基础。

第三章为基于元强化学习的单目标多类型车间调度方法。以最小化最大完工时间为目标，通过带注意力机制的异构图和多层感知机提取图特征，用于后续训练。实现元强化学习在多类型车间问题上的训练和适应算法，引入特征调控模块降低训练过程中的噪声，并在多类基准测试集上进行系统实验，验证方法的有效性与跨类型泛化能力。

第四章为基于分解的多目标多类型车间调度方法。在单目标框架基础上引入多目标优化扩展，以最大完工时间与总拖期为双优化目标，基于分解的思想构造权重子问题，通过邻域知识蒸馏和参数继承加快训练收敛和提高解集的连续性，并通过实验验证其在多目标调度场景下的性能优势。

第五章为总结与展望。对全文的研究工作进行系统总结，归纳全文的主要创新点，并对未来可能的研究方向进行展望。

本文的组织结构如图1.1所示。

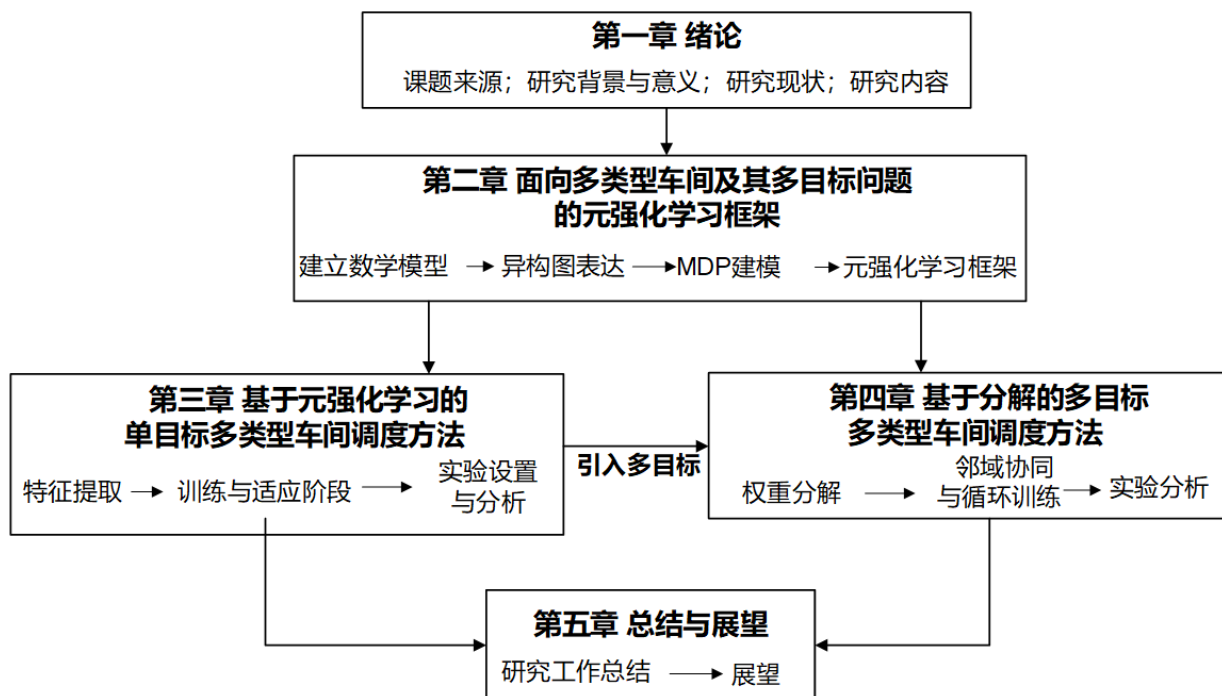


图 1.1 组织结构图

2 面向多类型车间及其多目标问题的元强化学习框架

在制造业智能化转型过程中,车间生产调度优化始终是提升效率、降低成本与保障质量的关键挑战。随着产品更迭速度加快,企业面临缩短制造周期、降低生产成本和快速响应需求的巨大压力,当产品结构发生变化时,往往会引发生产模式调整。例如,从生产标准件切换到定制化样品,意味着加工工序增加、对特定精密设备的依赖以及生产流程重组。这对调度方法的实时性与有效性提出了新的考验。传统调度方法通常面向特定场景设计,其调度效果与计算效率往往会随场景变化明显下降甚至失效。深度强化学习凭借较强的环境感知与序列决策能力,为复杂调度问题提供了新的思路。然而,现有 DRL 方法大多针对特定车间类型设计,缺乏一个通用、可扩展的框架来同时处理多种类型车间的优化问题。

本章提出一种面向多类型车间及其多目标问题的元强化学习框架。采用异构图表示不同生产流程下的调度问题,使用元强化学习提高模型的泛化能力,并将调度求解过程描述为马尔可夫决策过程,定义 MRL 架构下的状态、动作、状态转移与奖励函数。

2.1 问题描述与建模

随着产品在批量、定制化程度与工艺复杂度上的需求日益分化,生产车间已经从早期的单一机器和并行机器模型逐渐演化至多种各具特色的类型,使得调度问题的难度和复杂性不断增加。本节对所涉及的多类车间调度问题模型进行描述,为后续研究建立基础。

2.1.1 多类型车间的问题描述

流水车间问题 (Flow-shop Scheduling Problem, FSP) 可以这样描述: 存在 N 个待加工工件, M 台加工设备。所有待加工工件在所有设备上的加工顺序完全相同,且每道工序只能在一台指定设备上加工。在满足一定约束条件下,安排工件在每台机器上的加工顺序,使得某个调度结果达到最优解。根据工件在各台机器上的加工顺序之间的关系,流水车间问题可进一步细分为置换流水车间 (Permutation

Flow-shop Scheduling Problem, PFSP) 与非置换流水车间 (Non-Permutation Flow-shop Scheduling Problem, NPFSP)。置换流水车间要求所有机器上的工件加工顺序完全一致; 非置换流水车间则放宽了这一限制, 工件仍须依次通过所有机器, 但不同机器上的工件加工顺序可以相互独立地确定。PFSP 的决策规模小; NPFSP 的决策维度更高, 对策略泛化能力的要求更强。因此, 研究中选用非置换流水车间调度问题作为流水车间的对比场景, 以更全面地验证所提框架的有效性。作业车间问题 (Job-shop Scheduling Problem, JSP) 在机器分配上与流水车间存在本质区别, 可以描述为: 存在 N 个待加工工件, M 台加工设备。每个工件都有其独特的、固定的加工顺序, 且每道工序只能在一台指定设备上加工。在满足一定约束条件下, 安排不同工件的工序在每台机器上的加工顺序, 使得某个调度结果达到最优解。柔性作业车间 (Flexible Job-shop Scheduling Problem, FJSP) 在 JSP 的基础上考虑了机器柔性, 可以描述为: 存在 N 个待加工工件, M 台加工设备。某个工件的某道工序可在多个加工设备上加工, 在不同加工设备上的加工时间不同, 待加工工件的加工顺序固定。同时在满足一定约束条件下, 安排工件在每台机器上的加工顺序, 使得某个调度结果达到最优解。

多个类型的车间问题遵循以下假设:

- 1) 所有机器与工件在零时刻均可用。
- 2) 任一机器在同一时刻最多只能加工一道工序。
- 3) 工序一旦开始加工则不允许中断。
- 4) 同一工件的工序间存在严格的先后顺序约束。
- 5) 忽略工序的准备时间以及工件在机器间的传输时间。
- 6) 不考虑机器故障等随机扰动。

2.1.2 数学模型构建

根据对上述涉及到的车间调度问题的描述及假设, 可以建立相应的数学模型, 模型中的各变量定义如表2.1所示。

(1) 优化目标

1) 单目标模型优化目标

最大完工时间反映了生产车间的加工效率, 是调度领域中最重要且应用最为广泛的优化目标之一。通过最小化最大完工时间, 能够在理论上确定最短的生产周

表 2.1 变量及含义

参数	含义
n	工件总数量
J	$= \{J_1, J_2, \dots, J_n\}$, 工件集合
M	$= \{M_1, M_2, \dots, M_m\}$, 机器集合
O_{ij}	工件 i 的第 j 道工序
n_i	工件 i 的工序总数
C_{ij}	工序 O_{ij} 的完工时间
C_i	工件 i 的完工时间, $C_i = C_{i, n_i}$
d_i	工件 i 的交货期
$C_{\max}$	最大完工时间
p_{ijk}	工序 O_{ij} 在机器 M_k 上的加工时间
P_{ij}	工序 O_{ij} 的实际加工时间
M_{ij}	工序 O_{ij} 的可选机器集合
S_{ij}	工序 O_{ij} 的开始时间
x_{ijk}	工序 O_{ij} 分配机器 M_k 加工时为 1, 其余为 0

期, 为调度方案提供评价效率的基准, 同时为后续扩展多目标优化奠定模型基础。

$$\min f_1(x) = \max_{i,j} \{C_{ij}\} \quad (2.1)$$

2) 多目标模型优化目标

除了完工时间, 延迟时间(即拖期)也是一项备受关注的指标。延迟时间是指一个工件的完工时间与其交货期限之间的差值, 它衡量了工件实际完成时间与交货期限之间的偏离, 过长的延迟会导致客户满意度下降、违约风险增加乃至企业声誉受损。因此多目标模型的优化目标引入最小化总拖期作为另一关键目标。

$$\min f_2(x) = \sum_{i=1}^n \max(C_i - d_i, 0) \quad (2.2)$$

(2) 约束条件:

加工时间与完工时间约束如式(2.3)所示:

$$C_{ij} = S_{ij} + P_{ij}, \quad \forall i, j \quad (2.3)$$

其中, S_{ij} 表示工序 O_{ij} 的开始加工时间, P_{ij} 表示工序 O_{ij} 的实际加工时间。对于 NPFSP/JSP: $P_{ij} = p_{ij}$ 。对于 FJSP: $P_{ij} = \sum_{k \in M_{ij}} x_{ijk} \cdot p_{ijk}$ 。

机器分配约束如式(2.4)所示, 表示工序 O_{ij} 只能被加工一次。

$$\sum_{k \in M_{ij}} x_{ijk} = 1, \quad \forall i, j \quad (2.4)$$

工序顺序约束如式(2.5)所示, 保证同一工件内后续工序 $O_{i,j+1}$ 的开始时间不早于前序工序 O_{ij} 的完工时刻。

$$S_{i,j+1} \geq S_{ij} + P_{ij}, \quad \forall i, j \quad (2.5)$$

机器占用约束如式(2.6)所示, 用于保证同一台机器上任意两道工序的加工时间不能重叠:

$$S_{ij} \geq S_{i'j'} + P_{i'j'} \quad \text{或} \quad S_{i'j'} \geq S_{ij} + P_{ij}, \quad \forall (i, j) \neq (i', j'), \quad x_{ijk} = x_{i'j'k} = 1 \quad (2.6)$$

其中, (i, j) 与 (i', j') 表示两道不同工序, 条件 $x_{ijk} = x_{i'j'k} = 1$ 表示两道工序均被分配到同一台机器 M_k 。

综上, 多类型车间问题的优化可表示为:

$$\begin{aligned} \min F(x) = & \begin{cases} f_1(x), & \text{单目标} \\ \{f_1(x), f_2(x)\}, & \text{多目标} \end{cases} \\ \text{s.t.} & \begin{cases} C_{ij} = S_{ij} + P_{ij} \\ \sum_{k \in M_{ij}} x_{ijk} = 1 \\ S_{i,j+1} \geq S_{ij} + P_{ij} \\ S_{ij} \geq S_{i'j'} + P_{i'j'} \quad \text{或} \quad S_{i'j'} \geq S_{ij} + P_{ij}, \quad \forall (i, j) \neq (i', j') \end{cases} \end{aligned} \quad (2.7)$$

2.1.3 异构图模型

为使状态的图表示形式在降低复杂度的同时包含尽可能多的车间信息, 并进一步应用图神经网络获取图嵌入向量, 需要设计适用于多类型车间调度问题的异构图模型。本节给出多类型车间异构图的结构和特征向量设计方案, 包括节点和边的构成以及特征值信息。

多类型车间调度问题的异构图模型统一形式化定义为 $G = (V, C, D)$, 其各部分含义如下: 节点集合 V 包含两类实体节点和两类虚拟节点, 即工序节点集 O , 机器节点集 M , 虚拟开始节点 S 和结束节点 T , 满足 $V = (O \cup M) \cup \{S, T\}$ 。每个工序节点代表一个待加工的操作单元, 每个机器节点代表一个可用的加工资源。关系则

由有向弧集合 C 与无向弧集合 D 共同定义。有向弧集合 C 表示时序约束，用于连接同一工件内相邻的两道工序节点如 O_{ij} 与 $O_{i,j+1}$ ，从而规定工序执行的先后顺序，确保调度方案的可行性；无向弧集合 D 中的每条无向弧将一个工序节点连接到其所有可选加工机器。调度过程中的单步决策体现为：从当前待加工工序对应的可加工边集合中选择一条边，表示将该工序分配给对应的机器；同时，该工序节点连接至其他可选机器的边则被视为无用边，并在图中屏蔽。为进一步说明异构图的表示方法，图2.1给出了一个规模为 3×3 的 NPFSP 问题的异构图表示，并行的三条箭头链表示了三个工件的工艺路径，上方标注的三台机器各自通过象征可加工的边连接到对应加工阶段的所有工序，结合起来共同表示了三个工件的工艺路径。

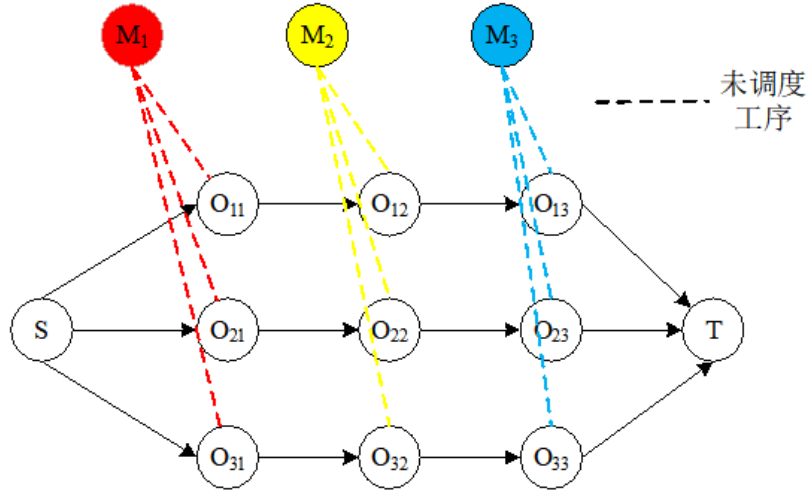


图 2.1 NPFSP 的异构图表示

图2.2展示了规模 3×3 的 FJSP 问题的一次调度决策过程。初始状态的工序 O_{11} 、 O_{21} 和 O_{31} 已加工，对应工序和机器的边变为已分配的实线，随后决策智能体从多条候选的“可加工”关系中，选择 O_{22} 分配至 M_3 。这一决策在图结构上被表征为一次状态跃迁：连接 O_{22} 与 M_3 的边由表示可能性的虚线转变为表示已分配的实线，而该工序到其他机器的可选连接则随之失效或移除。这种表示方法可以有效适配不同类型的车间调度问题，为后续的调度方法提供了有效的基础。

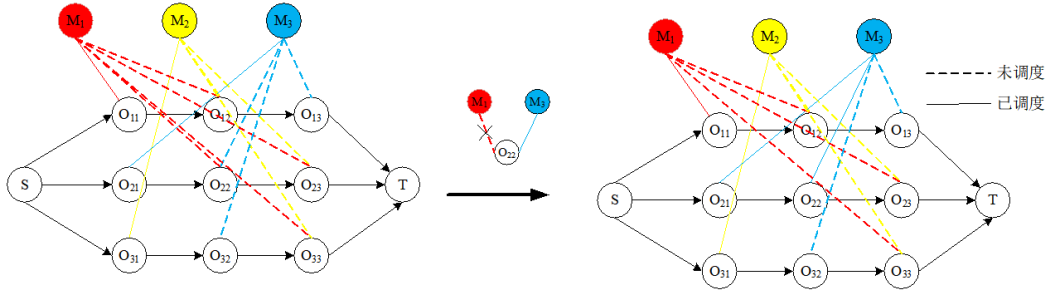


图 2.2 FJSP 的决策在异构图的映射

2.2 元强化学习基础

2.2.1 元学习的基本概念

元学习是指通过在多个相关任务上进行学习，使算法能够快速适应新任务的学习过程。与传统机器学习不同，元学习的目标是学习一个通用的学习策略或模型参数初始化，使得在面对新任务时仅需少量数据和训练步次便能达到较好的性能。

元学习的核心思想在于利用先前任务的学习经验来加速新任务的学习过程。在包含多个相关任务的任务分布 $p(\mathcal{T})$ 下，元学习通过在支持集上进行内循环学习来快速适应单个任务，再利用查询集上的性能来更新元参数，形成外循环优化。这种内外循环相结合的训练机制使得模型能够获得更强的泛化能力和快速适应能力。图2.3展示了元学习区别于传统方法的训练流程。

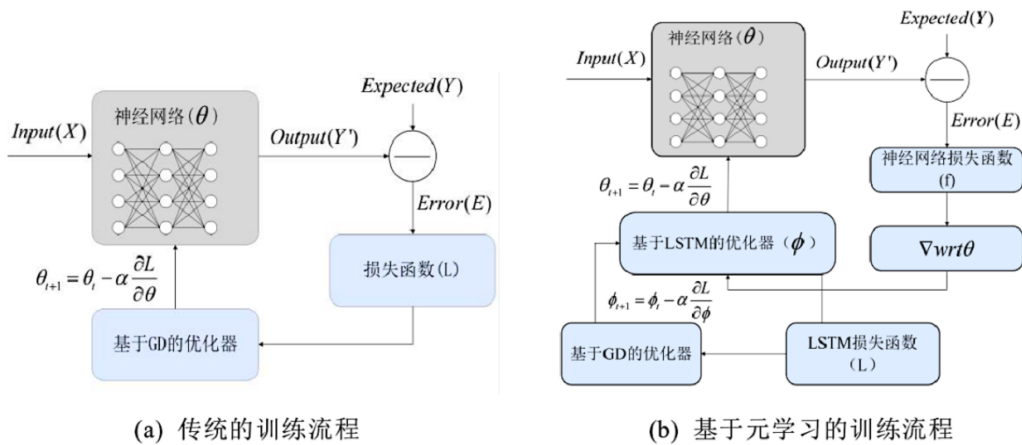


图 2.3 元学习和传统方法的训练流程

形式化地，假设任务分布 $p(\mathcal{T})$ 包含一组相关但各有差异的任务。每个任务 $T_i \sim p(\mathcal{T})$ 由其自身的数据集 $\mathcal{D}_i = \{\mathcal{D}_i^{sup}, \mathcal{D}_i^{qry}\}$ 构成，其中支持集 $\mathcal{D}_i^{sup}$ 用于任务内

部的学习（内循环），查询集 $\mathcal{D}_i^{qry}$ 用于评估适应效果并更新元参数（外循环）。元学习的目标可表述为：

$$\min_{\theta} \sum_{T_i \sim p(\mathcal{T})} \mathcal{L}_{T_i}(\mathcal{A}(\theta, \mathcal{D}_i^{sup}), \mathcal{D}_i^{qry}) \quad (2.8)$$

其中 θ 为元参数， $\mathcal{A}(\theta, \mathcal{D}_i^{sup})$ 表示以 θ 为起点、在支持集 $\mathcal{D}_i^{sup}$ 上执行适应算法后得到的任务特定参数， $\mathcal{L}_{T_i}$ 为在查询集上计算的任务损失。

2.2.2 模型无关的元学习 (MAML)

模型无关的元学习 MAML^[21] 是元学习领域的一个重要算法。MAML 的核心优势在于其模型无关性，不管采用何种神经网络架构或学习算法，MAML 都能通过优化初始参数来提升在新任务上的快速学习能力。

MAML 算法的基本思想是：寻找一个初始参数 θ^* ，使得从该参数出发，经过仅有的几步梯度更新就能在新任务上达到良好的性能。具体地，在元训练过程中，MAML 采用双层优化策略：内循环对每个任务 T_i 的支持集进行梯度更新，获得任务特定的参数 θ'_i ；外循环基于各任务查询集上的性能反馈，对元参数 θ^* 进行更新，使得更新后的参数对所有任务都具有更好的泛化性能。

对于从任务分布中采样的任务 T_i ，MAML 以当前元参数 θ 为起点，在该任务的支持集 $\mathcal{D}_i^{sup}$ 上执行 K 步梯度下降。以单步更新为例，任务特定参数的计算为：

$$\theta'_i = \theta - \alpha \nabla_{\theta} \mathcal{L}_{T_i}(\theta, \mathcal{D}_i^{sup}) \quad (2.9)$$

其中 α 为内循环学习率， $\mathcal{L}_{T_i}$ 为任务 T_i 上的损失函数。当执行 K 步更新时，递推地进行： $\theta_i^{(0)} = \theta$ ， $\theta_i^{(k)} = \theta_i^{(k-1)} - \alpha \nabla_{\theta_i^{(k-1)}} \mathcal{L}_{T_i}(\theta_i^{(k-1)})$ ，最终 $\theta'_i = \theta_i^{(K)}$ 。

外循环基于所有采样任务在查询集上的表现来优化元参数 θ 。其优化目标为：

$$\min_{\theta} \mathcal{L}_{meta}(\theta) = \sum_{T_i \sim p(\mathcal{T})} \mathcal{L}_{T_i}(\theta'_i, \mathcal{D}_i^{qry}) \quad (2.10)$$

其中 θ'_i 由式(2.9)得到，是 θ 的函数。元参数的更新规则为：

$$\theta \leftarrow \theta - \beta \nabla_{\theta} \mathcal{L}_{meta}(\theta) = \theta - \beta \sum_{T_i} \nabla_{\theta} \mathcal{L}_{T_i}(\theta'_i) \quad (2.11)$$

其中 β 为外循环学习率。

由于 θ'_i 是通过 θ 进行梯度下降得到的，因此 $\nabla_{\theta} \mathcal{L}_{T_i}(\theta'_i)$ 的计算需要通过链式法则展开。以单步内循环更新为例：

$$\nabla_{\theta} \mathcal{L}_{T_i}(\theta'_i) = \nabla_{\theta'_i} \mathcal{L}_{T_i}(\theta'_i) \cdot \frac{\partial \theta'_i}{\partial \theta} = \nabla_{\theta'_i} \mathcal{L}_{T_i}(\theta'_i) \cdot (I - \alpha \nabla_{\theta}^2 \mathcal{L}_{T_i}(\theta)) \quad (2.12)$$

其中 I 为单位矩阵, $\nabla_{\theta}^2 \mathcal{L}_{T_i}(\theta)$ 为损失函数关于 θ 的海森矩阵。该二阶导数项使得 MAML 能够感知参数空间的曲率, 从而找到更适合快速适应的初始化位置。在实际应用中, 为降低计算开销, 常采用一阶近似 (First-Order MAML, FOMAML), 即忽略海森矩阵项, 将元梯度简化为:

$$\nabla_{\theta} \mathcal{L}_{T_i}(\theta'_i) \approx \nabla_{\theta'_i} \mathcal{L}_{T_i}(\theta'_i) \quad (2.13)$$

2.2.3 MAML 与强化学习的结合

在强化学习场景下, MAML 的框架可以自然地与策略梯度方法结合, 形成元强化学习 Meta-RL^[22]。此时, 每个“任务” T_i 对应一个马尔可夫决策过程 (Markov Decision Process, MDP), 智能体的目标是学习一组元参数 θ , 使得面对新 MDP 时仅通过少量交互与梯度更新即可获得高性能策略。

假设存在一个 MDP 任务分布 $p(\mathcal{T})$, 每个任务 T_i 对应一个 MDP $\mathcal{M}_i = (S, A, R_i, T_i, \gamma)$ 。不同任务可能在奖励函数 R_i 、状态转移 T_i 或初始状态分布上存在差异。元强化学习的目标为:

$$\theta^* = \arg \max_{\theta} \mathbb{E}_{T_i \sim p(\mathcal{T})} [J_{T_i}(\theta'_i)] = \arg \max_{\theta} \mathbb{E}_{T_i \sim p(\mathcal{T})} \left[\mathbb{E}_{\tau \sim \pi_{\theta'_i}} \left[\sum_{t=0}^{H-1} \gamma^t r_t \right] \right] \quad (2.14)$$

其中 $J_{T_i}(\theta'_i)$ 为在任务 T_i 上使用适应后参数 θ'_i 的策略 $\pi_{\theta'_i}$ 所获得的期望累积奖励, γ 为折扣因子, H 为轨迹长度。

在强化学习中, MAML 的内循环通过策略梯度方法实现。对于采样到的任务 T_i , 智能体使用当前元策略 π_{θ} 与任务环境交互, 收集一批轨迹 $\mathcal{D}_i^{sup} = \{\tau_1, \tau_2, \dots, \tau_M\}$ 构成支持集, 其中每条轨迹 $\tau_j = \{(s_0, a_0, r_0), (s_1, a_1, r_1), \dots\}$ 。基于支持集, 利用策略梯度估计损失函数的梯度:

$$\nabla_{\theta} J_{T_i}(\theta) \approx \frac{1}{M} \sum_{j=1}^M \sum_{t=0}^{H-1} \nabla_{\theta} \log \pi_{\theta}(a_t^j | s_t^j) \cdot \hat{A}_t^j \quad (2.15)$$

其中 $\hat{A}_t^j$ 为估计的优势函数。随后执行梯度上升 (因为目标是最大化奖励):

$$\theta'_i = \theta + \alpha \nabla_{\theta} J_{T_i}(\theta) \quad (2.16)$$

内循环得到各任务的适应参数 θ'_i 后, 使用 $\pi_{\theta'_i}$ 在相应任务上继续采集轨迹数据 $\mathcal{D}_i^{qry}$ 作为查询集。外循环的元目标为:

$$\mathcal{L}_{meta}(\theta) = - \sum_{T_i \sim p(\mathcal{T})} J_{T_i}(\theta'_i) \quad (2.17)$$

的协同框架，如图2.4所示。元训练阶段整合元学习与强化学习机制，形成内外双层优化循环：内循环在异构图表示不同类型的车间问题的基础上，通过图神经网络提取异构图中工序与机器节点的深度特征嵌入，进而作为策略网络的输入，以生成当前动作选取的概率。智能体每执行一个动作，便会更新图中边的状态与类型，从而驱动图状态持续演进。通过多轮交互收集包含状态、动作、状态转移和奖励的轨迹数据，以此更新网络参数。外循环则基于内循环在调整后的策略在新任务上的表现，对元参数进行更新。为支撑该训练过程，任务数据被划分为支持集与查询集，分别用于内循环的策略优化与外循环的评估。在快速适应阶段，已训练的模型通过与新调度任务进行交互，生成对应轨迹并输入至 **Adaptor**；**Adaptor** 借助梯度更新的微调方法，经少量轮次调整即可得到适应新环境的目标模型，从而能够为不同类型的车间调度问题生成高效决策。

上述框架的实现依赖于将调度过程抽象为马尔可夫决策过程，这时系统可以定义为 $M = (S, A, R, T, H)$ 。其中，状态空间 S 表示所有可能系统状态的集合；动作空间 A 表示可执行的调度决策集合；奖励函数 $R(r_{t+1}|s_t, a_t, s_{t+1})$ 用于量化所选动作带来的即时收益；状态转移函数 $T(s_{t+1}|s_t, a_t)$ 描述了系统在执行动作后的状态转移规律； H 则为决策轨迹的长度。在标准强化学习框架下，智能体的目标是学习一个最优策略 π ，以最大化期望累积奖励。基于此，下文将进一步展开基于多任务环境的元强化学习马尔可夫决策过程描述。

元强化学习的训练阶段通常在满足特定分布的任务集合 $p(\mathcal{T})$ 上进行。在内循环中，首先从分布中采样一批任务，随后在每个任务上进行优化，学习通用的调度知识。该阶段的马尔可夫决策过程各组成成分的具体定义如下：

(1) 状态

使用异构图表示不同类型车间调度问题，为马尔可夫决策过程中的状态提供了结构化的统一编码。在强化学习的每一步决策前，系统将当前的调度问题实例映射为一异构图。该图通过图神经网络进行处理，经过多层消息传递与聚合后得到工序和机器节点的嵌入向量及全局图特征输入到策略网络中，构成了马尔可夫决策过程的状态表示。这种表示方法将异构的车间信息与调度状态编码为一个固定维度、连续的向量空间，为后续的策略学习提供了状态接口。

状态 s_t 表示在时刻 t 车间系统的全局配置，包括作业进度、设备状态及工序-设备匹配关系等关键信息。状态集合 S 表示调度周期内所有可能状态 s_t 的集合。每个

状态由多个特征组合而成，通常包含机器特征和工序特征。在基于异构图的调度模型中，我们分别为工序节点与机器节点定义结构化特征向量，以全面刻画系统动态状态。对于工序节点 O_{ij} ，定义六维特征向量 $\mu_{ij} \in \mathbb{R}^6$ ，依次包括：标识该工序是否已被调度的二值状态；其当前可选机器数量，即图结构中的一阶邻居机器节点数；该工序的加工时间（若未调度则取可选机器上的平均时间，若已分配则取实际值）；实际或预计的开始加工时间；所属工件中剩余未调度的工序数量；以及所属工件的预计总完工时间。对于机器节点 M_k ，定义三维特征向量 $\nu_k \in \mathbb{R}^3$ ，包括：机器预计进入空闲状态的可用时间；当前可加工工序数（即其邻居工序节点数量）；以及从调度开始到当前时刻的机器利用率。虚拟起始节点与终止节点则以零向量表示，主要用于维持图结构的完整性，其信息不参与实际决策。上述节点特征定义对不同车间类型保持统一，车间类型的差异主要体现在异构图的拓扑结构上：FJSP 中工序节点通过多条边连接若干台候选机器，体现机器柔性；NPFSP 和 JSP 中每个工序节点仅连接唯一指定机器，区别在于 NPFSP 所有工件共享相同的机器加工顺序，而 JSP 每个工件拥有各自独立的机器路径。

（2）动作

调度过程中的动作定义为在每一步决策时从当前可调度工序中选择一个工序，并为其分配合法机器。本文将动作空间统一为工序-机器对的形式，使同一策略网络可同时处理不同类型车间。为保证可行性，本文使用动作掩码屏蔽不可行动作。掩码由工序-机器可加工性、工件完成状态以及约束标志共同决定：FJSP 中同一工序可对应多个候选机器，NPFSP 和 JSP 中每道工序只对应唯一机器；本文聚焦于 FJSP、JSP、NPFSP 三类车间问题，但该框架的设计允许通过扩展类型编码与集成相应的约束处理规则，进一步支持无等待、阻塞和置换等其他约束。该掩码由以下约束层逐层取交集生成：1）工序-机器可加工约束（ $\mathcal{M}_{\text{proc}}$ ）：来源于异构图中的工序-机器邻接矩阵。对于 NPFSP 和 JSP，每道工序仅对应唯一指定机器，因此邻接矩阵的每一行有且仅有一个元素为 1；对于 FJSP，同一工序可在多台候选机器上加工，对应行中有多个元素为 1。这一矩阵在问题实例加载时即自动生成，无需人工设计规则，使三类车间的机器柔性差异自然编码于掩码之中。2）工件完工约束（ $\mathcal{M}_{\text{job}}$ ）：当某工件的所有工序均已被调度，则屏蔽该工件对应的全部动作，避免对已完成工件进行冗余决策。

这里给出其他可拓展的约束处理方式：3）置换约束（ $\mathcal{M}_{\text{perm}}$ ）：当调度问题存在

置换约束时激活。该约束要求所有机器上的工件加工顺序保持一致，通过跟踪各机器上已分配工件的顺序索引，仅允许满足全局一致排列的工序-机器对通过掩码。4) 无等待约束 ($\mathcal{M}_{nw}$) 与阻塞约束 ($\mathcal{M}_{blk}$)：分别在对应约束标志位激活时生效。无等待约束要求工件在前一道工序完成后立即开始下一道工序，不允许中间等待；阻塞约束则要求工件在当前机器加工完成后必须立刻转移到下一台机器，否则将阻塞当前机器。这两类约束进一步缩小可行动作集合，适用于特定的生产场景变体。

最终掩码为上述各层的逐元素与运算结果： $\mathcal{M} = \mathcal{M}_{proc} \wedge \mathcal{M}_{job}$ 。这种模块化的掩码设计使得同一策略网络无需修改结构即可适配 FJSP、NPFSP、JSP 等不同类型车间。具体而言，FJSP 的掩码特点在于 $\mathcal{M}_{proc}$ 的每一行包含多个可选机器；NPFSP 和 JSP 的 $\mathcal{M}_{proc}$ 每行仅有一个合法机器；而 NPFSP 与 JSP 的区别是 NPFSP 所有工件的第 j 道工序均指向同一台机器 M_j ，JSP 中各工件的机器路径各不相同。通过施加置换约束 $\mathcal{M}_{perm}$ ，无等待约束 $\mathcal{M}_{nw}$ 和阻塞约束 $\mathcal{M}_{blk}$ 等，可进一步扩展所支持的调度变体。

(3) 状态转移

当智能体选取一个动作后，环境执行该调度决策并更新系统状态，形成马尔可夫决策过程中从 s_t 到 s_{t+1} 的状态转移。该过程在所有车间类型上遵循统一的执行流程，但具体更新内容会因车间结构差异而有所不同，主要包括以下几个方面：1) 异构图结构更新：被选中的工序 O_{ij} 与机器 M_k 之间的边标记为“已分配”，同时该工序到其他候选机器的连接被移除。对于 NPFSP 和 JSP，由于每道工序仅对应唯一机器，此步骤主要体现为边状态更新；对于 FJSP，则会将多条候选边缩减为单一确定边，并同步更新对应加工时间。2) 工序与机器特征更新：工序 O_{ij} 由“未调度”切换为“已调度”，其开始时间、完工时间及可选机器数等特征随之更新；被分配机器 M_k 的可用时间向后推移，机器利用率和可加工工序数也随调度结果同步调整。3) 工件进度推进：工件 J_i 的工序指针前移一步，下一道工序 $O_{i,j+1}$ 进入可调度状态。若某工件的全部工序均已调度，则该工件被标记为完工，其对应动作在后续掩码中被屏蔽。当所有工件的全部工序均已调度完成时，环境判定实例终止，调度过程结束。

对于其他本文未涵盖的约束场景，这里给出预设的处理逻辑：在调度过程中，基于约束类型同步更新各机器上的工件加工顺序索引矩阵，并相应修正工序的开始时间及相关时间特征。

(4) 奖励

奖励函数用于引导智能体学习优化调度目标。本节聚焦于最小化总完工时间这一单目标，具体而言，每一步的即时奖励 r_t 设计为当前动作导致的完工时间增量的负值，即 $r_t = C_{\max}(s_t) - C_{\max}(s_{t+1})$ ，其中 $C_{\max}(s_t)$ 指根据时刻 t 已调度工序的完工时间和未调度工序的平均完工时间计算得到的最大完工时间， H 个时间步的累积奖励 $\sum_{t=0}^{H-1} r_t = C_{\max}(s_0) - C_{\max}(s_H)$ ，由于 $C_{\max}(s_0)$ 为常数，因此最大化累积奖励等价于最小化完工时间。

(5) 策略

策略 $\pi_{\theta}(a_t|s_t)$ 定义了状态 s_t 下各动作的选取概率分布。在每个决策时刻，策略网络与可行动作掩码 $\mathcal{M}_t$ 结合，对不可行动作进行屏蔽，仅在可行动作集合上经 Softmax 得到概率分布。元强化学习框架的核心目标在于获得一组元参数 θ ，使其在当前任务分布下具有最大化期望累积奖励的能力，对于待求解问题仅通过少量梯度更新即可获得接近最优的策略表现。

基于多类型车间的元强化学习框架，对于给定类型的调度问题，求解问题的适应阶段为：

- 1) 加载训练好的元模型，其参数为 θ^* ，并设定最大微调步数 K 和微调批大小。初始化经验回放缓冲区，并设置当前策略参数 $\phi = \theta^*$ 。
- 2) 根据问题的工件、工序、机器和加工时间矩阵等参数初始化调度仿真环境。
- 3) 状态提取：在每个决策时刻，获取环境当前状态，并将其构建为异构图。通过图神经网络处理得到节点嵌入，作为策略网络的输入。
- 4) 动作选取与执行：策略网络根据当前参数输出动作概率分布，根据贪婪采样选取调度动作。环境执行动作转移到新状态并产生奖励。
- 5) 判断是否完成微调：当前微调步数 i 若小于 K ，则转到第 6) 步，否则转到第 7) 步。
- 6) 将经验存入缓冲区。如果缓冲区数据量达到批大小，则计算策略梯度损失，并执行梯度下降更新参数 ϕ ，随后清空缓冲区。返回第 2) 步继续循环。
- 7) 若所有工序都已被调度，输出调度方案；否则返回第 3) 步

2.4 本章小结

本章围绕多类型车间调度问题的统一建模与求解，构建了面向多类型车间及其多目标问题的元强化学习框架。在问题建模方面，对多种车间典型问题进行了统一描述，建立了以最小化最大完工时间为单目标、以最小化最大完工时间与总拖期为多目标的数学优化模型。在图表示方面，设计了适用于多类型车间的异构图模型，实现了对不同类型车间结构的统一图表示。在算法基础方面，介绍了元学习、MAML与元强化学习的基本概念，阐述了内外双层优化循环的训练方式及其在多任务场景下的适用性。在此基础上，将异构图表示与元强化学习机制相结合，构建了包含元训练与快速适应两阶段的总体框架，将调度过程建模为马尔可夫决策过程，定义了状态、动作、状态转移、奖励与策略等核心要素，并针对 FJSP、NPFSP 和 JSP 三种车间类型详细阐述了动作掩码的多层级组合机制、状态转移中图结构与节点特征的差异化更新逻辑，给出了面向新调度实例的适应流程。该框架在设计上也保持了对其他车间类型的可扩展性。

3 基于元强化学习的单目标多类型车间调度方法

第二章构建了面向多类型车间的元强化学习框架，完成了多种车间类型的异构图建模，将调度过程形式化为马尔可夫决策过程，定义了包含动作、状态转移等核心要素。本章在上述框架基础上，以最小化最大完工时间为优化目标，提出基于元强化学习的多类型车间调度方法。首先，设计基于图注意力网络的机器节点特征提取与基于多层感知机的工序节点特征提取方法，从异构图中获取高质量的图嵌入表征。其次，构建包含元训练算法与适应算法的元强化学习方法，通过在覆盖 FJSP、NPFSP 和 JSP 的混合任务分布上进行内外循环优化，学习通用调度策略参数。此外，针对不同类型元训练中由于不同车间特征重要性差异导致的噪声梯度问题，引入基于特征调控模块的策略网络输入调节机制，提升元模型的收敛稳定性与泛化能力。最后，在多组随机生成算例和标准基准算例上进行系统的实验对比与分析，验证所提方法在求解质量、运行效率及跨类型泛化能力方面的有效性。

3.1 元强化学习模型的整体架构

基于第二章构建的多类型车间调度元强化学习框架，本章给出多类型车间的元强化学习算法（MAML with Reinforcement Learning, MRL）具体实现方案，并通过实验验证该框架的可行性与有效性。该方法利用异构图进行状态表征，将元学习快速适应机制与策略优化算法相结合，构建具备通用性与自适应能力的调度决策模型。

具体而言，通过异构图统一表达不同类型的车间问题：采用图注意力网络聚合机器节点的上下文信息，捕捉资源间的竞争与协作关系；使用多层感知机提取工序节点的特征；并引入特征调控模块，根据车间配置信息对节点特征进行自适应缩放，增强模型对不同调度场景的适应能力。训练方面，采用与模型无关的元学习方法与近端策略优化算法融合；元训练阶段从多样化的车间任务中学习通用的元策略参数，以最大化累积奖励为目标；快速适应阶段面对新实例时，加载元参数并通过少量轨迹样本微调，即可得到针对该实例的高效调度策略。

3.2 基于异构图的特征提取方法

3.2.1 基于图注意力机制的机器节点特征

在面向多类型车间调度的异构图表征中，一台机器通常与多道可加工工序相连，而这些工序在加工时间、紧急程度以及对整体调度目标的影响上均存在明显差异。采用简单的聚合操作来提取机器节点特征，往往难以有效刻画不同工序对于该机器状态的贡献，导致特征表示区分度不足、信息粗糙。为此，本节引入注意力机制，采用图注意力网络（Graph Attention Network, GAT）进行机器节点特征的提取。通过自适应学习为机器节点的每一个相邻工序节点计算一个动态权重，该权重能够反映在当前全局调度状态下，相应工序对于评估该机器状态的重要程度。这使得机器节点的特征表示能够随调度进程动态演化，从而为后续策略网络提供更精准、更具区分度的输入信息。

在任一调度时刻 t ，对于机器节点 M_k ，其嵌入生成过程如下：首先，构造信息传递的特征向量。机器节点 M_k 与任一邻居工序节点 $O_{i,j} \in \mathcal{N}_t(M_k)$ 之间仅存在一条无向边。为融合节点与边的信息，将工序节点特征向量 $\mu_{i,j}$ 与此边特征向量 $\lambda_{i,j,k}$ 进行拼接，构成沿该边传递的增强特征向量 $\mu_{i,j,k} = [\mu_{i,j} \parallel \lambda_{i,j,k}] \in \mathbb{R}^7$ 。其次，引入可学习的线性变换矩阵 $\mathbf{W}^O \in \mathbb{R}^{d \times 7}$ 和 $\mathbf{W}^M \in \mathbb{R}^{d \times 3}$ ，分别用于将工序特征 $\mu_{i,j,k}$ 及机器节点自身特征 v_k 映射到 d 维的共享特征空间。接着，计算注意力系数以评估各邻居节点（包括自环）对当前机器节点的重要性。对于邻居工序节点 $O_{i,j}$ ，其注意力系数 $e_{i,j,k}$ 通过一个共享的单层前馈神经网络 $\text{attn} \in \mathbb{R}^{2d}$ 计算得出：

$$e_{i,j,k} = \text{LeakyReLU}(\text{attn}^T [\mathbf{W}^M v_k \parallel \mathbf{W}^O \mu_{i,j,k}]) \quad (3.1)$$

同时，为保留机器节点自身状态信息，需计算自环注意力系数 e_{kk} ：

$$e_{kk} = \text{LeakyReLU}(\text{attn}^T [\mathbf{W}^M v_k \parallel \mathbf{W}^M v_k]) \quad (3.2)$$

然后，对所有邻居节点（包括自身）的注意力系数进行标准化处理，得到归一化的注意力权重 $\alpha_{i,j,k}$ 与 α_{kk} 。

最后，机器节点 M_k 的更新嵌入向量 v'_k 通过对其所有邻居节点（含自身）的变换后特征进行加权聚合而得：

$$v'_k = \sigma \left(\alpha_{kk} \mathbf{W}^M v_k + \sum_{O_{i,j} \in \mathcal{N}_t(M_k)} \alpha_{i,j,k} \mathbf{W}^O \mu_{i,j,k} \right) \quad (3.3)$$

其中, σ 为非线性激活函数。该方法将边特征融入注意力计算, 使模型能精细评估动作对机器状态的影响, 从而为后续的调度决策提供更精准的特征。

3.2.2 基于多层感知机的工序节点特征

与机器节点相比, 工序节点 $O_{i,j}$ 的邻居结构大不相同: 其邻居数量相对有限, 但节点类型与语义关系更为多样。具体来说, 工序节点 $O_{i,j}$ 在时刻 t 的邻居集合通常包含其前驱工序节点 $O_{i,j-1}$ 、后继工序节点 $O_{i,j+1}$ 、可选加工机器集合 $M_{i,j}$ 中的若干机器节点 M_k , 以及节点自身。尽管可选机器节点可能数量较多, 且不同工序节点也具有类似的邻居模式, 但由于前驱、后继节点与机器节点在物理语义上截然不同, 在异构图神经网络中不宜使用同一套参数化网络对其进行处理, 引入复杂的异构注意力机制可能带来大量冗余计算, 其性能提升在此处并不明显。

因此, 针对工序节点嵌入, 本节使用多个独立的多层感知机 (Multilayer Perceptron, MLP) 分别处理不同类型邻居关系传递的信息。具体而言, 共需五个不同的 MLP, 分别记为 MLP_{n_0} 至 MLP_{n_4} 。每个 MLP 均包含 L_h 层隐藏层, 隐藏层维度为 d_h , 输出层维度为 d , 并采用 ELU 函数作为激活函数以增强非线性表达能力。

基于上述设计, 工序节点 $O_{i,j}$ 的更新嵌入向量 $\mu'_{i,j}$ 通过聚合来自其各类邻居及自身的信息计算得到, 其过程形式化表示为:

$$\mu'_{i,j} = \text{MLP}_{n_0}(\text{ELU}[\text{MLP}_{n_1}(\mu_{i,j-1}) \parallel \text{MLP}_{n_2}(\mu_{i,j+1}) \parallel \text{MLP}_{n_3}(\overline{v'_{i,j}}) \parallel \text{MLP}_{n_4}(\mu_{i,j})]) \quad (3.4)$$

其中, 符号说明如下:

$\mu_{i,j-1}$ 和 $\mu_{i,j+1}$ 分别表示前驱与后继工序节点的当前特征向量。 $\overline{v'_{i,j}}$ 表示工序 $O_{i,j}$ 所有可选机器节点嵌入的聚合表征。该方法可以有效捕捉工序节点和其邻居节点的复杂关系, 提高信息的精准度。

3.3 多类型车间问题的 MRL 算法

3.3.1 MRL 训练算法

该算法用于训练一个能够快速适应新调度任务的元强化学习模型, 其核心是通过双层优化架构, 使模型在掌握跨任务通用知识的同时, 保留对特定任务快速微调的能力。如图3.1所示, 首先生成多个不同类型任务, 这些任务与元参数交互的数据

集被划分为支持集与查询集两部分。支持集用于内循环的训练，得到针对当前任务的特定参数 θ_i ，查询集则用于外循环中评估该次适应的效果并优化模型的元参数 θ_{meta} （对应第2.2节中的元参数 θ ）。

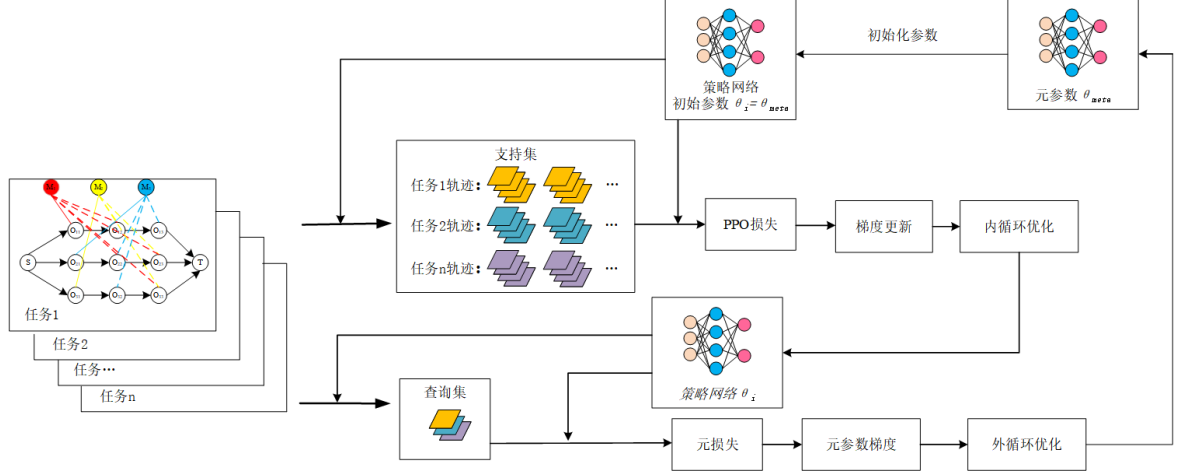


图 3.1 MRL 训练算法

整个训练过程遵循双层优化的框架进行迭代。首先从任务分布中采样一批任务，针对任务 T_i ，使用当前元参数 θ_{meta} 初始化策略网络与价值网络得到当前任务的策略 π_{θ_i} 。策略 π_{θ_i} 与具体任务环境交互，收集轨迹数据构成该任务的支持集 $\mathcal{D}_{sup}$ 。基于支持集数据，计算近端策略优化（Proximal Policy Optimization, PPO）损失函数，对当前参数 θ_i 进行若干步梯度更新，完成对该任务的快速适应。下面详细介绍 MRL 在时间步 t 的处理。

在强化学习中，策略梯度定理^[13]给出了策略目标函数 $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\sum_t r_t]$ 关于参数 θ 的梯度：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{H-1} \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot A^{\pi_\theta}(s_t, a_t) \right] \quad (3.5)$$

其中 $A^{\pi_\theta}(s_t, a_t)$ 为优势函数，衡量动作 a_t 在状态 s_t 下相对于平均水平的好坏。在实际训练中，由于策略参数在更新过程中会发生变化，而采集的轨迹数据来自旧策略 $\pi_{\theta_i}^{old}$ ，因此引入重要性采样比率：

$$r_t(\theta_i) = \frac{\pi_{\theta_i}(a_t | s_t)}{\pi_{\theta_i}^{old}(a_t | s_t)} \quad (3.6)$$

利用该比率，替代策略目标函数可写为：

$$L^{IS}(\pi_{\theta_i}) = \mathbb{E}_t[r_t(\theta_i) \cdot A_t] \quad (3.7)$$

然而，当 $r_t(\theta_i)$ 偏离 1 较远时，重要性采样的方差急剧增大，导致训练不稳定。

为解决上述问题，PPO 通过裁剪机制限制策略更新的幅度。裁剪策略损失 L^{clip} 定义为：

$$L^{clip}(\pi_{\theta_i}) = \mathbb{E}_t[\min(r_t(\theta_i)A_t, \text{clip}(r_t(\theta_i), 1 - \epsilon, 1 + \epsilon) A_t)] \quad (3.8)$$

其中 ϵ 为裁剪范围超参数（典型值为 0.2）。 $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$ 将比率限制在 $[1 - \epsilon, 1 + \epsilon]$ 区间内。该裁剪机制的作用机理如下：当优势 $A_t > 0$ 时，表示当前动作优于平均时，策略倾向于增大该动作的概率，但 $\min$ 操作使得 r_t 超过 $1 + \epsilon$ 后不再获得额外收益，避免了过度更新；当 $A_t < 0$ ，当前动作劣于平均时，策略倾向于减小该动作的概率，但 r_t 低于 $1 - \epsilon$ 后同样不再被惩罚。由此，裁剪目标构造了一个信赖域约束的下界，在保证策略单调改进的同时控制了更新步长。

优势函数 A_t 的准确估计对策略梯度的质量至关重要。本文采用广义优势估计（Generalized Advantage Estimation, GAE）方法，通过引入偏差-方差权衡参数 λ_{GAE} 来平衡估计精度与计算稳定性。定义时序差分残差为：

$$\delta_t = r_t + \gamma V_{\theta_i}(s_{t+1}) - V_{\theta_i}(s_t) \quad (3.9)$$

其中 $V_{\theta_i}(s_t)$ 为价值网络对状态 s_t 的价值估计， γ 为折扣因子。GAE 将优势函数定义为时序差分残差的指数加权和：

$$\hat{A}_t^{GAE} = \sum_{l=0}^{H-1-t} (\gamma \lambda_{GAE})^l \delta_{t+l} = \delta_t + \gamma \lambda_{GAE} \delta_{t+1} + (\gamma \lambda_{GAE})^2 \delta_{t+2} + \dots \quad (3.10)$$

当 $\lambda_{GAE} = 0$ 时， $\hat{A}_t = \delta_t$ 退化为单步时序差分估计，偏差较高但方差较低；当 $\lambda_{GAE} = 1$ 时， $\hat{A}_t = \sum_{l=0}^{H-1-t} \gamma^l \delta_{t+l} = G_t - V_{\theta_i}(s_t)$ 等价于蒙特卡洛回报减去基线的形式，偏差为零但方差较大。通过选取适中的 λ_{GAE} （通常取 0.99），GAE 在两者之间取得平衡，为策略梯度提供低方差且近似无偏的优势估计。目标回报 G_t 由递推计算得到： $G_t = r_t + \gamma G_{t+1}$ 。

价值网络 $V_{\theta_i}(s_t)$ 用于估计状态的期望回报，其训练目标为最小化预测值与实际回报之间的均方误差：

$$L^{value} = \frac{1}{2} (V_{\theta_i}(s_t) - G_t)^2 \quad (3.11)$$

其中 $G_t = \sum_{l=0}^{H-1-t} \gamma^l r_{t+l}$ 为从时刻 t 开始的折扣累积回报。准确的价值估计为 GAE 提供可靠的基线，降低策略梯度的方差。

为鼓励策略保持充分的探索能力、避免过早陷入局部最优，在损失函数中加入

策略熵正则项:

$$L^{entropy} = - \sum_a \pi_{\theta_i}(a|s_t) \log \pi_{\theta_i}(a|s_t) \quad (3.12)$$

较大的熵值意味着策略的动作分布更加均匀, 有助于智能体在训练初期广泛探索状态-动作空间。随着训练的推进, 策略逐渐收敛到高回报的动作上, 熵值自然降低。

为使策略更新统一采用梯度下降的形式, 将上述各项组合为需要最小化的综合损失函数:

$$L(\pi_{\theta_i}) = -L^{clip}(\pi_{\theta_i}) + c_1 L^{value} - c_2 L^{entropy} \quad (3.13)$$

其中 c_1, c_2 为超参数, c_1 控制价值函数损失的权重, c_2 控制熵正则项的强度。最小化 $L(\pi_{\theta_i})$ 等价于同时最大化裁剪策略目标与策略熵、最小化价值估计误差, 从而在策略性能、探索程度和价值精度三方面取得平衡。

在内循环中, 对于采样的每个任务 T_i , 以元参数 θ_{meta} 为起点初始化任务参数 θ_i , 使用当前策略 π_{θ_i} 在任务 T_i 的环境中采集 M 条轨迹; 计算 PPO 损失 $L(\pi_{\theta_i})$, 执行梯度更新: $\theta_i \leftarrow \theta_i - \alpha \nabla_{\theta_i} L(\pi_{\theta_i})$ 。经过 K 步更新后, 得到任务 T_i 的适应参数 θ'_i 。该参数与当前任务环境进行交互, 收集轨迹作为查询集。

外循环汇集所有任务在各自查询集上的 PPO 损失, 将其求和构成元损失:

$$L_{meta} = \sum_i L(\pi_{\theta'_i}) \quad (3.14)$$

其中 $L(\pi_{\theta'_i})$ 是使用适应后参数 θ'_i 在查询集上计算的 PPO 损失。元梯度的计算需要将 L_{meta} 对元参数 θ_{meta} 求导。由于 θ'_i 是从 θ_{meta} 出发经 K 步梯度更新得到的, 本文在外循环中采用完整的二阶元梯度进行更新 (详见第2.2.2节式(2.12)), 其形式为:

$$\nabla_{\theta_{meta}} L_{meta} = \sum_i \nabla_{\theta'_i} L(\pi_{\theta'_i}) \cdot \frac{\partial \theta'_i}{\partial \theta_{meta}} \quad (3.15)$$

据此更新元参数:

$$\theta_{meta} \leftarrow \theta_{meta} - \beta \cdot \nabla_{\theta_{meta}} L_{meta} \quad (3.16)$$

其中 β 为元学习率。通过多轮内外循环迭代, 模型最终收敛获得了一组高质量的元参数, 这组参数具备跨任务类型的泛化能力, 通过后续的适应算法可以快速得到近似最优的策略网络。MRL 的训练算法伪代码如算法1所示。

算法 1 MRL 训练算法

Input: 元策略网络参数初始化值 Φ ; 元学习率 β ; 内循环学习率 α

- 1: 任务分布 $p(\mathcal{T})$; 训练步数 H ; 内循环梯度步数 K
- 2: 每个任务的支持集样本数 M ; 查询集样本数 N

Output: 元策略网络参数 θ_{meta}

```

3: for  $h = 1, 2, 3, \dots, H$  do
4:   采样一批任务:  $\{T_i\}_{i=1}^B \sim p(\mathcal{T})$ 
5:   for 每个任务  $T_i$  do
6:     // 内循环
7:     复制当前元参数至任务网络:  $\theta_i \leftarrow \theta_{meta}$ 
8:     for  $k = 1$  to  $K$  do
9:       使用策略  $\pi_{\theta_i}$  在任务  $T_i$  中收集  $M$  条轨迹
10:      构建支持集  $\mathcal{D}_{sup}^{(i)}$ 
11:      基于  $\mathcal{D}_{sup}^{(i)}$  计算 PPO 损失:  $L(\pi_{\theta_i})$ 
12:      执行内循环梯度更新:  $\theta_i \leftarrow \theta_i - \alpha \nabla_{\theta_i} L(\pi_{\theta_i})$ 
13:    end
14:    使用更新后的策略  $\pi_{\theta_i}$  在任务  $T_i$  中收集  $N$  条轨迹
15:    构建查询集  $\mathcal{D}_{que}^{(i)}$ 
16:    计算查询集损失:  $L_{qry}(\pi_{\theta_i})$ 
17:  end
18:  聚合所有任务查询集损失, 构建元损失:
19:     $L_{meta}(\theta_{meta}) = \sum_{i=1}^B L_{qry}(\pi_{\theta_i})$ 
20:  计算二阶元梯度:  $g_{meta} = \nabla_{\theta_{meta}} L_{meta}$ 
21:  执行元更新:  $\theta_{meta} \leftarrow \theta_{meta} - \beta \cdot g_{meta}$ 
22: end

```

3.3.2 基于特征调控模块的策略网络输入

当前基于 MRL 框架的训练算法, 其梯度更新依赖于一个共享的策略网络, 通过内外循环机制从多类型车间任务中聚合经验以优化元参数。然而, 在这种更新方式

下，例如作业车间调度任务中一些机器节点的特征对训练决策的影响很小，但由于网络结构的固有设计，策略网络在训练初期会利用所有输入特征进行探索；同时由于策略梯度算法的信用分配模糊性，容易将作业车间任务轨迹中获得的正面或负面奖励，错误地归因于与该轨迹偶然关联的机器节点特征。这种误判导致在内循环训练作业车间任务时，在机器节点特征上产生方向不定的噪声梯度。这些噪声梯度会在外循环的元参数更新过程中形成干扰，不仅降低学习效率，还可能引发不同任务间知识的负迁移，从而削弱元模型的整体泛化能力。因此，为了提高元强化学习模型对不同类型问题的适应能力，提高整体模型的泛化能力，引入和车间类型有关的特征调控模块（Feature-wise Linear Modulation, FiLM），FiLM 将条件信息通过一个小型 MLP 生成一组调制参数，对每个特征通道进行缩放和平移，增强模组对任务的适应性。通过根据任务的条件信息抑制无关特征、增强关键特征，阻断了无关信息的干扰，并在反向传播中消除因为噪声梯度导致的冲突，使得元参数的更新能够最大限度的提高泛化能力。

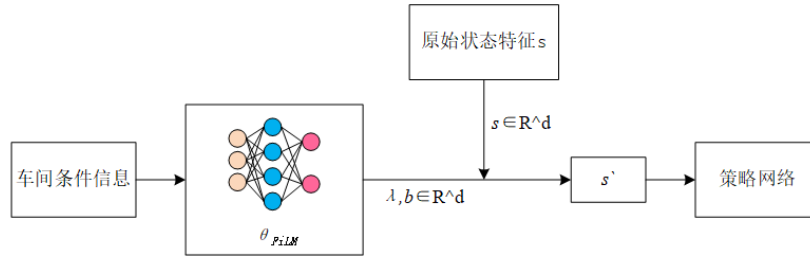


图 3.2 FiLM 特征调控机制

FiLM 的示意图如图3.2所示。该机制的目标是学习一个映射函数 $f_{\theta_{FiLM}}$ ，能够根据输入的车间类型特征向量 $type_i$ ，动态生成最优的特征调制参数 $(\lambda, b) = f_{\theta_{FiLM}}(type_i)$ 。其中 θ_{FiLM} 表示 FiLM 的可训练参数。该映射函数的优化目标是为下游的策略网络 π_{θ_i} 提供经过调制的状态表征 $s' = \lambda \odot s + b$ ，以最大化长期累积奖励。

结合第 3.3.1 节的训练算法，FiLM 的参数更新同样通过反向传播实现，梯度从奖励信号反向流经整个计算图。对于 FiLM 参数的梯度计算如下：

$$\nabla_{\theta_{FiLM}} L(\pi_{\theta_i}) = \frac{\partial L(\pi_{\theta_i})}{\partial s'} \cdot \frac{\partial s'}{\partial (\lambda, b)} \cdot \frac{\partial (\lambda, b)}{\partial (\theta_{FiLM})} \quad (3.17)$$

如图3.3所示，训练初始阶段，FiLM 的参数采用小随机初始化，使得输出的 $\lambda = 1$, $b = 0$ 。此时特征调制近乎恒等变换，策略网络接收近乎原始的特征输入。

随后策略网络与不同车间类型的实例产生多样化的轨迹和奖励信号。随着训练进行，FiLM 开始学习根据车间类型信息进行梯度信号的条件化分离。在作业车间场景中，机器选择的约束通过动态掩码机制实现，机器相关特征对决策质量的影响较弱；而在柔性作业车间中，机器选择的灵活性大幅提高，机器特征成为优化调度的关键信息。这种环境固有的差异，通过奖励信号的反向传播，被转化为不同的梯度模式，具体表现为对流水车间： $\frac{\partial L(\pi_{\theta_i})}{\lambda_{\text{machine}}} \approx 0$, $\frac{\partial L(\pi_{\theta_i})}{\lambda_{\text{option}}} < 0$ ，而对于柔性作业车间： $\frac{\partial L(\pi_{\theta_i})}{\lambda_{\text{machine}}} < 0$, $\frac{\partial L(\pi_{\theta_i})}{\lambda_{\text{option}}} < 0$ ，FiLM 模块通过差异化的特征调制策略进行自身参数的更新。

特征调控模块的输入为与车间类型有关的调度信息，本文采用如下特征描述车间类型的信息， $\text{type}_i = [Fr(t), H_q(t), CV(t)]$ 。

(1) 剩余工序机器柔性指数 $Fr(t)$ ：统计时间步 t 的未加工工序，计算它们可选机器数的平均值，并除以车间总机器数进行标准化，用于衡量剩余工序的柔性程度。

(2) 队列长度分布熵 $H_q(t)$ ：观察当前时刻每台机器可加工工序的数量，将此数量分布视为一个概率分布，计算其香农熵，体现了当前机器的负载分布。

(3) 工件进度系数 $CV(t)$ ：反映当前所有工件的进度是否同步推进。计算当前每个工件的进度完成率（已完成工序数/总工序数），然后计算所有工件进度完成率的标准差与平均值的比值。

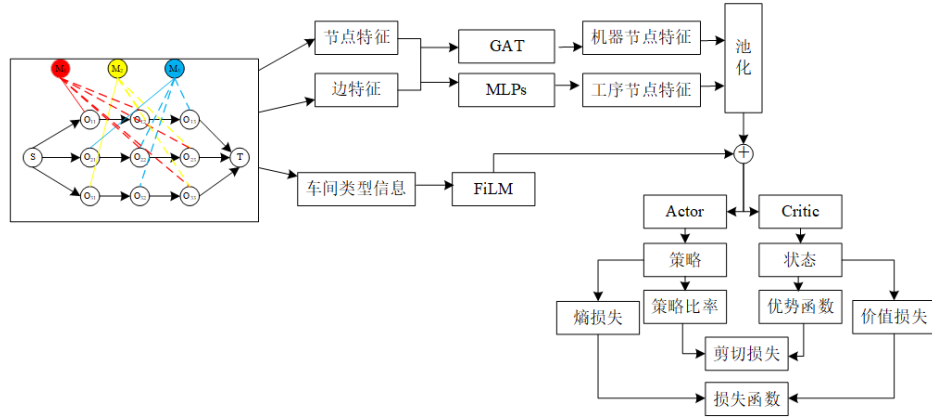


图 3.3 FiLM 下的网络结构

3.3.3 MRL 适应算法

适应阶段是元强化学习模型完成训练阶段后，针对待求解的车间调度任务进行快速部署与应用。如图3.4所示，该阶段利用训练阶段获得的元参数 θ_{meta} 作为初始

化起点，与目标车间环境进行初步交互，生成一个小规模的轨迹数据集。以元参数 θ_{meta} 作为初始化的任务参数 θ_{adp} ，并基于上述微调数据集计算策略损失，通过执行的几步梯度下降更新，对策略网络进行更新，迅速收敛到接近当前车间问题最优策略的参数 θ_{adp}^* 。随后基于调整后的参数 θ_{adp}^* 求解当前问题生成调度方案。

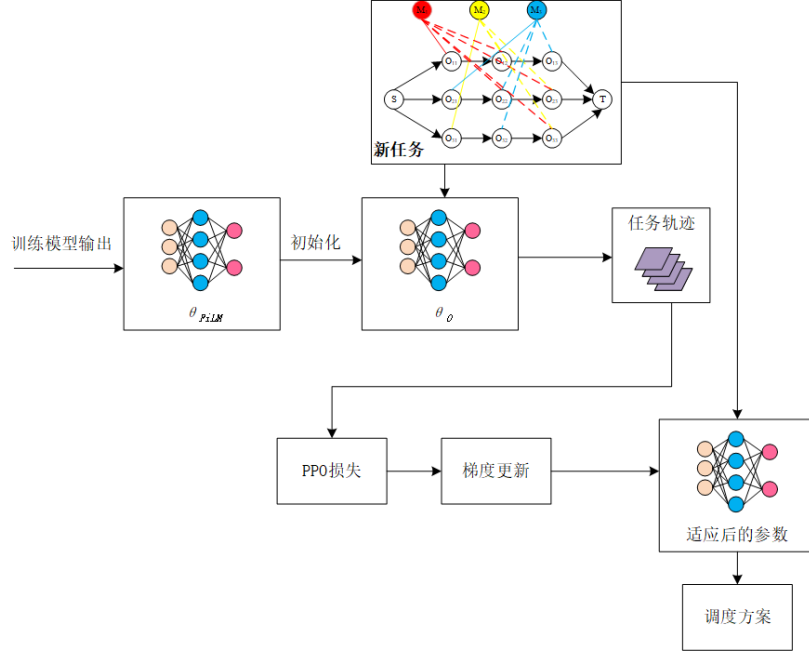


图 3.4 MRL 适应算法部分

算法 2 MRL 适应算法

Input: 训练完成的元参数 θ_{meta} ；适应学习率 α_{adp} ；适应步数 K_{adp} ；每步采样轨迹数 M_{adp} ；目标任务 T_{new}

Output: 适应后的任务参数 θ_{adp}^* 及对应调度方案

- 1: 初始化任务参数: $\theta_{adp} \leftarrow \theta_{meta}$
- 2: **for** $k = 1, 2, \dots, K_{adp}$ **do**
- 3: 使用策略 $\pi_{\theta_{adp}}$ 在任务 T_{new} 上采样 M_{adp} 条轨迹，构建适应数据集 $\mathcal{D}_{adp}$
- 4: 基于 $\mathcal{D}_{adp}$ 计算适应损失: $L_{adp}(\pi_{\theta_{adp}})$
- 5: 执行梯度更新: $\theta_{adp} \leftarrow \theta_{adp} - \alpha_{adp} \nabla_{\theta_{adp}} L_{adp}(\pi_{\theta_{adp}})$
- 6: **end**
- 7: 得到适应后参数: $\theta_{adp}^* \leftarrow \theta_{adp}$
- 8: 使用策略 $\pi_{\theta_{adp}^*}$ 在任务 T_{new} 上生成最终调度方案

3.4 实验与结果分析

为系统验证所提 MRL 方法的有效性, 本节从数据集构建、参数调优、模块消融以及多维度求解对比四个层面展开实验与分析。首先, 介绍数据集生成方式、实验环境配置, 以及元学习关键超参数的敏感性分析过程与最终取值。其次, 通过消融实验验证 FiLM 特征调控模块的效果。在此基础上, 分别采用随机算例与标准算例对 MRL 进行全面的对比评估。在随机算例实验中, 针对三类问题, 在 10×5 至 30×20 的 8 个规模上各生成多个独立测试算例, 将 MRL 与经典调度规则以及 PPO 模型进行对比, 通过大规模的多样化算例, 综合评估方法的整体性能, 同时通过跨类型交叉求解实验考察多类型任务联合训练相较于单类型训练在知识迁移与泛化方面的优势。在特定分布的标准算例实验中, 与现有文献的结果进行对比, 评估 MRL 的求解水平及其通用性。

3.4.1 数据集与参数设置

为实现面向多类型车间的元强化学习训练与评估, 所用数据集包括训练集、验证集与测试集。训练集用于执行元训练过程, 即模型在此数据集上进行内外循环迭代, 学习跨任务的通用调度知识与快速适应能力。验证集在元训练过程中保持不变, 不参与梯度更新, 其作用是在每个训练周期后评估模型的元性能。测试集则用于对训练完成的模型进行性能评估, 测试其面对新任务时的泛化与快速适应能力。

为构建适用于元强化学习训练的数据集, 分别针对 JSP, FJSP 及 NPFSP 三类典型场景生成随机实例。假设一批训练问题的规模为 $n \times m$, 各类型车间的规模参数共用, 工件数量和机器数量分别设置为 $[0.8n, 1.2n]$ 和 $[0.9m, 1.1m]$, 同时根据问题类型设定其他参数: 对于 JSP 与 FJSP, 各工件工序数 h_i 在一定区间内可变; 对于 NPFSP, 工序数固定等于机器数; 对于 FJSP, 还需设定工序的可选机器数量区间。具体区间设置如表 3.1 所示。所有参数均从其对应的参数的均匀分布中独立采样, 数值经取整处理。这样通过更改 n 和 m , 可以为训练算法生成不同规模的数据集。

训练初期先使用小规模实例 ($n=10, m=5$), 让模型首先学习基础的调度逻辑。随着训练进行, 逐步引入更大规模的实例, 以提升模型处理复杂问题的能力。在任一训练阶段, 三种车间类型问题的比例保持为 1:1:1, 确保模型能均衡的学习各类车间的优化策略, 避免对某一类型过拟合。验证集的同样由三种车间类型的问题构成,

表 3.1 多类型车间训练数据集参数区间

数据	JSP	FJSP	NPFSP
工件数量 N	$[0.8n, 1.2n]$	$[0.8n, 1.2n]$	$[0.8n, 1.2n]$
机器数量 M	$[0.9m, 1.1m]$	$[0.9m, 1.1m]$	$[0.9m, 1.1m]$
工件的工序数 h_i	$[0.8M, 1.2M]$	$[0.8M, 1.2M]$	M
工序的加工时间平均值 $\tilde{p}_{i,j}$	$[1, 20]$	$[1, 20]$	$[1, 20]$
工序的可加工机器数 $m_{i,j}$	1	$[1, M]$	1
对应机器上的加工时间 p_{i,j,m_k}	$\tilde{p}_{i,j}$	$[0.8\tilde{p}_{i,j}, 1.2\tilde{p}_{i,j}]$	$\tilde{p}_{i,j}$

比例和涵盖的规模与训练集相同。在整个元训练期间，验证集保持固定不变，用于在训练过程中合理地评估模型的性能。

本文所涉及的算法均基于 Python 3.10 编程语言实现，使用环境库 gym 0.22.0 与深度学习框架 PyTorch 2.5.0 进行算法开发与模型训练。实验在一台搭载 AMD Ryzen 5 7600 处理器与 NVIDIA GeForce RTX 4060 GPU 的计算机上完成。

针对元学习率 (β)、内循环学习率 (α) 和内层适应步数 (K) 等对模型性能影响较大的关键参数，进行敏感性分析从而获得最优参数组合，构建一个由规模 10×5 和 15×10 问题组成的训练集 A，记录不同参数下训练的模型在固定验证集的效果。

图3.5中，当元学习率 β 设置为 0.001 时，模型在验证集上的表现呈现较大波动，其性能曲线在前中期迭代中下降缓慢且振荡明显，整体收敛速度较慢，且最终稳定后的性能水平仍不理想。这表明元学习率 0.001 导致参数更新步长过大，不仅使优化过程难以稳定收敛，也阻碍了模型寻找多类型车间调度任务的参数平衡点。元学习率 0.0002 和 0.0001 对应的曲线轨迹较为平滑稳定，同时收敛较快，但是较小的学习率可能导致模型陷入局部最优。因此，综合考虑 $\beta=0.0002$ 。

内循环学习率 (α) 对比实验如图3.6所示， $\alpha=0.001$ 对应的曲线显示模型在验证集上的性能在初始阶段快速下降，但整个训练过程中存在明显的震荡，表明模型在内循环时更新步幅过大，导致在单问题上过度拟合影响了在验证集的效果。当 α 设置为 0.0001 时（蓝色曲线），其 makespan 的下降过程虽然平稳，但收敛速度过于缓慢。在相当长的迭代周期内，其性能提升幅度有限，这意味着模型针对新任务的快速适应效率低下，元训练过程趋近于普通的、低效的多任务联合学习，无法体现元学习的优势。 $\alpha=0.0002$ 时，对应曲线的下降平稳。这表明该学习率能使模型在针

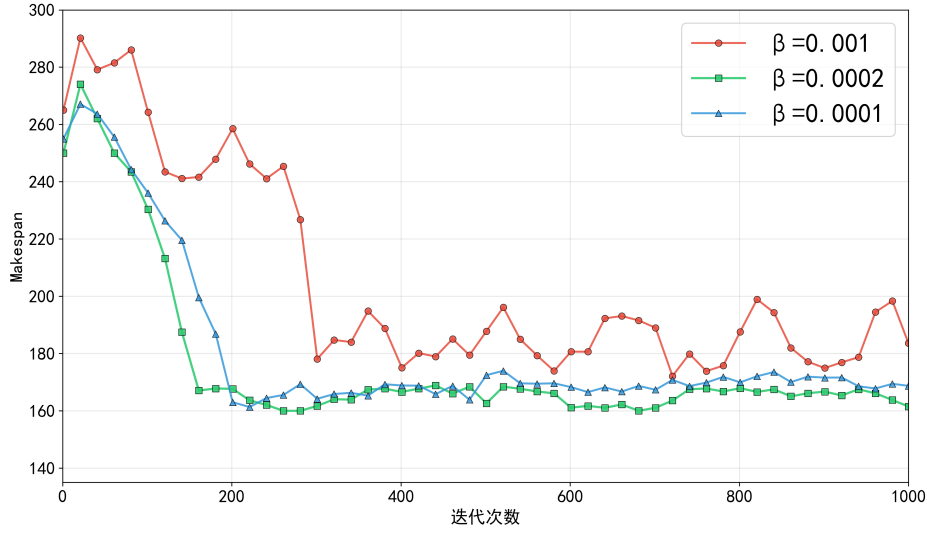


图 3.5 元学习率对比实验

对单个任务进行内循环适应时，能做出足够有效、快速的参数调整，同时能避免因更新过度而产生的震荡与过拟合风险。相比之下 $\alpha=0.0001$ 的收敛速度较慢，表明当前学习率的内循环参数调整幅度较小，导致外循环的梯度不明显。因此内循环学习率设置为 0.0002。

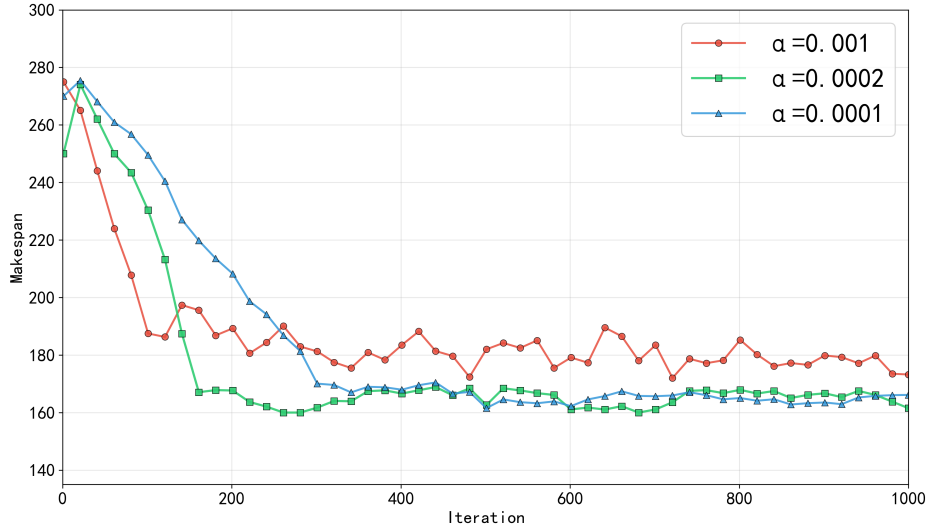


图 3.6 内循环学习率对比实验

内循环步数 (K) 对比实验如图3.7所示，四条曲线的下降均比较平稳。其中， $K=3$ 对应曲线的收敛速度最慢，表明过少的梯度更新步数限制了模型在训练任务上

的快速调整能力。 $K=4$ 对应的曲线收敛速度有所加快，但仍未达到最优效率。迭代步数增加到 $K=5$ 与 $K=6$ 时，收敛速度明显提升。但是由于元学习中，外循环更新元参数时需要计算二阶导数，要求完整保留内循环 K 步优化的计算图。 K 值增加会导致内存消耗与计算成本会大幅上升，因此综合考虑 $K=5$ 以平衡收敛速度和计算成本。其余超参数设置为：训练步数 1000、每批任务数 6、裁剪系数 0.2、价值函数系数 0.5、熵奖励系数 0.1、支持集轨迹数 3、查询集轨迹数 5、GNN 层数 $L = 2$ 。

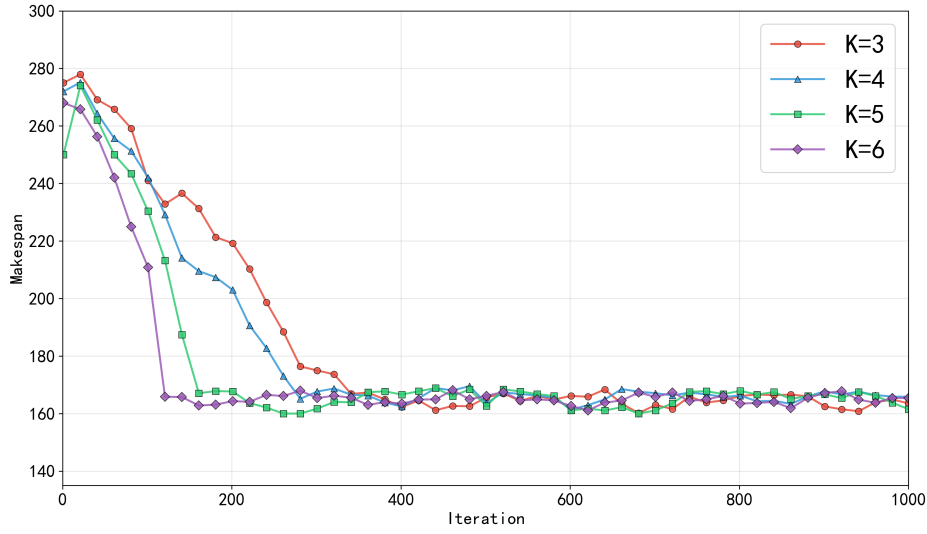


图 3.7 内循环步数对比实验

3.4.2 FiLM 有效性验证

FiLM 模块通过接收车间类型信息作为条件输入，动态生成调制参数 (λ, b) ，引导模型关注对当前任务类型最重要的特征，从而提升模型在训练稳定性、跨任务泛化及快速适应等方面的性能。为了验证 FiLM 模块在多类型车间调度元强化学习框架中的作用，对照组设置为特征调控信号 (λ, b) 固定为 1 和 0，即不对特征进行任何调制，其他结构与训练设置与原框架一致。对照组和原框架分别在 3.3.2 节构建的训练集 A 上进行训练，得到的模型分别记为 MRL-n 和 MRL-F，记录训练过程中模型在验证集上的性能变化，如下图 3.8 所示。

引入 FiLM 模块的 MRL-F 模型下降较快，在约 240 步时收敛并在后续训练中保持稳定，相比之下，未引入 FiLM 的 MRL-n 模型收敛较慢，在约 320 步时收敛，且收敛的 makespan 水平较差，后期波动明显。这表明 FiLM 通过动态调节特征表示，

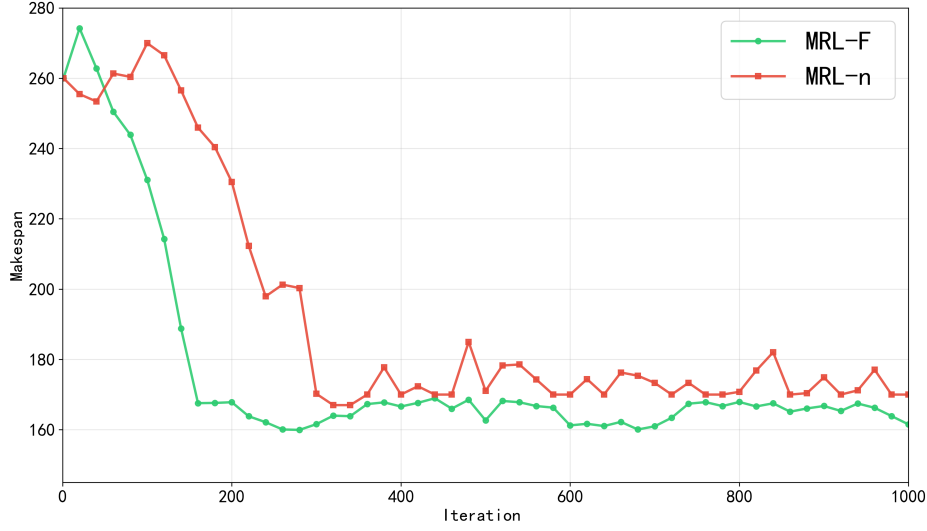


图 3.8 FiLM 模块对比实验

有效加速了模型收敛过程，同时明显降低了训练过程中的波动，提升了学习稳定性与效率。FiLM 模块通过对特征的动态线性调制，使得元强化学习模型能够有效地掌握多种类型车间的调度知识，增强了模型训练过程的稳定性。

3.4.3 随机算例结果分析

为了验证所训练的元强化学习模型在求解多种类型调度问题上的泛化能力与有效性，首先在自合成的数据集上进行对比实验，以初步评估模型的性能。对比算法选取经典的调度规则和采用单一类型车间数据训练的 PPO 模型。为清晰呈现实验的设计与结果，以下首先详细说明适用于所有实验部分的模型训练步骤、对比基线与评估指标。

MRL 训练步骤：训练数据由 NPFSP、JSP 和 FJSP 三类车间问题共同构成。训练初期从 10×5 小规模实例开始，使模型先学习基本调度规律；随后逐步引入更大规模实例，采用课程式扩展方式提升模型的学习能力。考虑训练效率与算力成本，训练阶段最大规模设置为 20×10 。

在此基础上，在以下对比基线在多种类型车间问题上与所提模型对比：

采用经典启发式调度规则作为快速求解的对比基线，包括以下规则：SPT (Shortest Processing Time, 优先选择当前加工时间最短工序)、FIFO (First In First Out, 按到达顺序)、MWKR (Most Work Remaining, 优先剩余总工时最大的工件)

和 LWKR (Least Work Remaining, 优先剩余总工时最小的工件)。在 FJSP 中, 采用工序选择规则 + 机器选择规则的组合形式, 工序的规则沿用提到的四个规则, 机器选择规则为 MOR (Earliest Available Machine, 最早可用机器) 和 SPT_{ma} (Shortest Processing Time on Machine, 候选机器中加工时间最短)。

先进的优化和学习方法, 包括采用强化学习自适应调整参数的自学习遗传算法 (SLGA)^[38], 针对 NPFSP/JSP 问题, 将其原有编码/解码机制替换为相应的编码/解码机制; 针对三类车间调度问题分别训练的专用 PPO 模型, 这些模型采用与 MRL 方法完全一致的网络结构及关键超参数; 以及当前文献中针对 NPFSP/JSP/FJSP 三类问题较为有效的优化算法。

所有实验均以最小化最大完工时间为优化目标。后文采用 Makespan 表示某个算例的完工时间或者某个规模中所有算例的平均完工时间。为衡量不同方法在同一算例上的相对解质量, 采用 RPD (Relative Percentage Deviation) 表示当前解相对最优解的偏差水平, 相对偏差指标 RPD 计算公式如下:

$$RPD(\%) = \frac{C - C^*}{C^*} \times 100\% \quad (3.18)$$

其中 C 为待评价方法在该算例上的均值结果, C^* 为该算例的已知最优解或者所有算法中的最优解。RPD 越小, 说明方法越接近最优解。

采用训练集的生成方式构造随机算例, 规模范围为 10×5 到 30×20 , 具体采用 8 个规模, 每个规模包含 50 个独立测试算例。

(1) NPFSP 随机算例。

本节在随机生成的不同规模的 NPFSP 算例上进行对比实验, 评估基于多类型数据联合训练的元强化学习模型在单一问题上的求解效果。表3.2给出各方法在随机 NPFSP 测试集上的 Makespan 结果。

表3.2的结果表明, MRL 模型在 NPFSP 随机算例上的求解质量整体优于经典调度规则。从求解质量看, 四类规则方法的 RPD 位于 14.68%–24.95% 范围内, 而 MRL 在 8 组规模上的 RPD 仅为 2.04%–4.17%, 平均 RPD 为 3.33%, 说明所提的 MRL 模型可以从多种类型问题构成的数据集中学习到有效的调度知识, 求得高质量解。进一步地, 与采用单一类型数据训练的 PPO-NPFSP 相比, MRL 在求解质量上存在约 3.33% 的微小差距, 这表明 MRL 模型在多类型问题的联合训练过程中提炼共性调度知识, 并在求解阶段通过少量交互完成快速适应, 从而使最终性能逼近专用模型,

表 3.2 NPFSP 随机算例结果对比

规模	指标	SPT	FIFO	MWKR	LWKR	MRL	PPO-NPFSP
10×5	M	172.04	189.18	188.96	186.74	154.84	150.19
	R	14.55	25.96	25.81	24.34	3.10	0.00
	T	0.039	0.037	0.057	0.060	1.20	0.95
15×5	M	229.48	251.24	247.28	251.96	208.83	204.65
	R	12.14	22.76	20.84	23.12	2.04	0.00
	T	0.075	0.065	0.105	0.116	1.45	1.15
15×10	M	308.20	334.40	336.08	333.92	274.30	263.33
	R	17.00	26.98	27.63	26.81	4.17	0.00
	T	0.122	0.119	0.212	0.233	1.95	1.55
20×5	M	284.80	315.38	316.94	314.12	256.32	248.63
	R	14.55	26.85	27.47	26.34	3.09	0.00
	T	0.109	0.103	0.168	0.190	1.85	1.45
20×10	M	373.92	408.00	400.34	404.40	329.05	313.58
	R	19.24	30.11	27.67	28.96	4.93	0.00
	T	0.197	0.188	0.340	0.382	2.40	1.85
30×10	M	491.90	540.28	536.56	535.40	447.63	438.68
	R	12.13	23.16	22.31	22.05	2.04	0.00
	T	0.384	0.360	0.948	0.786	3.20	2.45
30×15	M	586.26	633.38	623.94	631.78	527.63	506.52
	R	15.74	25.05	23.18	24.73	4.17	0.00
	T	0.622	0.514	1.025	1.228	4.10	3.10
30×20	M	670.56	710.56	712.04	712.76	616.92	598.41
	R	12.06	18.74	18.99	19.11	3.09	0.00
	T	0.684	0.659	1.434	1.721	5.20	3.95
Average	M	389.65	422.80	420.27	421.39	351.94	340.50
	R	14.68	24.95	24.24	24.43	3.33	0.00
	T	0.279	0.256	0.536	0.589	2.67	2.05

注：M 表示 Makespan，R 表示 RPD (%)，T 表示 Time (s)。

没有发生对 NPFSP 问题的失效。从计算效率看，MRL 平均求解时间为 2.67s，虽然高于启发式规则，但仍处于可接受范围；相较 PPO-NPFSP 的 2.05s，时间开销约增加 30%。这一差异的主要原因在于 MRL 在求解中包含适应阶段，需要以元参数为初始化，通过少量轨迹交互与梯度更新得到面向当前实例的策略参数，再输出最终调度方案。综上，MRL 模型通过元强化学习框架和少量的额外计算时间，在 10×5 至 30×20 规模下的 NPFSP 上表现出良好的求解质量和速度。

为进一步展示不同方法在求解质量上的分布差异，图3.9给出了 NPFSP 在 20×10 规模下各方法求解结果的分布图。由图可见，在同分布随机生成算例条件下，各方法解的极差整体接近；但 MRL 的箱体区间位置明显低于各调度规则，说明其在该规模下具有更优且更稳定的解质量。与此同时，MRL 与专用模型 PPO-NPFSP 的分布存在较大重叠，进一步验证所提模型在 NPFSP 上的良好求解性能。

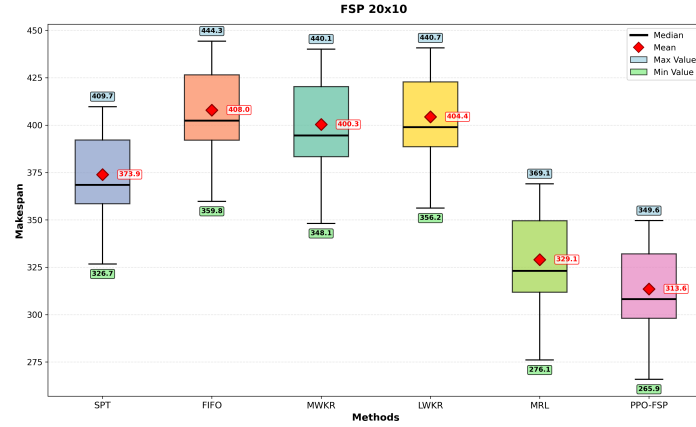


图 3.9 NPFSP 随机实验在 20×10 规模下的求解结果箱体图

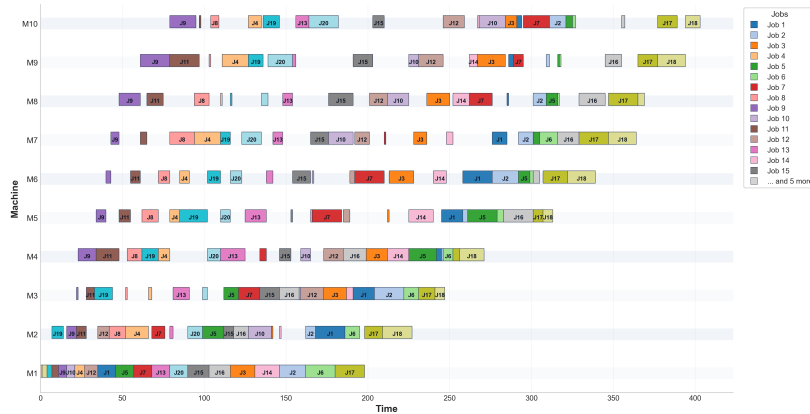


图 3.10 20×10 的 NPFSP 随机算例的 SPT 求解结果甘特图

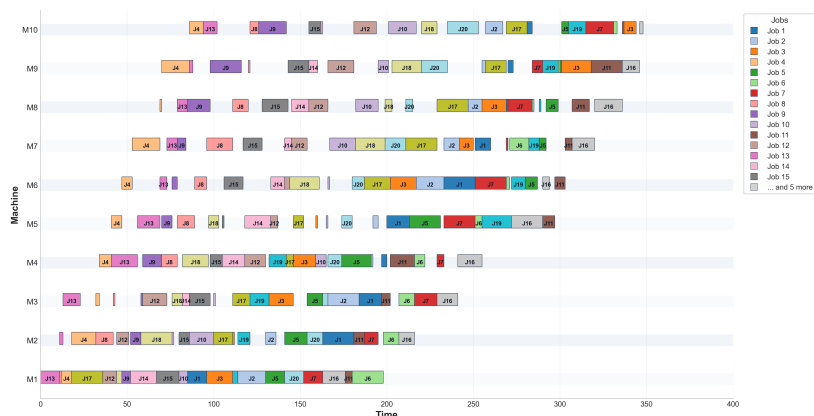


图 3.11 20×10 的 NPFSP 随机算例的 MRL 求解结果甘特图

图3.10与图3.11分别是调度规则中的 SPT 和 MRL 模型求解 NPFSP 20×10 规模的第一个随机算例的甘特图。MRL 生成的调度方案在结构层面依旧优于 SPT 规则：机器上的空闲间隔更短，工序的衔接更加紧凑，体现出更高的调度水平。

(2) JSP 随机算例。

不同求解算法对 JSP 随机算例的求解结果如表3.3所示，所提 MRL 在求解质量上依旧保持明显优势。从求解质量看，MRL 的平均 RPD 为 3.10%，而各调度规则的 RPD 位于 13.87%–41.52% 区间。与专门使用 JSP 数据训练的 PPO-JSP 相比，MRL 在 RPD 上存在约 3.10% 的差距，表明所提模型在 JSP 上依旧保持着较高的求解质量。在求解速度方面，MRL 模型的求解时间大于其他求解算法，但仍处于可接受范围。结合 NPFSP 的实验结果，模型性能未因问题类型由 NPFSP 切换至 JSP 而出现明显劣化，体现出良好的泛化能力。

表 3.3 JSP 随机算例结果对比

规模	指标	SPT	FIFO	MWKR	LWKR	MRL	PPO-JSP
10×5	M	152.68	147.24	145.08	176.60	132.02	127.93
	R	19.35	15.10	13.41	38.05	3.20	0.00
	T	0.038	0.038	0.057	0.055	1.10	0.88
15×5	M	213.90	210.12	203.78	245.20	184.40	179.24
	R	19.34	17.23	13.69	36.80	2.88	0.00
	T	0.068	0.071	0.106	0.103	1.35	1.08

续表 3.3 JSP 随机算例结果对比

规模	指标	SPT	FIFO	MWKR	LWKR	MRL	PPO-JSP
15 × 10	M	272.20	267.14	249.34	309.46	225.65	218.88
	R	24.36	22.05	13.92	41.38	3.09	0.00
	T	0.120	0.123	0.209	0.222	1.90	1.50
20 × 5	M	271.80	263.66	258.06	312.78	233.03	226.04
	R	20.24	16.63	14.15	38.37	3.09	0.00
	T	0.110	0.115	0.172	0.164	1.70	1.35
20 × 10	M	324.96	318.86	296.88	373.42	269.05	261.24
	R	24.39	22.06	13.65	42.94	2.99	0.00
	T	0.198	0.201	0.337	0.335	2.30	1.80
30 × 10	M	435.18	425.42	407.58	506.62	365.60	355.37
	R	22.46	19.71	14.68	42.56	2.88	0.00
	T	0.390	0.405	0.676	0.662	3.00	2.35
30 × 15	M	485.90	482.02	449.54	573.18	408.53	392.19
	R	23.90	22.91	14.62	46.15	4.17	0.00
	T	0.537	0.552	1.034	1.048	4.50	3.45
30 × 20	M	540.20	546.50	494.94	640.16	449.40	438.62
	R	23.15	24.59	12.84	45.94	2.46	0.00
	T	0.679	0.698	1.420	1.493	5.20	3.95
Average	M	337.10	332.62	313.15	392.18	283.46	274.94
	R	22.15	20.04	13.87	41.52	3.10	0.00
	T	0.268	0.275	0.501	0.510	2.63	2.04

注：M 表示 Makespan，R 表示 RPD (%)，T 表示 Time (s)。

(3) FJSP 随机算例。

不同算法对 FJSP 的求解结果如表3.4所示。从求解质量看，表现最优的规则组合 MWKR+SPT_{ma} 的 RPD 为 9.96%，而 MRL 的 RPD 仅为 2.77%，说明所提方法在柔性作业车间场景下仍能保持较高的解质量优势。进一步地，与 PPO-FJSP 的 Makespan=240.68 相比，MRL 的平均完工时间接近，表明多类型联合训练并未导致明显性能退化。

从计算效率看，MRL 在 FJSP 上的平均求解时间为 3.502 秒，高于调度规则方法的 0.389–0.721 秒，相较 PPO-FJSP 增加约 49%。这一时间开销的增加主要源于 FJSP 相较 NPFSP 与 JSP 需要更多决策步骤，模型在适应阶段需完成更复杂的工序与机器

联合决策。总体而言，面对结构更复杂的 FJSP 问题，MRL 以可控的计算成本换取了稳定且良好的求解结果，体现出在多类型数据联合训练下对不同调度场景的强泛化能力。

表 3.4 FJSP 随机算例结果对比

方法	M	R	T
SPT _{op} +MOR	318.52	32.34	0.393
SPT _{op} +SPT _{ma}	307.89	27.93	0.389
FIFO+MOR	288.20	19.74	0.450
FIFO+SPT _{ma}	283.81	17.92	0.450
MWKR+MOR	265.12	10.15	0.706
MWKR+SPT _{ma}	264.66	9.96	0.721
LWKR+MOR	345.34	43.49	0.603
LWKR+SPT _{ma}	334.69	39.06	0.599
MRL	247.35	2.77	3.502
PPO-FJSP	240.68	0.00	2.350

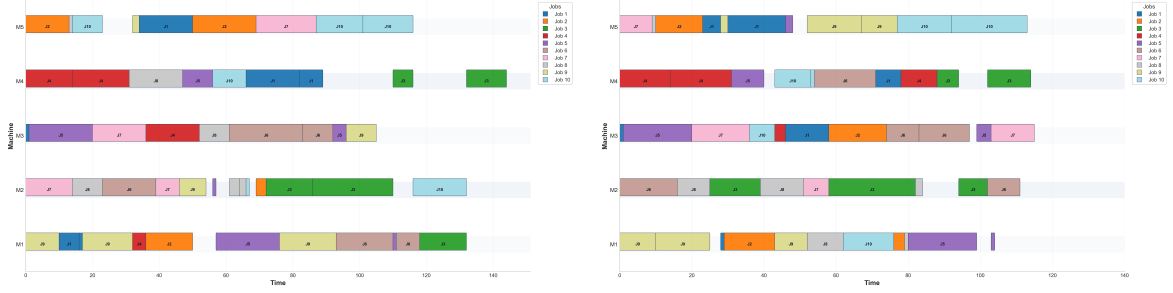
注：M 表示 Makespan，R 表示 RPD (%)，T 表示 Time (s)。

从 NPFSP、JSP 与 FJSP 三类问题的实验结果来看，MRL 模型在各项任务中均表现出稳定的求解性能，其求解质量优于调度规则。在计算效率方面，该模型在在线求解阶段的耗时高于调度规则及单任务类型训练的 PPO 模型，该差异主要源于适应阶段。同时，由于采用多任务联合训练及元学习双层优化机制，MRL 在离线训练阶段需付出高于普通 PPO 模型的成本，但该成本仅为一次性的前期投入。与 PPO 模型相比，MRL 在单一类型任务上的性能仅有小幅差距，但其优势在于能够同时学习多种类型问题，从而有效求解已见问题类型并保持良好的泛化能力。

表 3.5 跨类型训练-求解泛化对比

	MRL-FJSP	MRL-JSP	MRL
FJSP ($10 \times 5-30 \times 20$)	/	342.80	247.35
JSP ($10 \times 5-30 \times 20$)	334.19	/	283.46

尽管 JSP 和 NPFSP 在形式上可视为 FJSP 的特例，但这并不意味着三者的调度知识可以直接共用。原因在于不同调度问题的决策维度不同，直接迁移会出现知识



(a) MRL-JSP 模型在 FJSP 上的求解结果

(b) MRL 模型在 FJSP 的求解结果

图 3.12 10×5 的 FJSP 随机算例的不同模型求解结果

冗余或关键知识缺失。为验证上述观点，文中分别用单一的 FJSP 和 JSP 训练集训练元强化学习模型，得到 MRL-FJSP 和 MRL-JSP，并进行交叉求解实验。表3.5显示，两者均明显劣于多类型联合训练的 MRL，且 JSP 知识迁移到 FJSP 时性能损失更大。

图3.12分别给出了不同模型求解 FJSP 10×5 规模同一随机算例的甘特图。与图 (b) 相比，图 (a) 中基于 JSP 知识的模型在机器排序上具有一定合理性，机器间空闲时间较少、利用率较高；但其工序的机器选择较多依赖初始化参数，机器选择相关参数在训练中不更新或仅被动更新，导致机器选择的质量差，完工时间较长。相比之下，多类型联合训练模型能够更有效地处理工序与机器的联合决策，在图 (b) 中获得了更优的调度方案。

3.4.4 标准算例实验结果分析

根据随机实验结果，每类问题选取两种较优规则作为规则基线：NPFSP 选 SPT 与 FIFO，JSP 选 MWKR 与 FIFO，FJSP 选 MWKR+MOR 与 FIFO+SPT_{ma}。与其他对比算法一起进行对比实验。

(1) 标准算例下与通用求解方法的对比

本节选取求解过程与问题类型无关或者相关性小的方法进行对比，SLGA。这类方法的求解过程与问题类型无关，作为评估 MRL 跨类型泛化能力的参照。实验从公开算例中选取了覆盖不同尺度的算例。下文统一使用“算例 N”指代：NPFSP 测试集包含 Taillard 针对 FSP 问题提出的 ta01-04（算例 1-4）、ta11-14（算例 5-8）及 ta21-22（算例 9-10）；JSP 选取 Taillard 中 JSP 标准算例下的 ta01-05（算例 1-5）与 ta11-15（算例 6-10）；FJSP 测试集包含 Brandimarte 的 MK01-10（算例 1-10）。

表3.6给出了三类问题在规则与 SLGA 基线下的统一对比结果，由表可知，MRL

在 NPFSP、JSP 与 FJSP 三类标准算例上的平均 RPD 分别为 10.61%、25.07% 和 20.69%，均显著优于对应的调度规则基线。这表明 MRL 通过多类型联合训练习得的通用调度策略，在标准算例上依然能保持对传统规则方法的稳定优势，其泛化能力得到了进一步验证。

另一方面，SLGA 在 NPFSP、JSP 与 FJSP 上的平均 RPD 分别为 2.01%、21.04% 和 6.21%，整体优于 MRL。但 SLGA 作为元启发式算法，求解过程的计算成本远高于 MRL。综合而言，MRL 在多类型问题调度中实现了求解速度和解质量的平衡。

表 3.6 标准算例下与通用求解方法的对比

类型	算法	指标	算例 1	算例 2	算例 3	算例 4	算例 5	算例 6	算例 7	算例 8	算例 9	算例 10	平均 RPD(%)
NPFSP	Rule1	M	1448	1470	1480	1470	1950	2057	1863	1633	2716	2425	-
		R	13.30	8.25	37.93	13.78	25.00	25.12	25.37	19.37	18.45	15.92	20.25
	Rule2	M	1500	1545	1579	1608	2004	2104	1874	1726	2763	2514	-
		R	17.37	13.77	47.16	24.46	28.46	27.98	26.11	26.17	20.50	20.17	25.22
	SLGA	M	1306	1386	1094	1318	1592	1676	1518	1396	2336	2130	-
		R	2.19	2.06	1.96	2.01	2.05	1.95	2.15	2.05	1.88	1.82	2.01
	MRL	M	1353	1432	1208	1429	1724	1820	1738	1560	2477	2330	-
		R	5.87	5.45	12.58	10.60	10.51	10.71	16.96	14.04	8.02	11.38	10.61
	UB	M	1278	1358	1073	1292	1560	1644	1486	1368	2293	2092	-
JSP	Rule1	M	1872	1709	2009	1825	2044	2212	2414	2346	2109	2163	-
		R	52.07	37.38	64.94	55.32	65.64	63.01	76.59	74.68	56.80	65.49	61.19
	Rule2	M	1786	1944	1947	1694	1892	2117	2213	2026	2164	2180	-
		R	45.08	56.27	59.85	44.17	53.32	56.01	61.89	50.86	60.89	66.79	55.51
	SLGA	M	1442	1491	1483	1426	1429	1650	1708	1678	1601	1621	-
		R	17.14	19.86	21.76	21.36	15.80	21.59	24.95	24.94	19.03	24.03	21.04
	MRL	M	1571	1539	1608	1427	1453	1762	1640	1693	1697	1648	-
		R	27.62	23.71	32.02	21.45	17.75	29.84	19.97	26.06	26.17	26.09	25.07
	UB	M	1231	1244	1218	1175	1234	1357	1367	1343	1345	1307	-
FJSP	Rule1	M	46	44	211	75	192	94	202	531	338	263	-
		R	17.95	69.23	3.43	25.00	11.63	62.07	45.32	1.53	10.10	33.50	27.98
	Rule2	M	47	47	215	76	193	96	206	555	347	278	-
		R	20.51	80.77	5.39	26.67	12.21	65.52	48.20	6.12	13.03	41.12	31.95
	SLGA ^[38]	M	40	27	204	60	172	69	144	523	320	254	-
		R	2.56	3.85	0.00	0.00	0.00	18.97	3.60	0.00	4.23	28.93	6.21
	MRL	M	42	38	204	69	186	83	199	523	332	259	-
		R	7.69	46.15	0.00	15.00	8.14	43.10	43.17	0.00	8.14	31.47	20.69
	UB	M	39	26	204	60	172	58	139	523	307	197	-

注：M 表示 Makespan，R 表示 RPD (%)。

(2) 标准算例下与类型专用算法对比

为进一步评估 MRL 的性能，本部分选取三类问题的代表性算法进行对比。NPFSP 中包括基于 NPFSP 析取图的蚁群算法 IACO^[74]，与针对 NPFSP 最优解带有少量局部作业反转的特点提出的混合粒子群算法 HPSO^[75]；JSP 选取图神经网络与

强化学习结合的 GNN^[17]，以及结合深度卷积神经网络和双重 Q 网络的 DDQN^[15]；FJSP 选择 VND-hGA^[76] 与 GRL^[37]，前者针对 FJSP 的特点设计变异和邻域搜索操作，后者结合了图卷积网络和强化学习。这些算法针对问题特点进行设计，或采用某一问题类型进行训练，是当前相关标准算例上具有代表性的高效算法。表3.7汇总了三类标准算例在这组专用算法下的对比结果。

表 3.7 三类标准算例在专用模型与学习型算法下的统一对比结果

类型	算法	指标	算例 1	算例 2	算例 3	算例 4	算例 5	算例 6	算例 7	算例 8	算例 9	算例 10	平均 RPD(%)
NPFSP	IACO ^[74]	M	1387	1383	1203	1377	1681	1749	1554	1490	2428	2281	-
		R	8.53	1.84	12.12	6.58	7.76	6.39	4.58	8.92	5.89	9.03	7.16
	HPSO ^[75]	M	1290	1389	1100	1344	1693	1785	1583	1452	2396	2225	-
		R	0.94	2.28	2.52	4.02	8.53	8.58	6.53	6.14	4.49	6.36	5.04
	MRL	M	1353	1432	1208	1429	1724	1820	1738	1560	2477	2330	-
		R	5.87	5.45	12.58	10.60	10.51	10.71	16.96	14.04	8.02	11.38	10.61
JSP	UB	M	1278	1358	1073	1292	1560	1644	1486	1368	2293	2092	-
		R	12.22	24.12	18.23	39.32	31.20	32.20	32.04	43.86	23.72	32.36	29.43
	GNN ^[17]	M	1443	1544	1440	1637	1619	1794	1805	1932	1664	1730	-
		R	17.22	24.12	18.23	39.32	31.20	32.20	32.04	43.86	23.72	32.36	29.43
	DDQN ^[15]	M	1498	1608	1489	1675	1668	1856	1872	1995	1728	1786	-
		R	21.69	29.26	22.25	42.55	35.17	36.77	36.94	48.55	28.48	36.65	33.83
FJSP	MRL	M	1571	1539	1608	1427	1453	1762	1640	1693	1697	1648	-
		R	27.62	23.71	32.02	21.45	17.75	29.84	19.97	26.06	26.17	26.09	25.07
	VND-hGA ^[76]	M	1231	1244	1218	1175	1234	1357	1367	1343	1345	1307	-
		R	2.56	0.00	0.00	0.00	0.58	1.72	0.00	0.00	0.00	2.03	0.69
	GRL ^[37]	M	40	27	204	64	173	70	148	523	307	255	-
		R	2.56	3.85	0.00	6.67	0.58	20.69	6.47	0.00	0.00	29.44	7.03
	MRL	M	42	38	204	69	186	83	199	523	332	254	-
		R	7.69	46.15	0.00	15.00	8.14	43.10	43.17	0.00	8.14	28.93	20.69
	UB	M	39	26	204	60	172	58	139	523	307	197	-
		R	2.56	0.00	0.00	0.00	0.58	1.72	0.00	0.00	0.00	2.03	0.69
	GRL ^[37]	M	40	27	204	64	173	70	148	523	307	255	-
		R	2.56	3.85	0.00	6.67	0.58	20.69	6.47	0.00	0.00	29.44	7.03

注：M 表示 Makespan，R 表示 RPD (%)。

从表3.7可以看出，在 JSP 标准算例上，MRL 的平均 RPD 为 25.07%，优于基于学习的 GNN 和 DDQN，在 NPFSP 标准算例上，MRL 的平均 RPD 为 10.61%，结果接近两个元启发式算法。在 FJSP 的标准算例上，MRL 的平均 RPD 为 20.69%，与元启发式算法 VND-hGA 有一定差距，但是接近基于学习的 GRL。综合来看，MRL 在标准算例上取得了先进算法可比的求解质量。结合其强大的泛化能力，为多类型车间调度提供了一种有效的解决途径。

(3) Hurink-vdata 分组算例。

进一步地，针对 Hurink 算例中的 vdata 实例集，按每 5 个算例为一组进行分组对比平均值。比较算法包括：GRL^[37]；基于智能体的深度强化学习方法（CARL）^[35]和基于图注意力机制的强化学习方法（GAT）^[50]。表3.8给出了测试结果。

表 3.8 Hurink vdata 分组对比结果

算例组	指标	GRL ^[37]	CARL ^[35]	GAT ^[50]	MRL	UB
la01–la05	M	508	544.4	550.4	525.4	507
	R	0.19	7.38	8.56	3.63	-
la06–la10	M	794.4	822.2	821.8	808.8	794
	R	0.05	3.55	3.50	1.86	-
la11–la15	M	1042	1062.8	1064.0	1054.4	1040.8
	R	0.11	2.11	2.23	1.31	-
la16–la20	M	679.8	695.0	703.2	683.2	679.8
	R	0.00	2.24	3.44	0.50	-
la21–la25	M	784.2	858.4	825.2	822.2	777.2
	R	0.90	10.45	6.18	5.79	-
la26–la30	M	1055.2	1100.0	1085.4	1076.0	1054.2
	R	0.09	4.34	2.96	2.07	-
la31–la35	M	1552.4	1580.0	1614.0	1592.6	1552.2
	R	0.01	1.79	3.98	2.60	-
la36–la40	M	950.8	970.8	983.4	983.8	950.8
	R	0.00	2.10	3.43	3.47	-
平均 RPD(%)		0.17	4.25	4.28	2.65	-

注：M 表示 Makespan，R 表示 RPD (%)，T 表示 Time (s)。

由表中结果可知，MRL 的平均 RPD 为 2.65%，优于 CARL 和 GAT，且接近 GRL 的水平。其中，MRL 在 la16–la20 组上的 RPD 为 0.50%，与 GRL 差距极小；在 la21–la25 和 la26–la30 等较大规模算例组上，MRL 同样稳定优于 CARL 和 GAT，表明该模型具有较好的泛化能力与求解稳定性。

综上，MRL 在多类型车间调度问题上实现了求解效率与解质量的较好平衡。凭借统一的模型结构，所提方法在三类调度场景中均表现出与现有方法相当的竞争力；同时，其泛化能力有助于避免重复开发模型的额外成本，为多品种、快切换的智能制造应用提供了具有实用价值的解决方案。

3.5 本章小结

本章提出了基于异构图表征与元强化学习融合的 MRL 方法，以提高跨多类型车间调度问题的泛化能力与适应性。该方法采用图注意力网络聚合机器节点特征，多

层感知机提取工序节点特征，并设计 FiLM 特征调控模块将车间类型信息融入图嵌入，缓解跨任务训练中的噪声梯度干扰。训练采用包含内外循环的元训练算法，并设计了面向新实例的快速适应算法。实验方面，首先通过敏感性分析确定了稳定的超参数配置；消融实验表明 FiLM 模块能加速训练收敛并提升稳定性；在多组随机算例和标准算例上的测试表明，MRL 在求解质量和运行效率上优于多种调度规则和学习型方法，且与单类型专用模型的性能差距较小。以上结果验证了 MRL 方法能够在多类型车间调度问题中快速提供高质量调度方案，为后续多目标调度问题的研究提供了基础。

4 基于分解的多目标多类型车间问题调度方法

实际车间调度往往需要同时考虑多个相互冲突的目标，单目标优化难以满足真实生产需求。因此，在前一章工作的基础上，研究中将调度问题拓展为多目标优化，引入总拖期作为新的优化指标。为了求解多目标的多类型车间问题，引入分解的思想，提出一种基于分解的多目标元强化学习调度方法（Multi-objective MAML with Reinforce Learning, MMRL），通过权重分解，动作蒸馏和参数继承实现对于 Pareto 解集的求解。最后，通过算例研究与实验分析验证了该方法的有效性。

4.1 多目标优化理论

上一章介绍了单目标下的多类型车间调度方法，然而在实际生产场景中，调度决策往往需要同时兼顾多个优化目标（如完工时间、拖期、能耗等），且这些目标之间通常存在冲突。因此，这一章将问题拓展为多目标优化，其一般形式可表示为：

$$\min_{x \in \mathcal{X}} \mathbf{F}(x) = (f_1(x), f_2(x), \dots, f_m(x)) \quad (4.1)$$

其中 $\mathbf{F}(x)$ 为多目标问题的目标函数向量， x 为决策向量， $\mathcal{X}$ 为可行解空间， m 为目标数量。

当缺乏先验知识、无法事先明确各目标的重要性权重时，单一解往往难以满足实际需求，因此需要先求得 Pareto 前沿（Pareto Front）的近似解集，再依据决策者的偏好与实际生产需求，从中选择一个或多个 Pareto 解作为最终方案。Pareto 支配（Pareto Dominance）定义如下：

对于两个可行解 $x^a, x^b \in \mathcal{X}$ ，若满足：

- (1) x^a 的所有子目标不劣于 x^b ，即 $f_k(x^a) \leq f_k(x^b), \forall k \in \{1, \dots, m\}$ 。
- (2) x^a 至少存在一个子目标优于 x^b ，即 $\exists k: f_k(x^a) < f_k(x^b)$ 。

则称 x^a 支配 x^b ，记为 $x^a \prec x^b$ 。所有不被任何其他解支配的解构成 Pareto 最优解集，其在目标空间中的映射即为 Pareto 前沿。

此时元强化学习的目标不再是学习一个通用的单一元参数，而是学习一组与权重子问题一一对应的参数集合。每个通过相应权重的标量化目标进行优化，使得在

目标空间形成一组策略解。通过不断学习迭代，这组策略的期望是覆盖多目标调度问题的 Pareto 前沿。

4.2 面向多目标的元强化学习方法

MOEA/D^[18] 通过将多目标问题分解为一系列标量化子问题，并利用子问题之间的邻域关系进行协同优化，从而获得 Pareto 解集。本节在上一章单目标元强化学习框架的基础上，引入 MOEA/D 的分解机制，提出一种求解多目标问题的元强化学习方法 MMRL。具体而言，首先构建一组权重向量 $\Lambda = \{\lambda_1, \lambda_2, \dots, \lambda_Q\}$ ，将多目标调度问题分解为 Q 个子问题；每个子问题对应独立的策略参数 θ_i ，并按照权重向量的欧氏距离划分邻域。训练时，在每个邻域中选取代表权重执行内外循环元更新，邻域内其余权重以代表策略的动作分布作为教师进行蒸馏更新。通过周期性验证与参数继承，加快收敛和提高在解空间的连续性。

4.2.1 多目标分解和权重子问题建模

在第二章中，引入了总拖期和最大完工时间一起构成多目标问题。针对该多目标优化问题，本节提出将分解思想应用在元强化学习中：首先利用单纯形网格设计法生成一组均匀分布的权重向量：

$$\Lambda = \left\{ \lambda \in \mathbb{R}^m \mid \sum_{k=1}^m \lambda_k = 1, \lambda_k \geq 0, \lambda_k = \frac{h_k}{H}, h_k \in \{0, 1, \dots, H\}, \sum_{k=1}^m h_k = H \right\} \quad (4.2)$$

权重向量的个数 $Q = C_{H+m_o-1}^{m_o-1}$ ，其中 m_o 为目标数， H 为网格划分参数，通过改变 H 的数值控制生成权重向量的个数。随后采用加权求和法（Weighted Sum, WS）构造子问题目标函数：

$$g^{\text{ws}}(x \mid \lambda_i) = \sum_{k=1}^m \lambda_i^{(k)} \cdot f_k(x) \quad (4.3)$$

对于权重子问题的求解，使用第三章的 MRL 框架对子问题进行训练和求解。但同时需要对奖励函数进行补充以匹配权重子问题的双目标形式。具体而言，在每个调度步骤 t 结束后，分别计算两个目标的分量奖励。其中，最大完工时间分量奖励已在第二章给出；总拖期分量奖励定义为：

$$r_2^t = \sum_{i=1}^n \max\left(0, \hat{C}_i^t - d_i\right) - \sum_{i=1}^n \max\left(0, \hat{C}_i^{t+1} - d_i\right) \quad (4.4)$$

其中，拖期分量的定义见式(4.4)。式(4.4)中的 $\hat{C}_i^t$ 与 $\hat{C}_i^{t+1}$ 分别表示时刻 t 与时刻 $t+1$ 下第 i 个工件的预计完工时间（由已调度工序的完工时间与未调度工序的平均加工时长计算得到）， d_i 表示工件 i 的交期。由此， r_1^t 与 r_2^t 分别刻画当前动作在两个目标上的改进增量，在权重 $\lambda_i = (\lambda_i^{(1)}, \lambda_i^{(2)})$ 下进行标量化得到单步奖励：

$$r^t(\lambda_i) = \lambda_i^{(1)} \cdot r_1^t + \lambda_i^{(2)} \cdot r_2^t \quad (4.5)$$

4.2.2 基于邻域动作蒸馏的协作机制

加权求和法的目标函数关于权重向量连续，因此相邻权重所对应的最优策略在目标偏好上往往具有一定相似性。基于这一性质引入基于邻域动作蒸馏的协作机制，利用子问题之间的参数关联性与策略相似性，通过知识蒸馏加速邻域内子问题的训练收敛，并通过参数继承在子问题间传递高质量参数：

(1) 邻域构建与重划分。将权重向量按固定顺序编号为 $\{\lambda_i\}_{i=1}^Q$ ，设邻域大小为 K ，并记 $M = \lceil Q/K \rceil$ 。在第 t 轮迭代中，先计算轮换偏移量 $s_t = ((t-1) \bmod K) + 1$ ，再按步长 K 对权重集合做循环分块，得到第 u 个邻域块：

$$\mathcal{B}_u^{(t)} = \left\{ [(u-1)K + s_t + q - 2]_Q + 1 \mid q = 1, \dots, K \right\}, \quad u = 1, \dots, M \quad (4.6)$$

其中 $[\cdot]_Q$ 表示对 Q 取模。该划分满足 $\mathcal{B}_u^{(t)} \cap \mathcal{B}_v^{(t)} = \emptyset (u \neq v)$ 且 $\bigcup_{u=1}^M \mathcal{B}_u^{(t)} = \{1, \dots, Q\}$ ，划分后的邻域两两不重叠并覆盖整个权重集合。在每轮迭代开始时都执行上述轮换重划分，使每个权重都有机会进行完整的元更新。对任一子问题 i ，记其所属邻域块为 $\mathcal{B}_{u(i)}^{(t)}$ ，则其用于协作与继承的邻域集合定义为：

$$\mathcal{N}(i) = \mathcal{B}_{u(i)}^{(t)} \setminus \{i\} \quad (4.7)$$

(2) 教师权重元更新。在每轮训练中，对每个邻域块 $\mathcal{B}_u^{(t)}$ 选取其中间位置的权重作为教师 $r_u^{(t)}$ 。教师子问题先执行一次完整的内外循环元更新：在内循环中，以当前参数 θ_r 为起点，在任务环境上采集轨迹并执行 K_{inner} 步 PPO 更新，得到适应后参数 θ'_r ；在外循环中，使用 θ'_r 在验证集上计算元损失并求取元梯度 g_{meta} ，更新教师参数：

$$\theta_r \leftarrow \theta_r - \beta \cdot g_{meta} \quad (4.8)$$

在此过程中，代表策略与环境交互采样产生的轨迹数据 $\mathcal{D}_r = \{(s_b, a_b, R_b)\}_{b=1}^B$ 被保留下来，供邻域内其余子问题复用，避免了重复的环境采样开销。

(3) 邻域知识蒸馏。对同邻域内的其余权重子问题 $i \neq r$ ，不再独立与环境交互采样，而是直接复用代表子问题的轨迹数据执行纯蒸馏训练。蒸馏的核心思想是让学生策略 π_{θ_i} 模仿教师策略 π_{θ_r} 的动作分布，从而将教师的知识迁移给学生。

具体操作如下：固定教师参数 θ_r ，从代表轨迹中抽取状态批次 $\mathcal{B} = \{s_b\}_{b=1}^B$ 。在每个状态 s_b 上，教师策略输出动作概率分布 $\pi_{\theta_r}(\cdot|s_b) = \{p_r(a|s_b)\}_{a \in \mathcal{A}}$ ，学生策略输出 $\pi_{\theta_i}(\cdot|s_b) = \{p_i(a|s_b)\}_{a \in \mathcal{A}}$ 。两个分布之间的 KL 散度定义为：

$$D_{KL}(\pi_{\theta_r}(\cdot|s_b) \parallel \pi_{\theta_i}(\cdot|s_b)) = \sum_{a \in \mathcal{A}} p_r(a|s_b) \log \frac{p_r(a|s_b)}{p_i(a|s_b)} \quad (4.9)$$

KL 散度衡量了两个概率分布之间的差异，具有非负性 ($D_{KL} \geq 0$) 且仅当两个分布完全一致时取零。按批次平均构造蒸馏损失：

$$\mathcal{L}_{\text{distill}}^{(i)} = \frac{1}{B} \sum_{b=1}^B D_{KL}(\pi_{\theta_r}(\cdot|s_b) \parallel \pi_{\theta_i}(\cdot|s_b)) \quad (4.10)$$

将 KL 散度展开，蒸馏损失可进一步写为：

$$\mathcal{L}_{\text{distill}}^{(i)} = \frac{1}{B} \sum_{b=1}^B \left[\sum_{a \in \mathcal{A}} p_r(a|s_b) \log p_r(a|s_b) - \sum_{a \in \mathcal{A}} p_r(a|s_b) \log p_i(a|s_b) \right] \quad (4.11)$$

其中第一项为教师分布的负熵，与学生参数 θ_i 无关，在梯度计算中可视为常数。因此，蒸馏损失关于学生参数 θ_i 的梯度简化为：

$$\nabla_{\theta_i} \mathcal{L}_{\text{distill}}^{(i)} = -\frac{1}{B} \sum_{b=1}^B \sum_{a \in \mathcal{A}} p_r(a|s_b) \cdot \nabla_{\theta_i} \log p_i(a|s_b) \quad (4.12)$$

这一梯度形式类似于策略梯度中的加权似然比梯度，其中教师的动作概率 $p_r(a|s_b)$ 充当了权重的角色，引导学生策略学习教师的良好动作。基于蒸馏损失进行梯度下降，得到更新后的学生参数：

$$\theta'_i = \theta_i - \alpha_{\text{distill}} \cdot \nabla_{\theta_i} \mathcal{L}_{\text{distill}}^{(i)} \quad (4.13)$$

其中 α_{distill} 为蒸馏学习率。

(4) 学生权重的外循环更新。蒸馏完成后，学生需要根据自身权重子问题的特定目标进行进一步调整，以确保策略适应自身权重对应的目标偏好。学生在外循环执行 FOMAML 更新：在任务环境 τ_i 上使用蒸馏后的参数 θ'_i 进行采样交互，计算任务专属损失：

$$\mathcal{L}_{\tau_i}^{\text{task}}(\theta'_i) = -\mathbb{E}_{\tau \sim \pi_{\theta'_i}} \left[\sum_{t=0}^{H-1} r^t(\lambda_i) \right] \quad (4.14)$$

其中 $r^t(\lambda_i) = \lambda_i^{(1)} \cdot r_1^t + \lambda_i^{(2)} \cdot r_2^t$ 为根据权重 λ_i 标量化后的即时奖励。然后用该损失的梯度直接更新学生参数 θ_i :

$$\theta_i \leftarrow \theta_i - \beta_{\text{student}} \cdot \nabla_{\theta_i} \mathcal{L}_{\tau_i}^{\text{task}}(\theta'_i) \quad (4.15)$$

这里采用一阶近似的 FOMAML 计算梯度（详见第2.2.2节），使用蒸馏后参数 θ'_i 处的梯度作为对 θ_i 的更新方向：

$$\nabla_{\theta_i} \mathcal{L}_{\tau_i}^{\text{task}}(\theta'_i) \approx \nabla_{\theta'_i} \mathcal{L}_{\tau_i}^{\text{task}}(\theta'_i) \quad (4.16)$$

这种近似在保证迭代效率的同时显著降低了计算开销。同时，反向传播时教师权重的参数 θ_r 保持不变，确保教师策略的稳定性不受学生更新的影响。

4.2.3 参数继承机制

在训练过程中，不同子问题由于权重偏好和训练轨迹的差异，策略质量可能存在差距。本节采用参数继承机制通过利用邻域内表现优异的子问题参数来加速落后子问题的训练。具体而言，在每隔 T_K 轮的验证集验证中，收集子问题在验证集上各目标函数的值，并在其当前所属邻域进行继承操作。

为评估子问题 j 的策略在权重 λ_i 对应的目标偏好下的表现，定义得分函数。对于子问题 j ，使用策略 π_{θ_j} 在验证集上求解，得到每个验证任务的目标向量 $\mathbf{F}_j^v = (f_1^v(\theta_j), f_2^v(\theta_j))$ ， $v = 1, \dots, V$ 。在权重 λ_i 下的标量化得分为：

$$\text{score}_j(\lambda_i) = \frac{1}{V} \sum_{v=1}^V \sum_{k=1}^m \lambda_i^{(k)} \cdot \tilde{f}_k^v(\theta_j) \quad (4.17)$$

其中 $\tilde{f}_k^v(\theta_j)$ 为归一化后的第 k 个目标值。归一化方法采用当前所有子问题在该验证任务上的最大最小值进行缩放：

$$\tilde{f}_k^v(\theta_j) = \frac{f_k^v(\theta_j) - f_k^{v,\min}}{f_k^{v,\max} - f_k^{v,\min} + \varepsilon} \quad (4.18)$$

其中 $f_k^{v,\min} = \min_{q \in \{1, \dots, Q\}} f_k^v(\theta_q)$ ， $f_k^{v,\max} = \max_{q \in \{1, \dots, Q\}} f_k^v(\theta_q)$ ， ε 为防止除零的小常数。

归一化确保不同目标维度的量纲差异不会影响加权得分的计算，使得各目标对得分的贡献由权重向量 λ_i 公正地控制。由于优化目标为最小化，得分越低表示策略越优。

对每个子问题 i ，在其邻域 $\mathcal{N}(i)$ 中寻找在权重 λ_i 下得分最优（即得分最低）的邻居：

$$j^* = \arg \min_{j \in \mathcal{N}(i)} \text{score}_j(\lambda_i) \quad (4.19)$$

只有当邻居 j^* 的得分显著优于当前子问题 i 时，才触发参数继承，避免微小差异导致的频繁且无意义的参数扰动。判断条件为：

$$\text{score}_i(\lambda_i) - \text{score}_{j^*}(\lambda_i) > \delta \quad (4.20)$$

其中 $\delta > 0$ 为预设的改进阈值。

当继承条件满足时，计算继承系数 α_i 用于控制融合的力度。继承系数根据当前子问题与最优邻居之间的得分差距自适应调整：

$$\alpha_i = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min}) \cdot \frac{\text{score}_i(\lambda_i) - \text{score}_{j^*}(\lambda_i) - \delta}{\text{score}_i(\lambda_i) + \varepsilon} \quad (4.21)$$

其中 $\alpha_{\min}$ 和 $\alpha_{\max}$ 分别为继承系数的下界和上界（通常取 $\alpha_{\min} = 0.1$ ， $\alpha_{\max} = 0.5$ ）。

通过自适应机制加速落后子权重问题的迭代，对表现尚可的子问题则以较小的力度进行微调。使用继承系数 α_i 对当前参数 θ_i 与最优邻居参数 θ_{j^*} 进行凸组合：

$$\theta_i^{\text{new}} = (1 - \alpha_i) \theta_i + \alpha_i \theta_{j^*} \quad (4.22)$$

该融合操作在参数空间中将子问题 i 的参数向邻居 j^* 的方向移动 α_i 的比例。由于 $\alpha_i \in [\alpha_{\min}, \alpha_{\max}] \subset (0, 1)$ ，融合后的参数始终位于两者的凸组合路径上，保持了参数的合理性。

参数融合并不总是带来改进。为防止融合导致性能下降，在融合后立即在验证集上重新评估子问题 i 的表现：

$$\text{score}_i^{\text{new}}(\lambda_i) = \frac{1}{V} \sum_{v=1}^V \sum_{k=1}^m \lambda_i^{(k)} \cdot \tilde{f}_k^v(\theta_i^{\text{new}}) \quad (4.23)$$

若融合后的得分不优于融合前，则回退继承操作，恢复原始参数：

$$\theta_i \leftarrow \begin{cases} \theta_i^{\text{new}}, & \text{若 } \text{score}_i^{\text{new}}(\lambda_i) < \text{score}_i(\lambda_i) \\ \theta_i, & \text{否则回退} \end{cases} \quad (4.24)$$

该回退机制确保参数继承是一种安全操作：只有在确认带来改进时才接受新参数，否则保持原有参数不变。这一设计有效避免了因参数空间中的非凸性（即两组好参数的线性插值未必仍然好）而导致的策略退化。

面向多目标的元强化学习训练算法的伪代码如算法3所示：

算法 3 基于邻域动作蒸馏的多目标元强化学习训练

Input: 权重 $\Lambda = \{\lambda_i\}_{i=1}^Q$; 学习率 $\beta, \alpha, \beta_{\text{student}}$; 邻域大小 $K, T_v, \delta, [\alpha_{\min}, \alpha_{\max}]$; 轮次 R
Output: 参数 $\{(\lambda_i, \theta_i)\}_{i=1}^Q$
1: 初始化: $\theta_i \sim \mathcal{N}(0, \sigma^2)$, $i = 1, \dots, Q$
2: **while** $t < R$ **do**
3: $t \leftarrow t + 1$
4: 计算偏移 $s_t = ((t - 1) \bmod K) + 1$, 按步长 K 轮换划分邻域块 $\{\mathcal{B}_u^{(t)}\}_{u=1}^M$
5: **for** 每个邻域 Nbhd **do**
6: 选择中间权重作为教师 $r = \text{mid}(\text{Nbhd})$
7: 教师更新: 采样任务, 内循环 K_{inner} 步 PPO, 在验证集上计算元梯度, 更新 $\theta_r \leftarrow \theta_r - \beta g_{\text{meta}}$
8: 保存轨迹 $\mathcal{D}_r$
9: 学生更新: $\forall i \neq r$, 先以代表轨迹做蒸馏得到 θ'_i , 再在 τ_i 上计算 $\mathcal{L}_{\tau_i}^{\text{task}}(\theta'_i)$, 按一阶近似更新 $\theta_i \leftarrow \theta_i - \beta_{\text{student}} \nabla_{\theta_i} \mathcal{L}_{\tau_i}^{\text{task}}(\theta'_i)$
10: **end**
11: **if** $t \bmod T_v = 0$ **then**
12: **for** $i = 1$ **to** Q **do**
13: 计算得分: $\text{score}_i, \text{score}_{j^*}$, 其中 $j^* = \arg \min_{j \in \mathcal{N}(i)} \text{score}_j(\lambda_i)$
14: **if** $\text{score}_i - \text{score}_{j^*} > \delta$ **then**
15: 计算继承系数 $\alpha_i \in [\alpha_{\min}, \alpha_{\max}]$
16: 融合: $\theta_i^{\text{new}} \leftarrow (1 - \alpha_i)\theta_i + \alpha_i\theta_{j^*}$
17: 验证 θ_i^{new} : 若 $\text{score}_i^{\text{new}} < \text{score}_i$, 则 $\theta_i \leftarrow \theta_i^{\text{new}}$; 否则回退
18: **end**
19: **end**
20: **end**
21: **end**

4.3 实验与结果分析

本节验证所提的 MMRL 的有效性, 给出数据集构成、验证方案与关键参数设置; 其次, 选取多个多目标进化算法作为对比基线, 通过若干指标评估解集质量; 最后, 分别从训练收敛行为、随机算例综合表现与标准算例结果三个层面分析 MMRL 在解集质量与计算效率上的表现。

4.3.1 数据集与参数设置

为验证所提出的基于分解的多目标元强化学习方法在多类型车间调度场景中的有效性, 实验采用第 3.4.1 节构建的训练集 A 作为训练与验证基础。该数据集覆盖 JSP、FJSP 与 NPFSP 三类典型车间问题, 并保持三类问题在训练过程中的比例一致,

从而保证多目标训练过程中不同类型问题之间的均衡性。与第三章保持一致，验证阶段采用固定验证集进行周期性评估，以排除任务采样波动对模型判断的干扰。在多目标场景下，优化目标分别为最大完工时间与总拖期。训练过程中，对每个权重子问题分别记录在验证集上的两个目标值，并进一步根据对应权重计算归一化标量化分数，用于执行邻域参数继承判断。

第四章实验的基础训练环境、编程框架与第三章保持一致。元学习率、内循环学习率与 PPO 基本参数等沿用第三章有效配置。其他参数为：网格划分参数 $H = 64$ ，控制生成 65 个均匀分布的权重（ $Q = 65$ ）；邻域大小 $K = 4$ 。

参数继承的触发阈值设为 $\delta = 0.01$ ，继承系数 α_i 由得分差距动态计算并裁剪到 $[\alpha_{\min}, \alpha_{\max}] = [0.1, 0.5]$ 区间。邻域在每次验证与继承完成后依据当前权重重新划分，以适应训练过程中策略分布的变化。该设置保证参数迁移幅度与邻域优势程度自适应匹配，避免固定继承系数在不同训练阶段过强或过弱的问题。

在多目标训练中新增的交期参数设置如下：对训练集中的每个算例，各工件的交期 d_i 采用估算完工时间的 1.3 倍生成，即 $d_i = 1.3 \times \hat{C}_i$ ，其中估算完工时间 $\hat{C}_i$ 定义为工件 i 全部工序的平均加工时长之和：

$$\hat{C}_i = \sum_{j=1}^{n_i} \bar{p}_{ij}, \quad \bar{p}_{ij} = \frac{1}{|\mathcal{M}_{ij}|} \sum_{k \in \mathcal{M}_{ij}} p_{ijk} \quad (4.25)$$

其中 n_i 为工件 i 的工序数， $\mathcal{M}_{ij}$ 为工序 O_{ij} 的候选机器集合， p_{ijk} 为工序 O_{ij} 在机器 k 上的加工时间。Makespan 是一个全局性效率指标，其值仅由最后完工的那个工件决定，是所有工件中的最差情况。而总拖期用于衡量整体延误水平，关系到每个工件的完工时间与其截止期的差值。即使两个调度方案的 Makespan 相同，工件的加工顺序不同也可能导致两者的总拖期完全不同。

4.3.2 对比算法与评价指标

为全面评估所提出的基于分解的多目标元强化学习方法（MMRL）的性能，选取以下四种代表性多目标进化算法作为对比基线：

(1) NSGA-III (Non-dominated Sorting Genetic Algorithm III)^[77]：在 NSGA-II 基础上引入参考点机制，通过非支配排序保留精英个体，并利用均匀分布的参考点引导种群在目标空间中均匀分散，在多目标车间调度等高维目标问题中广泛应用。

(2) MOGA (Multi-Objective Genetic Algorithm)^[78]：基于 Pareto 支配关系进行适

应度排序的多目标遗传算法，通过交叉与变异算子在解空间中进行全局搜索，利用小生境机制维持种群多样性。

(3) RVEA (Reference Vector Guided Evolutionary Algorithm)^[79]: 通过预设的参考向量集合引导种群进化方向，采用角度惩罚距离度量个体与参考向量的匹配程度，并自适应调整参考向量分布以适应不规则 Pareto 前沿形状。

(4) MOEA/D (Multi-Objective Evolutionary Algorithm based on Decomposition)^[80]: 将多目标优化问题分解为一组标量化子问题并行求解，利用子问题之间的邻域关系进行协同进化。该算法框架开销较小，求解速度快，但在前沿形状不规则时可能出现多样性不足的问题。

上述四种算法均采用种群规模 $N_{pop} = 100$ 、最大迭代次数 $G_{max} = 200$ 的统一参数配置，以保证比较的公平性。

采用下面两个指标评价 Pareto 解集整体质量：

(1) 超体积 (Hypervolume, HV): 衡量近似 Pareto 解集 S 在目标空间中所支配的超体积大小，定义为：

$$HV(S) = \text{Vol} \left(\bigcup_{s \in S} \prod_{k=1}^m [s_k, r_k] \right) \quad (4.26)$$

其中 $\mathbf{s} = (s_1, \dots, s_m)$ 为解集中的目标向量， $\mathbf{r} = (r_1, \dots, r_m)$ 为参考点。HV 值越大，表示解集在目标空间中的覆盖范围越广、质量越高。

(2) 反向代际距离 (Inverted Generational Distance, IGD): 衡量近似解集 S 对真实 Pareto 前沿 P^* 的逼近程度，定义为：

$$IGD(S, P^*) = \frac{1}{|P^*|} \sum_{p \in P^*} \min_{s \in S} \|\mathbf{p} - \mathbf{s}\|_2 \quad (4.27)$$

IGD 值越小，表示近似解集对真实前沿的覆盖与逼近程度越好。由于多目标车间调度问题的真实 Pareto 前沿未知，实验中以所有对比算法在同一算例上产生的全部非支配解构成的近似参考前沿替代 P^* 。

4.3.3 实验分析

(1) MMRL 训练结果分析

MMRL 在第 3.4.1 节构建的训练集 A 上进行训练，训练集 A 中算例的每个工件交期由式 (4.25) 给出，为直观展示不同权重偏好下的训练动态，选取两个具有代表性

的权重子问题分析训练过程：权重 1 对应 $\lambda_1 = (1.0, 0)$ ，即仅优化 $C_{\max}$ ；权重 2 对应 $\lambda_2 = (0.5, 0.5)$ 。

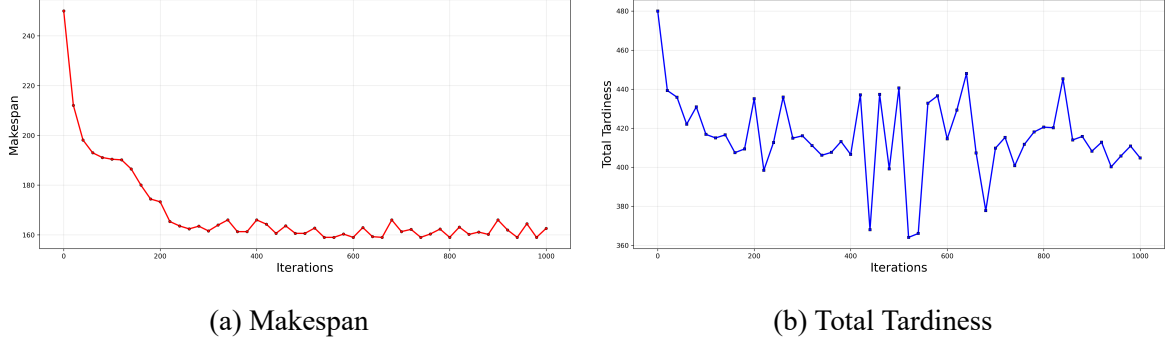


图 4.1 MMRL 训练过程中权重 1 对应子问题的目标收敛曲线

图 4.1 给出了权重 1 在训练过程中两个目标的验证集收敛曲线。从图 4.1(a) 可以观察到，Makespan 在约 280 代时收敛。从图 4.1(b) 可以看到，尽管权重 1 对总拖期的权重为零，但在训练初期仍观察到有下降趋势，这是因为对最大完工时间的优化在一定程度上改善了工件的按时完工率，随着迭代的进行，调度方案普遍较优，总拖期缺乏显式的优化引导，在验证集上的水平波动明显。

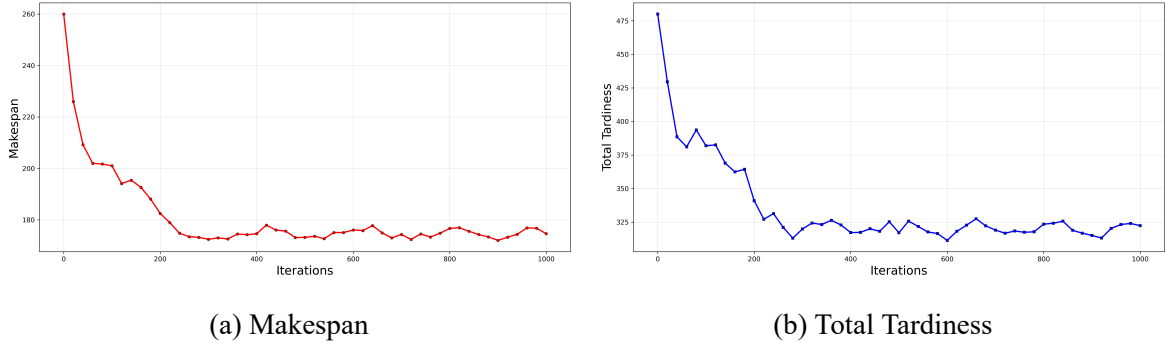


图 4.2 MMRL 训练过程中权重 2 对应子问题的目标收敛曲线

图 4.2 给出了权重 2 的训练收敛曲线。相较于图 4.1，两个目标的优化较为同步。对比两组权重的收敛结果，说明 MMRL 的分解训练机制能够有效地将不同权重偏好传导至策略优化过程，使得各子问题收敛至目标空间中对应权重方向的不同区域，验证了基于分解的元强化学习框架在多目标车间调度场景下具备良好的收敛性与权重区分度。

(2) 随机和标准算例结果分析

表 4.1 给出了 MMRL 与四种多目标进化算法在 NPFSP、JSP 与 FJSP 三类问题、八种规模下的随机算例综合对比。其中 MMRL 在测试阶段将 65 个权重子问题按每批至多 8 个在 GPU 上并行推理，共执行 9 批；四种进化算法均采用种群规模 100、迭代次数 200 运行，对于 NSGA-III，RVEA 和 MOEA/D 等需要参考方向的算法，同样采取 $H = 64$ 的单纯形网格方法生成参考方向。

表 4.1 不同算法在随机算例下的结果

规模	类型	MMRL			MOGA			NSGA-III			RVEA			MOEA/D		
		HV	IGD	Time	HV	IGD	Time	HV	IGD	Time	HV	IGD	Time	HV	IGD	Time
10_5	FJSP	0.69	0.24	6.21	0.77	0.17	7.60	0.92	0.15	8.95	0.80	0.18	9.85	0.60	0.24	4.64
	JSP	0.99	0.09	4.62	0.92	0.09	5.91	1.03	0.05	7.07	0.90	0.10	7.69	0.67	0.25	3.64
	NPFSP	1.01	0.47	4.98	1.01	0.47	5.62	1.12	0.02	6.37	1.01	0.47	7.08	0.48	0.61	3.23
15_5	FJSP	1.08	0.08	7.65	0.90	0.11	16.35	0.96	0.18	21.36	1.04	0.07	24.03	0.93	0.11	9.82
	JSP	1.08	0.01	5.67	0.83	0.20	12.74	0.95	0.08	16.68	0.99	0.07	18.56	0.78	0.20	7.86
	NPFSP	0.43	0.61	6.02	0.41	0.64	11.83	1.21	0.00	15.07	0.39	0.73	16.74	0.15	1.01	6.94
15_10	FJSP	0.89	0.06	10.75	0.71	0.12	37.29	0.87	0.14	43.82	0.91	0.04	45.83	0.70	0.15	22.43
	JSP	0.93	0.17	7.98	0.70	0.31	28.47	1.06	0.23	33.54	1.09	0.07	35.74	0.88	0.22	17.25
	NPFSP	0.91	0.15	8.09	0.82	0.24	27.16	0.94	0.18	30.89	0.99	0.07	32.67	0.13	0.89	15.38
20_5	FJSP	1.06	0.03	9.60	0.96	0.09	31.72	1.00	0.05	33.47	1.08	0.04	43.28	0.84	0.22	19.37
	JSP	0.94	0.04	7.14	0.74	0.10	24.38	0.83	0.31	26.59	0.98	0.05	32.74	0.62	0.19	15.28
	NPFSP	0.84	0.36	7.68	1.05	0.16	23.05	1.16	0.06	24.03	1.12	0.13	30.17	0.39	0.67	13.47
20_10	FJSP	0.91	0.12	13.01	0.81	0.13	55.63	0.91	0.08	76.15	0.87	0.17	72.64	0.75	0.13	40.76
	JSP	1.03	0.05	9.66	0.66	0.33	44.81	0.89	0.13	59.73	1.07	0.04	58.37	0.91	0.17	31.43
	NPFSP	0.62	0.77	9.96	0.69	0.76	42.37	0.32	0.22	53.46	0.07	0.91	52.89	0.40	0.62	27.56
30_10	FJSP	0.94	0.07	16.96	0.79	0.10	149.34	0.83	0.12	152.87	0.90	0.05	189.45	0.70	0.18	76.89
	JSP	1.00	0.07	12.60	0.92	0.09	112.56	0.85	0.17	122.38	1.05	0.04	144.73	0.97	0.07	62.34
	NPFSP	0.89	0.78	13.28	0.44	0.66	107.43	1.21	0.00	110.24	0.60	0.52	131.28	0.52	0.61	55.27
30_15	FJSP	0.83	0.04	25.46	0.71	0.12	193.47	0.66	0.20	259.43	0.79	0.07	258.72	0.61	0.20	142.37
	JSP	0.98	0.04	18.90	0.87	0.11	152.28	0.91	0.08	198.74	0.89	0.08	211.34	0.87	0.08	103.85
	NPFSP	0.78	0.73	17.02	0.43	0.46	145.61	1.12	0.00	179.62	0.28	0.66	196.47	0.33	0.60	94.62
30_20	FJSP	0.86	0.03	29.41	0.78	0.10	278.34	0.62	0.19	316.47	0.84	0.05	361.83	0.55	0.21	159.71
	JSP	1.01	0.27	21.84	0.79	0.20	219.48	0.79	0.22	248.36	1.05	0.09	289.27	0.82	0.20	124.58
	NPFSP	0.90	0.68	21.58	0.30	0.78	209.75	1.21	0.00	224.58	0.19	0.93	261.74	0.39	0.71	112.47

从 Pareto 解集质量来看，MMRL 模型表现出随规模增大而愈发突出的综合优势。在 FJSP 问题上，MMRL 在小规模的 10×5 算例上的 HV 为 0.69，弱于 NSGA-III (0.92)，随着规模的增大，MMRL 的求解质量的优势开始体现，在 20×10 及以上四个算例中均取得最优或并列最优 HV，IGD 在 30×15 和 30×20 上达到全组最优 (0.04

和 0.03)。在 JSP 问题上, MMRL 在 15×5、30×15 等规模上取得全组最优, 在其余规模的表现接近最优; 在 NPFSP 问题上, MMRL 在 15×5 规模上较弱, 但在其他规模的 NPFSP 算例上的解质量普遍接近最优。

从计算时间来看, 在小规模的 10×5 上, 各进化算法耗时与 MMRL 相近,; 但基于迭代的算法的计算成本随规模急剧上升: 以 FJSP 为例, MOGA 耗时从 10×5 的 7.60 秒增至 30×20 的 278.34 秒, NSGA-III 达 316.47 秒, RVEA 高达 361.83 秒, 而 MMRL 仅需 29.41 秒, 约为 RVEA 的 1/12、MOGA 的 1/9。除最小规模 (10×5) 外, MMRL 在三类问题的求解时间低于所有对比算法。MOEA/D 虽然求解时间在所有元启发式算法中最短, 但其 HV/IGD 在多数规模下表现最差。

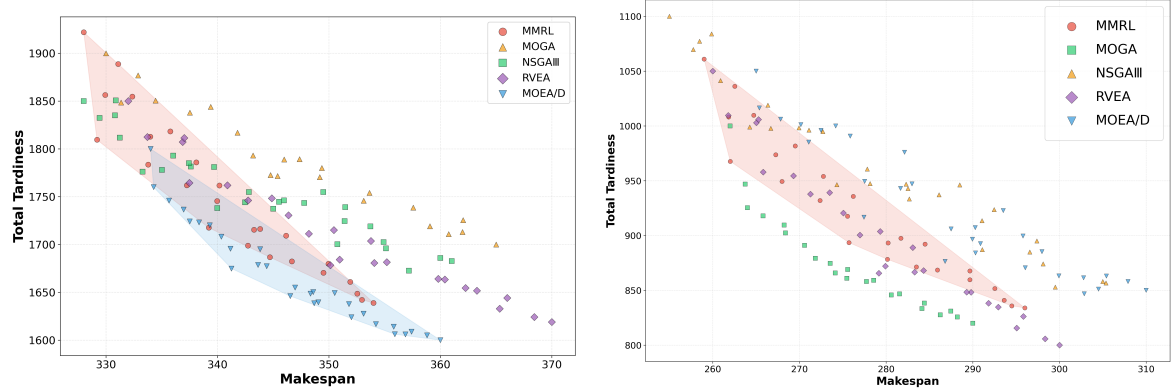
为进一步细化各算法的 Pareto 解集质量, 表 4.2 给出了 MMRL、MOGA、NSGA-III、RVEA 与 MOEA/D 五种算法在 Brandimarte Mk01–Mk10 上的 HV 与 IGD 对比结果。

表 4.2 FJSP 各算例下各算法的 HV/IGD 性能对比

算例	MMRL		MOGA		NSGA-III		RVEA		MOEA/D	
	HV	IGD	HV	IGD	HV	IGD	HV	IGD	HV	IGD
Mk01	0.862	0.286	0.239	0.513	0.690	0.294	0.912	0.286	0.044	0.512
Mk02	0.762	0.149	0.878	0.128	0.806	0.167	0.784	0.149	0.669	0.142
Mk03	0.876	0.161	0.847	0.067	0.892	0.150	0.884	0.161	0.734	0.154
Mk04	0.632	0.240	0.405	0.378	0.633	0.118	0.774	0.088	0.468	0.260
Mk05	0.496	0.185	0.580	0.145	0.353	0.352	0.496	0.185	0.678	0.092
Mk06	0.742	0.236	0.511	0.229	0.546	0.291	0.742	0.236	0.549	0.182
Mk07	0.523	0.215	0.781	0.015	0.439	0.287	0.523	0.215	0.623	0.167
Mk08	0.714	0.068	0.684	0.145	0.656	0.130	0.754	0.075	0.687	0.195
Mk09	0.678	0.305	0.417	0.185	0.556	0.348	0.633	0.365	0.654	0.060
Mk10	0.642	0.199	0.616	0.066	0.553	0.383	0.642	0.199	0.572	0.097
Avg	0.693	0.204	0.596	0.187	0.612	0.252	0.714	0.196	0.568	0.186

从表 4.2 可以观察到, MMRL 在 FJSP 标准算例上展现出较强的稳定性与竞争力。就 HV 而言, MMRL 在 Mk06、Mk09 和 Mk10 三个算例上取得最优, 在多数剩余算例上也保持了接近最优的水平, 十个算例的平均 HV 仅次于 RVEA。就 IGD 而言, MMRL 仅在 Mk01 和 Mk08 算例上取得最优。但综合十个算例的平均值, MMRL 的

综合表现仅仅略差于 RVEA，相较于剩下三个算法的质量提升明显，结合 MMRL 基于端到端学习的高效求解策略，其在保持与 RVEA 可比解质量的同时具备远优于进化算法的推理速度，在实际调度场景中更具应用价值。



(a) Mk09

(b) Mk10

图 4.3 不同算法在 Mk09 与 Mk10 算例上的解集分布图

图 4.3(a) 与图 4.3(b) 展示了不同算法解集的二维分布，红色区域是 MMRL 的解集包络区域，可以看出，与对比算法相比，MMRL 的解集前沿覆盖较稳定，质量高且分布均匀，验证了 MMRL 稳定性与泛化性能。图 4.3(a) 中蓝色区域是 MOEA/D 的解集包络区域，尽管 MOEA/D 在 Mk09 算例上的指标较好，但是探索最优完工时间的能力差，解集的整体完工时间较差。

进一步地，为了更清晰的展现 MMRL 的解集质量，选择综合表现较优的 NSGAIII 和 RVEA，表 4.3 从优化目标的最小值角度对比了三种算法在各 FJSP 算例上所能找到的最优 $C_{\max}$ 与最优总拖期。

从表的结果看，MMRL 在 Pareto 前沿端点解上的保持能力较强。对于 $C_{\max}^{\min}$ ，MMRL 在 10 个算例中有 6 个达到最优，说明 MMRL 在优化完工时间方向具备稳定的探索能力。对于 $T_{sum}^{\min}$ ，MMRL 在 Mk01、Mk05、Mk06 和 Mk07 达到最优，表明其在优化拖期方向同样具有竞争力。将表 4.3 中 MMRL 的 $Makespan^{\min}$ 与第三章单目标 MRL 模型在相同 Brandimarte 算例上的完工时间进行对比，MMRL 在该标准算例上的结果基本接近单目标模型。

图 4.4 与图 4.5 分别展示了 MMRL 与第三章 MRL 模型求解 Mk10 的调度方案甘特图，Mk10 算例在生成时没有对机器 11，12，14 和 15 分配工序，实际参与调度的只有 11 台机器。MMRL 模型取得的调度方案完工时间为 259，接近单目标模型

表 4.3 MMRL、NSGA-III 与 RVEA 在 FJSP 各算例上的最优目标值对比

算例	MMRL		NSGA-III		RVEA	
	$Makespan^{min}$	T_{sum}^{min}	$Makespan^{min}$	T_{sum}^{min}	$Makespan^{min}$	T_{sum}^{min}
Mk01	42	38	42	47	42	38
Mk02	31	23	31	22	31	23
Mk03	204	656	204	650	204	656
Mk04	69	134	67	159	69	126
Mk05	184	792	182	1277	184	792
Mk06	69	47	69	53	69	49
Mk07	159	1128	158	1231	159	1128
Mk08	523	2957	526	3226	527	2779
Mk09	328	1639	328	1686	332	1619
Mk10	259	834	255	858	260	800

的 254。综上，MMRL 能够生成接近 Pareto 前沿的多样化解集，验证了基于分解的 MMRL 模型求解多目标的多类型车间的有效性。



图 4.4 MMRL 的子权重求解 Mk10 算例的甘特图



图 4.5 MRL 求解 Mk10 算例的甘特图

4.4 本章小结

针对多类型车间调度中最大完工时间与总拖期的多目标优化需求，本章提出了基于分解的多目标元强化学习方法 MMRL。该方法引入 MOEA/D 的分解思想，构建了权重子问题建模方法与标量化奖励机制，将多目标问题转化为一组相互关联的强化学习子问题。在此基础上，设计了基于邻域动作蒸馏的元训练方法：以邻域为单位组织训练，在每个邻域中选取代表权重执行内外循环元更新，邻域内其余权重通过代表策略的动作纯蒸馏快速获取决策知识，显著降低了权重数量增多时的训练开销。同时引入周期性验证与参数继承机制，其中邻域由欧氏距离构建，参数按表现选择，继承幅度按得分差距动态调整，并在每次继承后重新划分邻域以适应策略分布的演化。实验方面，首先在随机算例上与四种多目标进化算法进行对比，实验结果表明 MMRL 在 Pareto 解集质量上具有竞争力，且求解速度远优于进化算法。在 FJSP 标准算例的评估中，MMRL 的综合解质量接近最优进化算法，同时具备端到端的高效推理能力。综合来看，MMRL 方法能够为多类型车间的多目标调度问题提供兼顾解集质量与计算效率的端到端求解方案。

5 总结与展望

5.1 研究工作总结

面向制造业多品种、小批量与多目标协同优化需求，本文围绕多类型车间调度中的泛化能力不足和针对多目标 **pareto** 解集获取质量和效率低下的两个关键问题，构建了融合异构图神经网络与元强化学习的统一求解框架。研究表明，该框架能够在保持较高求解质量的同时兼顾计算效率，并具备跨车间类型的稳定泛化能力。

第二章构建了面向多类型车间及其多目标问题的通用元强化学习求解框架。针对 NPFSP、JSP 与 FJSP 三类问题，建立了单目标与多目标数学模型，并将调度过程统一建模为基于异构图的马尔可夫决策过程，形成了可共享的状态表示与求解流程，为后续算法设计与跨类型迁移奠定基础。

第三章提出了基于元强化学习融合的 **MRL** 方法，用于多类型车间单目标调度。方法采用图注意力网络与多层感知机提取异构节点特征，并通过 **FiLM** 特征调控模块缓解跨任务训练中的噪声梯度干扰。消融实验以及随机和标准算例的结果表明，所提方法在求解质量与计算效率上具有优势，且多类型联合训练未引起明显性能退化。

第四章针对最大完工时间与总拖期的冲突优化，提出了基于分解的多目标元强化学习方法 **MMRL**。该方法引入权重子问题建模与标量化奖励，结合邻域蒸馏、参数继承与回滚机制实现跨子问题协同优化。实验表明，**MMRL** 在 **Pareto** 解集质量上具有竞争力，并在计算效率上显著优于多目标进化算法。

5.2 创新点归纳

本文的主要创新点如下：

(1) 设计了面向多类型车间的异构图表示方法。以工序节点和机器节点为核心实体，以先后序关系、可加工关系与已分配关系为边类型，将不同类型车间统一编码为图结构数据，为多类型策略学习提供了统一输入表示。

(2) 将元强化学习应用于多类型车间调度，并通过特征调控模块抑制训练噪声。采用图注意力网络与多层感知机提取异构节点特征，结合内外循环元训练与快速适

应机制，有效提升了跨类型任务训练的收敛速度与稳定性。

(3) 在上述方法基础上拓展多目标场景，提出了基于分解的多目标元强化学习更新策略。通过权重分解、邻域协同蒸馏与参数继承机制，实现了多类型车间在多目标调度下的高效训练与稳定求解。

5.3 研究工作展望

本文基于元强化学习与图神经网络开展了多类型车间单目标与多目标调度研究。后续工作可沿以下方向进一步推进。

(1) 考虑更多实际约束。后续可将动态工件到达、机器故障维修、换型准备时间、运输时间和工人技能等因素纳入模型，在异构图表示与奖励函数中引入对应约束与惩罚项，以提升方法在真实生产环境中的适用性。

(2) 性能改进与方法泛化协同推进。一方面，可通过奖励塑形、任务分布扩展、网络结构升级及权重分解改进提升算法的求解性能；另一方面，可在分布式车间、混合流水车间、装配车间、开放车间等场景中扩展统一异构图建模，并将新类型纳入元学习任务分布。进一步增强框架的泛化性。

致 谢

时光荏苒，又三年的华中科技大学研究生生活即将画上句号。回首这段求学历程，心中满怀感恩。

衷心感谢我的导师刘琼教授在学术研究中给予的悉心指导与大力支持。从日常学习、课题方向的选定、研究方案的论证，到论文结构的反复推敲与修改，导师严谨求实的治学态度和循循善诱的教导方式，让我受益匪浅，刘老师始终以严谨的治学态度和深厚的学术涵养引领我前行。每一次讨论中的启发，每一稿批注中的用心，不仅让我在科研能力上获得扎实锻炼，更让我深刻体会到求真务实、探索不止的学术精神。

感谢课题组的伙伴们在科研思路和方法上的无私分享与热情帮助。感谢张强、刘俊、何家禧、杨沛轩、宁广帅师兄；感谢喻琳婕师姐；感谢同窗郭子辰；感谢师弟谢凯旋、邓整坤、吴建鹏以及师妹杨娜。

感谢我的家人。特别感谢父母多年来的养育之恩和无条件的理解与支持，您们尽自己所能呵护我的成长，您们的关爱与期盼是我在求学路上不断前行的最大动力。

感谢室友陈李杰、陈光溥，在日常生活中的陪伴与照顾，让三年的研究生宿舍生活充满温暖。也感谢在华中科技大学认识的每一位朋友，感谢你们在这段时光里留下的美好回忆，让我的研究生岁月更加丰富而难忘。感谢林博文所带来的轻松愉快的时光，感谢徐卓逸一路的快乐相伴。

最后，向所有在学习、科研和生活中给予过我帮助的老师、同学和朋友们致以最诚挚的谢意！

参考文献

- [1] C. R. H. Marquez, C. C. Ribeiro. Shop scheduling in manufacturing environments: a review. *International Transaction in Operational Research*, 2022, 29(6):3237–3293.
- [2] 王无双, 骆淑云. 基于强化学习的智能车间调度策略研究综述. *计算机应用研究*, 2022, 39(06):1608–1614.
- [3] 李新宇, 黄江平, 李嘉航等. 智能车间动态调度的研究与发展趋势分析. *中国科学: 技术科学*, 2023, 53(07):1016–1030.
- [4] 罗梓琿, 江呈羚, 刘亮等. 基于深度强化学习的智能车间调度方法研究. *物联网学报*, 2022, 6(01):53–64.
- [5] 越民义, 韩继业. n 个零件在 m 台机床上的加工顺序问题 (I). *中国科学*, 1975, : 462–470.
- [6] M. C. Portmann, A. Vignier, D. Dardilhac, et al. Branch and bound crossed with GA to solve hybrid flowshops. *European Journal of Operational Research*, 1998, 107:389–400.
- [7] E. Neron, P. Baptise, J. N. D. Gupta. Solving hybrid flow shop problem using energetic reasoning and global operations. *Omega*, 2001, 29:501–511.
- [8] C. J. Liao, E. Tjandradjaja, T. P. Chung. An approach using particle swarm optimization and bottleneck heuristic to solve hybrid flow shop scheduling problem. *Applied Soft Computing Journal*, 2012, 12(6):1755–1764.
- [9] O. Engin, A. Döyen. A new approach to solve hybrid flow shop scheduling problems by artificial immune system. *Future Generation Computer Systems*, 2004, 20(6):1083–1095.
- [10] M. Zandieh, S. M. T. F. Ghami, S. M. M. Hussein. An immune algorithm approach to hybrid flow shops scheduling with sequence-dependent setup times. *Applied Mathematics and Computation*, 2006, 180(1):111–127.
- [11] T. Yang, Y. Kuo, I. Chang. Tabu-search simulation optimization approach for flow-shop scheduling with multiple processors – a case study. *International Journal of Production Research*, 2004, 42(19):4015–4030.
- [12] H. M. Wang, F. D. Chou, F. C. Wu. A simulated annealing for hybrid flow shop scheduling with multiprocessor tasks to minimize makespan. *International Journal of Advanced Manufacturing Technology*, 2011, 53(5-8):761–776.

- [13] R. S. Sutton, A. G. Barto. Reinforcement learning: an introduction. London: MIT Press, 2018.
- [14] R. Naimi, M. Nouiri, O. Cardin. A Q-learning rescheduling approach to the flexible job shop problem combining energy and productivity objectives. Sustainability, 2021, 13(23):13016.
- [15] B. A. Han, J. Yang. Research on adaptive job shop scheduling problems based on dueling double DQN. IEEE Access, 2020, 8:186474–186495.
- [16] S. Luo. Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. Applied Soft Computing, 2020, 91:1–44.
- [17] J. Park, J. Chun, S. H. Kim, et al. Learning to schedule job-shop problems: representation and policy learning using graph neural network and reinforcement learning. International Journal of Production Research, 2021, 59(11):3360–3377.
- [18] Q. Zhang, H. Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. IEEE Transactions on Evolutionary Computation, 2007, 11(6):712–731.
- [19] K. Deb, A. Pratap, S. Agarwal, et al. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation, 2002, 6(2):182–197.
- [20] B. Xiao, H. Zhang, H. Wang, Z. Zhang, S. Peng, X. Song. An improved MOEA/D for multi-objective flexible job shop scheduling by considering efficiency and cost. Computers & Operations Research, 2024, 168:106675.
- [21] C. Finn, P. Abbeel, S. Levine. Model-agnostic meta-learning for fast adaptation of deep networks. in: International Conference on Machine Learning. PMLR, 2017:1126–1135.
- [22] J. Beck, R. Vuorio, E. Z. Liu, et al. A survey of meta-reinforcement learning. arXiv preprint arXiv:2301.08028, 2023.
- [23] 谭晓阳, 张哲. 元强化学习综述. 南京航空航天大学学报, 2021, 53(05):653–663.
- [24] X. Y. Jia, S. Gao, W. He. Meta-reinforcement learning-based collision avoidance for autonomous ship. Ocean Engineering, 2025, 339:122064.
- [25] A. Belmonte-Baeza, J. Lee, G. Valsecchi, et al. Meta reinforcement learning for optimal design of legged robots. IEEE Robotics and Automation Letters, 2022, 7(4):12134–12141.
- [26] J. Shin, A. Hakobyan, M. Park, et al. Infusing model predictive control into meta-reinforcement learning for mobile robots in dynamic environments. IEEE Robotics and Automation Letters, 2022, 7(4):10065–10072.

- [27] J. N. D. Gupta, E. A. Tunc. Scheduling a two-stage hybrid flowshop with separable setup and removal times. *European Journal of Operational Research*, 1994, 77:415–428.
- [28] C. Y. Lee, G. L. Vairaktarakis. Minimizing makespan in hybrid flowshops. *Operational Research Letters*, 1994, 16:149–158.
- [29] J. L. Hunsucker, J. R. Shah. Comparative performance analysis of priority rules in a constrained flow-shop with multiple processors environment. *European Journal of Operational Research*, 1994, 72:102–114.
- [30] 方子丞, 李新宇, 高亮. 带负载均衡的混合算法求解分布式异构作业车间调度问题. *控制理论与应用*, 2024, 41(6):977–989.
- [31] C. T. Tseng, C. J. Liao. A particle swarm optimization algorithm for hybrid flow-shop scheduling with multiprocessor tasks. *International Journal of Production Research*, 2008, 46(17):4655–4670.
- [32] C. Wang, P. Jiang, L. Zheng. Deep reinforcement learning in smart manufacturing: A review and prospects. *CIRP Journal of Manufacturing Science and Technology*, 2022, 40:75–88.
- [33] Y. Zhao, Y. Wang, Y. Tan, et al. Dynamic jobshop scheduling algorithm based on deep Q network. *IEEE Access*, 2021, 9:122995–123011.
- [34] Y. Pu, F. Li, S. Rahimifard. Multi-Agent Reinforcement Learning for Job Shop Scheduling in Dynamic Environments. *Sustainability*, 2024, 16(8):3234.
- [35] W. Zhang, F. Zhao, Y. Li, C. Du, X. Feng, X. Mei. A novel collaborative agent reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for flexible job shop scheduling problem. *Journal of Manufacturing Systems*, 2024, 74:329–345.
- [36] S. Luo, L. Zhang, Y. Fan. A deep multi-agent reinforcement learning approach to solve dynamic job shop scheduling problem. *Computers & Operations Research*, 2023, 159:106294.
- [37] 景轩. 基于多智能体深度强化学习的柔性作业车间调度问题研究: [博士学位论文]. 广州: 华南理工大学, 2023.
- [38] R. Chen, B. Yang, S. Li, S. Wang. A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. *Computers & Industrial Engineering*, 2020, 149:106778.
- [39] X. J. Long, J. T. Zhang, X. Qi, et al. A self-learning artificial bee colony algorithm based on reinforcement learning for a flexible job-shop scheduling problem. *Concurrency and Computation: Practice and Experience*, 2022, 34(4):1–20.

- [40] V. Mnih. Playing Atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602, 2013.
- [41] H. Van Hasselt, A. Guez, D. Silver. Deep reinforcement learning with double Q-learning. in: Proceedings of the AAAI Conference on Artificial Intelligence, 2016.
- [42] M. Hessel, J. Modayil, H. Van Hasselt, et al. Rainbow: Combining improvements in deep reinforcement learning. in: Proceedings of the AAAI Conference on Artificial Intelligence, 2018.
- [43] V. Mnih, et al. Asynchronous methods for deep reinforcement learning. arXiv preprint arXiv:1602.01783, 2016.
- [44] P. F. Christiano, J. Leike, T. Brown, et al. Deep reinforcement learning from human preferences. Advances in Neural Information Processing Systems, 2017, 30:1–9.
- [45] 肖鹏飞, 张超勇, 孟磊磊等. 基于深度强化学习的非置换流水车间调度问题. 计算机集成制造系统, 2021, 27(1):192–205.
- [46] 李宝帅, 叶春明. 深度强化学习算法求解作业车间调度问题. 计算机工程与应用, 2021, 57(23):248–254.
- [47] Y. Zhao, H. Zhang. Application of machine learning and rule scheduling in a job-shop production control system. International Journal of Simulation Modelling, 2021, 20(2):410–421.
- [48] 王美林, 吴耿枫, 梁凯晴等. Dueling Double DQN 在基于 MPN 混流制造车间实时调度中的应用. 计算机集成制造系统, 2021, : 1–19.
- [49] 谭远良, 吕佑龙, 左丽玲, 等. 基于强化学习的航天产品装配线投产排序研究. 组合机床与自动化加工技术, 2022, : 160–164.
- [50] W. Song, X. Chen, Q. Li, Z. Cao. Flexible Job-Shop Scheduling via Graph Neural Network and Deep Reinforcement Learning. IEEE Transactions on Industrial Informatics, 2023, 19(2):1600–1610.
- [51] R. Wang, G. Wang, J. Sun, et al. Flexible job shop scheduling via dual attention network-based reinforcement learning. IEEE Transactions on Neural Networks and Learning Systems, 2023.
- [52] Y. Du, J. Li, C. Li, et al. A reinforcement learning approach for flexible job shop scheduling problem with crane transportation and setup times. IEEE Transactions on Neural Networks and Learning Systems, 2022, 35(4):5695–5709.
- [53] Y. Liu, L. Peng, H. Wang, Y. Huang, C. Gu. A literature review on deep reinforcement learning for machine scheduling: Novel formulations and benchmarks. Computers & Industrial Engineering, 2025, 200:110856.

- [54] T. Hospedales, A. Antoniou, P. Micaelli, A. Storkey. Meta-Learning in Neural Networks: A Survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022, 44(9):5149–5169.
- [55] H. Lee, N. T. Vu, S. W. Li. Meta learning and its applications to natural language processing. in: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: Tutorial Abstracts*, 2021:15–20.
- [56] S. Luo, Y. Li, P. Gao, et al. Meta-seg: a survey of meta-learning for image segmentation. *Pattern Recognition*, 2022, 126:108586.
- [57] S. M. Richards, N. Azizan, J. J. Slotine, et al. Control-oriented meta-learning. *The International Journal of Robotics Research*, 2023, 42(10):777–797.
- [58] L. C. Melo. Transformers are meta-reinforcement learners. in: *International Conference on Machine Learning*. PMLR, 2022:15340–15359.
- [59] 黄宁馨, 尹翔, 乐云亮等. 一种基于元学习的改进深度强化学习算法. *扬州大学学报 (自然科学版)*, 2021, 24(03):19–23.
- [60] 邓天翊, 张耕培. 基于元学习和数据增强优化小样本模型泛化性能研究. *现代信息技术*, 2024, 8(08):93–96.
- [61] K. Jin, J. Wang, H. D. Wang, et al. Soft formation control for unmanned surface vehicles under environmental disturbance using multi-task reinforcement learning. *Ocean Engineering*, 2022, 260:112035.
- [62] K. Lee, J. Li, D. Isele, et al. Robust driving policy learning with guided meta reinforcement learning. in: *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2023:4114–4120.
- [63] 沈欢. 基于元学习和强化学习的微电网实时能源调度: [硕士学位论文]. 杭州: 杭州电子科技大学, 2024.
- [64] 王志强. 基于元强化学习的自适应边缘计算任务调度方法研究: [硕士学位论文]. 兰州: 兰州交通大学, 2024.
- [65] 马瑞琳. 基于元强化学习的作业车间调度系统设计与实现: [硕士学位论文]. 哈尔滨: 哈尔滨工业大学, 2025.
- [66] 吴林聪. 基于元强化学习的动态柔性作业车间调度问题研究: [硕士学位论文]. 武汉: 华中农业大学, 2025.
- [67] 姜一啸, 吉卫喜, 何鑫等. 基于改进非支配排序遗传算法的多目标柔性作业车间低碳调度. *中国机械工程*, 2022, 33(21):2564–2577.

- [68] J. Sassi, I. Alaya, P. Borne, et al. A decomposition-based artificial bee colony algorithm for the multi-objective flexible job shop scheduling problem. *Engineering Optimization*, 2022, 54(3):524–538.
- [69] Y. Chen, H. Li, L. Zhang, G. Sun. An Enhanced Multi-Objective Evolutionary Algorithm with Reinforcement Learning for Energy-Efficient Scheduling in the Flexible Job Shop. *Processes*, 2024, 12(9):1976.
- [70] R. Li, W. Gong, C. Lu. A reinforcement learning based RMOEA/D for bi-objective fuzzy flexible job shop scheduling. *Expert Systems with Applications*, 2022, 203:117380–117393.
- [71] X. Jing, X. Yao, M. Liu, J. Zhou. Multi-agent reinforcement learning based on graph convolutional network for flexible job shop scheduling. *Journal of Intelligent Manufacturing*, 2024, 35:75–93.
- [72] 张丽丛. 基于深度强化学习的柔性车间调度方法研究: [博士学位论文]. 大连: 大连理工大学, 2024.
- [73] 张韵. 基于改进深度强化学习的作业车间调度方法研究: [博士学位论文]. 武汉: 华中科技大学, 2022.
- [74] A. Rossi, M. Lanzetta. Scheduling flow lines with buffers by ant colony digraph. *Expert Systems with Applications*, 2013, 40(9):3328–3340.
- [75] 熊福力, 陈思远, 熊宁馨, et al. 基于两阶段混合迭代贪婪算法的分布式异构非置换流水车间调度. *计算机集成制造系统*, 2025, 31(8):2870–2883.
- [76] Z. Liu, J. Wang, C. Zhang, et al. A hybrid genetic-particle swarm algorithm based on multilevel neighbourhood structure for flexible job shop scheduling problem. *Computers & Operations Research*, 2021, 135:1–19.
- [77] 宋存利, 朱建伟, 李金泰. 基于 NSGA-III 算法求解柔性作业车间调度问题. *机电工程技术*, 2024, 53(5):11–15, 85.
- [78] 陈昊鹏. 面向智能工厂的混合流水车间调度算法研究: [硕士学位论文]. 宁波: 宁波大学, 2023.
- [79] R. Cheng, Y. Jin, M. Olhofer, B. Sendhoff. A Reference Vector Guided Evolutionary Algorithm for Many-Objective Optimization. *IEEE Transactions on Evolutionary Computation*, 2016, 20(5):773–791.
- [80] 范英建. 基于改进 MOEA/D 算法的多目标作业车间调度: [硕士学位论文]. 沈阳: 沈阳大学, 2022.

附录 1 攻读硕士学位期间取得的研究成果

专 利

- [1] 一种柔性作业车间鲁棒调度方法。AJ2600370.